



天津大学
Tianjin University

基于动态多轮纠错和大语言模型智能体的 数学文字问题求解研究

Research on Math Word Problem Solving Based on Dynamic Multi-Turn Error Correction and Large Language Model Agent

专业类别： 电子信息

研究领域： 计算机技术

作者姓名： 胡涵清

导师姓名： 王博

企业导师： 赵东明

答辩时间	2025 年 05 月 27 日		
答辩委员会	姓名	职称	工作单位
主席	周智海	研究员	国家海洋技术中心
委员	张小旺	教授	天津大学
	严程	副研究员	天津大学

天津大学智能与计算学部

二〇二五年六月

独创性声明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作和取得的
研究成果，除了文中特别加以标注和致谢之处外，论文中不包含其他人已经发表
或撰写过的研究成果，也不包含为获得 天津大学 或其他教育机构的学位或证
书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中
作了明确的说明并表示了谢意。

学位论文作者签名：胡涵清 签字日期： 2025 年 5 月 20 日

学位论文版权使用授权书

本学位论文作者完全了解 天津大学 有关保留、使用学位论文的规定。
特授权 天津大学 可以将学位论文的全部或部分内容编入有关数据库进行检
索，并采用影印、缩印或扫描等复制手段保存、汇编以供查阅和借阅。同意学校
向国家有关部门或机构送交论文的复印件和磁盘。

（保密的学位论文在解密后适用本授权说明）

学位论文作者签名：胡涵清

导师签名：王

签字日期： 2025 年 5 月 20 日

签字日期： 2025 年 5 月 20 日

摘要

数学文字问题是由自然语言描述的数学问题,也是大语言模型面临的挑战之一。经典方法主要分为自然语言推理和外部工具集成。前者以思维链为代表,其计算结果的正确性难以保证。后者以程序辅助提示为代表,能够利用外部工具提供准确计算,但会出现难以用代码表示的逻辑。最新研究结合思维链和工具集成,并通过多轮对话来改进性能。但这类方法往往基于特定大语言模型或进行微调,从而降低了灵活性和泛化能力。另一类新方法利用基于检索的上下文学习在避免微调的情况下提升模型解决问题的能力,但在检索代码样本时,相似性难以量化,样本检索精度受限。针对上述挑战,本文提出以下创新性工作:

首先,针对微调灵活性低,而非微调上下文学习难以检索相似样本的问题,本文提出基于错误类型检索和多轮程序辅助纠错提示的方法。该方法在每次生成代码出错时,根据错误类型动态检索错误样本进行多轮纠错。其创新性贡献包括:(1) 提出无需微调的多轮交互式纠错框架,实现渐进式代码修正,显著提升复杂数学问题的求解成功率;(2) 设计基于推理路径的层次化错误样本结构,融合自然语言推理、程序代码与执行状态三元组信息,并可自动生成包含错误模式与修正路径的多样化训练样本;(3) 开发结合错误类型匹配与上下文相似度的动态错误样本检索算法,实时分析异常特征,自适应选取相关纠错案例指导代码修复。在 MATH-minival 和 GSM8K 数据集上的实验表明该方法优于现有基线模型。

其次,为在新方法基础上形成更加系统化的解题策略,本文面向数学文字问题创新性地设计了包含感知、记忆和执行模块的自学习智能体:(1) 感知模块:为持续构造高质量数学文字问题,感知模块基于错题样本库,利用大语言模型持续构造数学文字问题,并利用多模型自一致性进行质量控制;(2) 记忆模块:建立针对单轮和多轮交互的样本记忆库,及相应记忆更新策略,通过语义相似度、代码结构差异和错误类型多维比较,有效维护记忆库多样性和准确性;(3) 执行模块:提出基于记忆样本动态检索的多策略融合方法,在解题阶段灵活选择多轮或单轮交互,以及相似度匹配或错误类型分析策略,利用历史经验辅助当前问题求解。实验中,该智能体不仅在记忆库维护上呈现更好的质量和增长性,并且在 MATH-minival 和 GSM8K 数据集上较之基线方法解题准确性更高。

关键词: 数学文字问题, 大语言模型, 基于检索的上下文学习, 智能体

ABSTRACT

Math word problem is a mathematical problem described by natural language, and it is also one of the challenges faced by large language models. Classical approaches are mainly divided into natural language inference and external tool integration. The former is represented by the chain of thought, and the correctness of its calculation results is difficult to guarantee. The latter is represented by program assistance hints, which can use external tools to provide accurate calculations, but may encounter logic that is difficult to express in code. Recent research combines chain of thought and tool integration, and uses multiple rounds of dialogue to improve performance. However, these methods are often based on a specific large language model or fine-tuning, which reduces the flexibility and generalization ability. Another class of new methods uses retrieval-based contextual learning to improve the model’s ability to solve problems without fine-tuning. However, when retrieving code samples, the similarity is difficult to quantify, and the sample retrieval accuracy is limited. In response to the above challenges, this thesis proposes the following innovative work:

Firstly, aiming at the low flexibility of fine-tuning, but the difficulty of retrieving similar samples without fine-tuning context learning, this thesis proposes a method based on error type retrieval and multi-round procedure assisted error correction suggestion. When there is an error in the generated code, the method dynamically retrieves the error samples for multiple rounds of error correction according to the error type. His innovative contributions include: (1) a multi-round interactive error correction framework without fine-tuning is proposed to achieve progressive code correction, which significantly improves the success rate of solving complex mathematical problems; (2) A hierarchical error sample structure based on inference paths is designed, which integrates the triplet information of natural language inference, program code and execution state, and can automatically generate diverse training samples containing error patterns and correction paths. (3) A dynamic error sample retrieval algorithm combining error type matching and context similarity is developed to analyze abnormal features in real time and adaptively select relevant error correction cases to guide code repair. Experiments on MATH-minival and GSM8K datasets show that the proposed method is superior to the existing baseline models.

Secondly, in order to form a more systematic problem-solving strategy on the basis of the new method, this thesis innovatively designs a self-learning agent including perception, memory and execution modules for math word problems: (1) Perception module: In order to continuously construct high-quality math word problems, the perception module uses a large language model to continuously construct math word problems based on the error sample database, and uses multi-model self-consistency for quality control. (2) Memory module: The sample memory database for single-round and multi-round interactions is established, and the corresponding memory update strategy is established. The diversity and accuracy of the memory database are effectively maintained by multi-dimensional comparison of semantic similarity, code structure differences and error types. (3) Execution module: a multi-strategy fusion method based on dynamic retrieval of memory samples is proposed, which flexibly selects multi-round or single-round interactions in the problem-solving stage, and similarity matching or error type analysis strategies, and uses historical experience to assist the current problem solving. In the experiments, the agent not only shows better quality and growth in memory maintenance, but also has higher accuracy than the baseline method on MATH-minival and GSM8K datasets.

KEY WORDS: Math Word Problems, Large Language Model, Retrieval-based In-Context Learning, Agent

目 录

摘 要	I
ABSTRACT	III
第 1 章 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	3
1.2.1 基于大语言模型和思维链求解数学问题	3
1.2.2 基于大语言模型和工具集成求解数学问题	5
1.2.3 基于大语言模型智能体求解数学问题	6
1.3 论文研究内容	7
1.4 论文组织结构	9
第 2 章 相关理论与技术	11
2.1 面向大语言模型的提示工程	11
2.1.1 上下文学习	12
2.1.2 思维链	13
2.1.3 工具集成	16
2.2 基于大语言模型的智能体	17
2.2.1 智能体的概念与定义	17
2.2.2 智能体的架构	17
2.2.3 智能体的相关应用	19
2.3 本章小结	20
第 3 章 基于错误类型检索和多轮程序辅助纠错的数学文字问题求解	21
3.1 引言	21
3.2 问题定义	23
3.2.1 问题定义	23
3.3 方法介绍	24
3.3.1 多轮程序辅助纠错方法概览	24
3.3.2 错题样本的结构	24
3.3.3 错题样本的生成	27
3.3.4 基于错题样本的动态错误类型检索	30

3.4	实验与结果	33
3.4.1	数据集介绍	33
3.4.2	评价方法	33
3.4.3	基线方法	34
3.4.4	实验设置	35
3.4.5	预实验	35
3.4.6	实验结果	36
3.4.7	消融实验	37
3.4.8	错误类型分布统计与分析	38
3.4.9	案例分析	41
3.5	本章小结	41
第 4 章	基于自学习大语言模型智能体的数学文字问题求解	43
4.1	引言	43
4.2	问题定义和方法概览	45
4.2.1	问题定义	45
4.2.2	整体架构	45
4.3	自学习智能体框架设计	46
4.3.1	智能体感知模块：题目生成与验证	46
4.3.2	智能体执行模块：解题策略选择与自动做题系统	50
4.3.3	智能体记忆模块：成功解题的记录	51
4.4	实验及结果	54
4.4.1	实验设置	54
4.4.2	预实验	56
4.4.3	实验结果	57
4.4.4	案例分析	60
4.5	讨论	66
4.5.1	自学习智能体框架的优势和局限性	66
4.5.2	潜在的改进方向和扩展应用	66
4.6	本章小结	67
第 5 章	总结与展望	69
5.1	论文工作总结	69
5.2	未来研究展望	70
	参考文献	71
	发表论文和参加科研情况说明	79
	致 谢	81

第1章 绪论

1.1 研究背景与意义

近年来,人工智能技术,尤其是大语言模型(如 GPT-3.5^[1]、GPT-4^[2,3]等)在自然语言处理和人工智能领域取得了突破性进展。这些模型展现出了惊人的语言理解和生成能力,不仅在日常对话和文本创作中表现出色,还在各种复杂任务中显示出强大的潜力,比如语言生成、文本分类、机器翻译、语义理解等,但目前大语言模型尚未有足够成熟的应用落地。这很大程度上是因为模型的规划能力、逻辑推理能力^[3]不足和幻觉^[4](Hallucination)的影响。而数学问题的求解不仅需要语言理解能力,还要求模型具备逻辑推理、抽象思维和精确计算的能力,这使得数学问题的研究成为提升和测试大语言模型全面认知能力的理想场景^[5-7]。数学问题是一个比较宽泛的概念,包含各种类型,例如算术、数学文字问题^[8-10](Math Word Problem)、几何^[11]、自动定理证明和多模态数学问题^[8,12]等。

本文聚焦于数学文字问题的求解,数学文字问题是数学问题的一种,通常以文字形式呈现,也叫数学应用题,常见于小学、初中、高中等教育场景。这种形式的问题要求解题者深入理解用文字表述的题目信息,识别相关的数学概念,并构建适当的方程式或表达式来解决问题^[7,13]。在结构上,每个数学文字问题都包含一个核心问题 Q 和与之对应的最终答案 A ,这构成了最基本的“问题-答案”对。然而,根据提供的额外信息,数学文字问题可能呈现出不同的变体。例如,有些题目可能包含解题所需的方程式 E ,形成“问题-方程-答案”的结构;而另一些则可能提供详细的解题步骤或推理过程 R ,构成“问题-推理-答案”的形式。

数学文字问题的形式非常适合被大语言模型处理,因为这些问题通常以自然语言描述,包含了问题背景、已知条件和待求解的问题,这与大语言模型擅长处理的文本输入格式高度契合。此外,数学文字问题的难度通常介于简单的算术运算和高度抽象的数学证明之间,为大语言模型提供了一个理想的挑战水平^[14]。这种中等难度不仅能够测试模型的语言理解能力,还能评估其逻辑推理和问题解决能力,同时又不会过于复杂以至于完全超出模型的能力范围。因此,数学文字问题成为了评估和改进大语言模型认知能力的理想测试场景。

在这一背景下,研究者们开始探索各种方法来增强大语言模型在数学文字问题求解方面的能力。这些方法包括但不限于:设计更有效的提示策略^[15-19]、结合外部工具和知识库^[20-23]、改进模型的训练数据和方法^[24-27]等。这些努力不仅

旨在提高模型在数学领域的表现，还有望为人工智能在其他需要精确推理和问题解决能力的领域带来启示和进展。这些方法虽然使得大语言模型在数学文字问题求解方面取得了显著进步，但是仍存在诸多挑战^[3,14]。

基于大语言模型解决数学文字问题的基本挑战如下：

1. 推理与计算能力的整合不足：以思维链^[15]（Chain of Thought）为代表的自然语言推理方法在解决数学问题时，虽然能够显著提高模型的推理能力，但仍然存在计算能力较弱以及受制于数据规模等问题。

2. 代码生成与纠错机制不完善：结合外部工具的方法，主要是利用 Python 解释器运行模型生成的代码。虽然能够弥补模型在计算上的不足，但在解决复杂数学问题时，无法像在简单数据集上那样轻松地生成可执行的代码，缺乏有效的错误诊断和纠正策略。

3. 检索策略单一：使用基于检索的上下文学习时，难以检索包含代码的相似演示样本，现有检索的方法有语义相似度和结构相似度，而基于结构相似度的研究较少，这通常是由于在许多任务中提取查询结构时存在挑战^[28]。

为了克服这些挑战，研究者们开始探索更加灵活和智能的方法。其中，基于智能体^[29]（Agent）的方法因其自主性、适应性和持续学习能力而受到广泛关注。智能体方法通过引入记忆增强、任务分解等技术，展现出在复杂任务中的潜力。在数学文字问题求解中，智能体可以利用其自主性在推理与计算之间动态切换，通过任务分解将复杂问题拆解为多个子任务，并调用外部工具进行计算。同时，智能体的适应性使其能够根据问题的复杂性调整策略，而持续学习能力则帮助其在解决新问题时不断优化自身表现。然而，智能体方法仍面临一些挑战。

基于智能体解决数学文字问题的基本挑战如下：

1. 记忆机制不完善：缺乏有效的记忆结构来存储和利用过往解题经验，导致无法从历史错误中学习。且缺乏高质量数学文字问题，导致无法持续构造高质量数学文字问题。

2. 缺乏系统性纠错能力：现有智能体在遇到推理或计算错误时，纠错往往是临时性的，缺乏系统化的纠错策略。

上述挑战的核心原因可以分析如下：

1. 代码生成与纠错机制不完善：现有方法系统化的纠错能力不足，主要因为模型的反馈机制不完善。在推理或计算过程中，模型通常只能被动调整策略，而无法主动检测和纠正错误。这种临时性的纠错方式不仅效率低下，还可能导致错误的传播和累积。

2. 检索策略的单一：使用基于检索的上下文学习时，难以检索包含代码的相似演示样本，现有检索的方法有语义相似度和结构相似度，而由于代码的结构相似度难以表示，基于结构相似度进行的检索存在挑战。

3. 记忆机制的局限性：智能体的记忆机制不完善，主要体现在无法有效存储和利用过往解题经验。传统的记忆结构通常只能存储静态信息，而无法动态关联推理路径和计算结果。这种局限性使得模型在面对类似问题时，无法从历史错误中学习，也无法复用已有的解题策略。并且记忆的基础，高质量的数学文字问题规模有限，如何持续创建高质量的数学文字问题存在挑战。

4. 智能体架构的局限性：智能体方法的潜力受限于其架构设计。例如，记忆增强和任务分解技术虽然能提升模型的适应性，但缺乏对推理与计算的深度整合。此外，模型的持续学习能力不足，导致其在面对新问题时无法快速调整策略。

针对上述挑战，本文提出以下创新性的研究思路：

1. 提出了一种使用多轮对话进行错误修正，并利用提前建立的错题样本库进行基于检索的上下文学习从而提高大语言模型解决数学文字问题的方法。具体而言，通过上下文学习的方式，在样本中结合思维链和程序辅助提示，规范化大语言模型的输出，使其结合推理与计算。随后利用手动构造的少量固定样本，通过运行代码后返回的代码执行错误类型信息，反馈给大语言模型进行多轮修正，建立反馈和纠错机制。利用这种方式基于 MATH 数据集的训练集构造出错题样本库，并通过同时检索问题语义相似度和错误类型的方式解决数据与检索策略的局限性的问题。

2. 在上述方法基础上，为进一步解决智能体架构的局限性，形成更加系统化的解题策略，建立了自主选择解题策略的智能体架构，分为感知，执行，记忆三大模块。具体而言，为持续构造高质量数学文字问题，解决数据的局限性的挑战，感知模块基于原有错题样本库，利用大语言模型的生成能力持续构造数学文字问题，并利用多模型自一致性进行质量控制。执行模块使用策略选择机制，根据对话阶段的不同，灵活选择不同的检索策略。记忆模块通过构建单次样本库和多次样本库，以及相应的记忆更新策略，通过语义相似度、代码结构差异和错误类型等多维比较，有效维护记忆库的多样性和准确性，实现“错题-分析-修正-强化”的闭环学习，使得智能体的记忆机制的局限性得到缓解。

1.2 国内外研究现状

1.2.1 基于大语言模型和思维链求解数学问题

思维链是 Wei 等人^[15]提出的一种基于上下文学习的方法，通过少量样本引导模型生成一系列中间推理步骤，显著提升了模型在复杂数学问题上的表现。该方法使模型能够逐步推理，分解问题，从而更有效地寻找解决方案。尽管该方法主要在数学问题上进行了验证，但其基本原理可推广至其他需要复杂推理的领

域，如科学研究、法律推理等，为跨领域应用开辟了广阔前景。研究表明，随着语言模型规模的增加，思维链提示方法的效果愈发显著，这意味着在更大规模的语言模型中，思维链方法能够发挥更强的推理能力增强作用。

这种引导模型生成中间推理步骤的方法可追溯至谷歌 Brain 团队 Nye 等人^[30]的研究，他们最早提出使用包含中间步骤的示例来引导模型进行“step by step”推理。随后，零样本思维链^[31] (Zero Shot Chain of Thought) 仅通过简单提示词“Let’s think step by step”即使大型语言模型在无任务特定示例情况下进行复杂多步推理，展现出零样本推理能力。该研究表明大语言模型不仅是优秀的少样本学习者，还能在缺乏明确示例的情况下解决复杂任务，扩展了大语言模型在需要复杂推理任务中的应用范围。进一步地，Google DeepMind 的 Yang 等人^[32]探索出更多有效增强模型推理能力的提示词，其中最有效的为“Take a deep breath and work on this problem step-by-step”。他们提出 OPRO (Optimization by PROMpting) 方法，其核心思路类似于自举 (bootstrapping)，即在给定初始提示和目标函数情况下，利用大型语言模型作为优化器，根据最终效果持续改进提示，该方法相较于人工逐步尝试，具有更高的效率和效果。

在思维链方法的成功基础上，衍生出多种改进版本，包括自一致性思维链^[19] (CoT-Self Consistency, Cot-SC)、思维树^[33] (Tree of Thoughts, ToT) 和思维图^[34] (Graph of Thoughts, GoT) 等。

自一致性思维链通过采用自一致性方法^[19]对不同推理路径的结果进行筛选，优化传统的贪婪解码策略^[35]。该策略显著提升了大型语言模型在复杂推理任务中的表现，原因在于其能够更有效地捕捉和利用多样化的推理路径，从而获得最一致且可靠的答案。此外，该方法易于集成到各类推理任务中，具有较高的实用性和有效性。

思维树框架受人类大脑解决复杂推理任务的启发，通过试错法探索解决方案空间，采用树状思维过程允许在必要时进行回溯。该框架模拟人类解决复杂问题时的试错过程，使模型能够在探索不同解决方案路径时进行回溯和调整，从而寻找最佳解决方案。通过引导大语言模型采用树状结构进行推理，该框架显著提高了模型处理复杂问题时的推理能力。

思维图框架超越了思维链和思维树等现有方法，将大语言模型生成的信息建模为任意图结构，其中信息单元 (“大语言模型思维”) 作为顶点，顶点间的依赖关系作为边。此方法允许将任意大语言模型思维组合形成复杂推理路径，提高了模型处理复杂问题时的灵活性和有效性。该框架在多种复杂问题的解决中展示了其优势，证明将信息建模为图结构能显著增强大语言模型的推理能力和问题解决能力。

最少到最多提示过程^[17] (Least to Most prompting, LtM) 采用类似思维链的

策略，但引入更细致的问题分解和递进式解决方案。该方法首先将复杂问题拆解为一系列相互关联的子问题，然后逐步解决。与传统思维链不同，LtM 在解决每个后续子问题时，将先前子问题的解决方案纳入提示中，为模型提供更丰富的上下文信息。这种渐进式方法不仅有助于模型更全面地理解问题结构，还能在解决过程中不断积累和利用已获得的信息，提高解决复杂问题的效率和准确性。

复杂思维链^[18] (Complex CoT) 基于相似度和复杂度的检索器，能提高特定数学推理任务的性能。该方法利用问题长度或推理步骤数来度量复杂度，在提示中加入相似度较高的推理路径，从而提高模型在复杂数学问题上的表现。这启发了本研究采用基于检索的少样本学习方法，在提示中纳入相似度较高的推理路径，以提升模型在复杂数学问题上的表现。

1.2.2 基于大语言模型和工具集成求解数学问题

鉴于大语言模型在计算能力方面的局限性，研究者在思维链基础上开始探索将外部工具与模型集成，以增强其解决复杂数学问题的能力。

程序辅助提示^[20] (Program-aided Language, PAL) 通过结合程序辅助方式，显著提升了语言模型在复杂推理任务中的表现。该方法使模型生成并执行程序来解决问题，由于程序能够明确定义步骤和逻辑，PAL 方法有效减少了语言模型在推理过程中的模糊性和错误，提高了回答的准确性。与之相似的思维程序^[36] (Program of Thoughts Prompting, PoT) 同样利用代码表达推理逻辑，强调推理与计算的分离。程序辅助方式不仅帮助模型更好地理解问题结构和逻辑，还能在未来的推理任务中提升其表现。

LPML^[37] (LLM-Prompting Markup Language) 采用类似 XML 的标记语言，对推理过程进行结构化表示。这种结构化表示使语言模型能更清晰地理解和执行各推理步骤，减少推理过程中的错误。LPML 将思维链方法与外部工具（如 Python REPL）相结合，使语言模型能调用这些工具进行复杂计算，不仅提升了模型的计算能力，还保证了推理过程的准确性。与 PAL 方法相比，LPML 的显著区别在于其非单轮特性，能将模型输出结果再次输入模型，实现逐步完善的效果。

然而，PAL 等代码转换方法主要适用于解决简单的程序性问题，对需要声明性推理^[38] (declarative reasoning) 的复杂问题效果欠佳。He-Yueya 等人^[39] 提出结合语言模型和符号求解器的方法，克服了 PAL 方法的局限性。通过集成符号求解器，该方法能更有效地处理复杂数学推理任务，提升推理的准确性和灵活性。结合符号求解器的语言模型支持多轮推理，将每步结果再次输入模型，逐步完善推理过程，从而更准确地解决复杂问题。

除了使用 Python 运行代码外，部分研究采用 Wolfram^[21] 等外部求解器，但使用方式大同小异。值得注意的是，在解决 MATH 数据集^[24] 等高难度数学问题

时,程序辅助提示难以像在简单数据集上那样生成可执行代码,导致其表现可能不如直接使用思维链方法。

尽管如此,程序辅助提示在解决简单数学问题上展现了卓越表现。本研究受 LPML 等利用多轮交互纠正错误方法的启发,旨在改进程序辅助提示在复杂数学问题上的表现。

1.2.3 基于大语言模型智能体求解数学问题

尽管思维链方法在提升大语言模型推理能力方面取得了显著成效,但仍存在若干局限性。首先,思维链方法高度依赖示例的质量和相关性,不恰当的示例可能误导模型,导致推理错误;其次,在处理需要精确计算或复杂逻辑的问题时,思维链方法可能因累积误差而产生不准确结果;最后,思维链方法通常是一次性推理过程,缺乏对推理结果的自我验证和修正机制。

为克服这些局限性,研究者开始探索更具灵活性和智能性的方法。其中,基于智能体^[29] (Agent) 的方法因其自主性、适应性和持续学习能力而受到广泛关注。智能体能够在复杂环境中执行任务并做出决策,在数学问题求解领域的应用主要体现在以下几个方面:

1. 工具使用能力: ToRA (Tool-Integrated Reasoning Agent)^[22] 智能体不仅能使用计算器、Python 解释器等基本工具,还能调用 Wolfram Alpha 等专业数学工具,显著增强解决复杂数学问题的能力。

2. 自反思与迭代能力: MathChat^[40] 引入自反思机制,使智能体能在解题过程中进行多轮推理和修正。该方法模拟人类解题时的思考过程,能持续优化解题策略。

3. 知识整合与推理能力: MathSensei^[21] 提出结合大语言模型和 Wolfram Alpha 等外部工具的方法,通过多步推理和知识整合解决复杂数学问题。该方法虽未直接使用知识图谱,但通过有效整合多种知识源和推理步骤,提高了解题的准确性和可解释性。

值得注意的是,智能体在解决数学问题上实质上是对思维链和工具集成的综合应用与拓展,与本研究的思路高度契合。智能体方法通过整合多种技术和工具,展现了解决复杂数学问题的潜力。然而,现有智能体方法仍存在若干局限性:一方面,过度强调问题分解和规划,而在复杂数学问题面前,大语言模型的任务分解能力有限,影响有效解决问题;另一方面,当前智能体普遍忽视记忆模块的构建,主要依赖模型本身能力,导致无法有效利用过往解题经验。

1.3 论文研究内容

针对大语言模型和智能体解决数学文字问题中出现的，包括推理与计算的分离、反馈与纠错机制的缺失、检索策略的单一、数据的局限性、记忆机制的局限性和智能体架构的局限性在内的挑战，本文的研究内容及创新性贡献如下：

本文主要研究了如何利用基于检索的上下文学习（动态少样本学习），对结合思维链推理和程序辅助提示解决数学文字问题过程中产生的运行错误进行修正。并且在此基础上，提出了一个包含感知、记忆和执行三大模块的智能体架构。具体来说，本文的研究内容如图1-1所示：

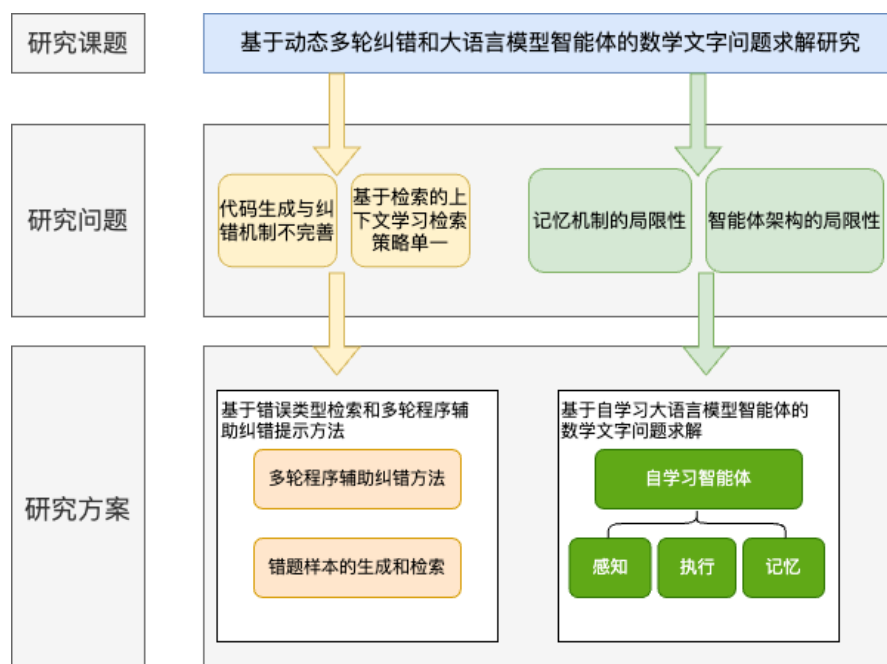


图 1-1 论文研究内容

1. 基于错误类型检索和多轮程序辅助纠错的数学文字问题求解

在基于大模型解决数学文字问题的任务上，现有通过预训练和微调模型进行改进的方式需要大量的计算资源和数据，而现有不依赖微调的方法面临着推理与计算分离，反馈与纠错机制缺失的挑战，使用基于检索的上下文学习事还面临着检索策略单一的局限性，为缓解上述挑战和局限，本文在第三章提出基于错误类型检索和多轮程序辅助纠错方法，旨在避免微调模型的情况下，通过结合基于检索的上下文学习和多轮对话纠错提高大语言模型在解决数学文字问题上的表现。该方法的创新性贡献包括：

(1) 多轮程序辅助纠错框架：提出无需模型微调的多轮交互式纠错框架，通过动态调用 Python 解释器验证执行结果并检测错误，实现渐进式代码修正，显著提升复杂数学问题的求解成功率；

(2) 结构化样本生成：设计基于推理路径的层次化样本结构，每个节点融合自然语言推理、程序代码与执行状态三元组信息，并利用公开数据集自动生成包含错误模式与修正路径的多样化训练样本；

(3) 动态错误类型检索机制：开发结合错误类型匹配与上下文相似度的动态检索算法，通过实时分析程序异常特征，从样本库中自适应选取最相关的纠错案例指导模型修复代码逻辑。

(4) 在 MATH-minival 和 GSM8K 数据集上的实验结果表明，该方法在解决数学文字问题上的准确率较基线模型提升 1.4%-7.5%，验证了错误类型检索和多轮程序辅助纠错方法的有效性。

2. 基于自学习大语言模型智能体的数学文字问题求解

为进一步提升大语言模型在数学文字问题的表现，在上述方法基础上，本文在第四章提出基于自学习大语言模型智能体。为缓解智能体架构的局限性，形成更加系统化的解题策略，建立了自主选择解题策略的智能体架构，分为感知，执行，记忆三大模块。为持续构造高质量数学文字问题，解决数据的局限性的挑战，感知模块基于原有错题样本库，利用大语言模型的生成能力持续构造数学文字问题，并利用多模型自一致性进行质量控制。执行模块使用策略选择机制，根据对话阶段的不同，灵活选择不同的检索策略。记忆模块通过构建单次样本库和多次样本库，实现”错题-分析-修正-强化”的闭环学习，使得智能体的记忆机制的局限性得到缓解。该方法的创新性贡献包括：

(1) 为进一步提升大语言模型在数学文字问题的表现，在第三章方法的基础上，为缓解智能体架构的局限性，形成更加系统化的解题策略，建立了自主选择解题策略的智能体架构，分为感知，执行，记忆三大模块。

- 感知模块：为持续构造高质量数学文字问题，解决数据的局限性的挑战，感知模块基于原有错题样本库，利用大语言模型的生成能力持续构造数学文字问题，并利用多模型自一致性进行质量控制。
- 记忆模块：记忆模块通过构建单次样本库和多次样本库，实现”错题-分析-修正-强化”的闭环学习，使得智能体的记忆机制的局限性得到缓解。
- 执行模块：提出了基于样本动态检索的多策略融合方法，在解题阶段灵活选择多轮或单轮交互，以及相似度匹配或错误类型分析等策略，利用历史解题经验辅助当前问题的求解。

(2) 在实验中，该智能体不仅在持续构造高质量数学文字问题上表现出更好的质量和增长性，有效缓解了数据的局限性和智能体的记忆机制的局限性，并且较之基线方法，在 MATH-minival 和 GSM8K 数据集上具有更高的解题准确性。

1.4 论文组织结构

本文共分为 5 个章节，每个章节的主要内容如下：

第一章引言。本章开篇概述了利用提示工程技术提升大型语言模型数学问题解决能力的研究背景和重要性。接着，详细探讨了大型语言模型和基于在数学问题求解领域的应用现状和存在的局限，突出了当前研究面临的主要困难。最后，简明扼要地概括了本论文的核心研究内容。

第二章相关工作概述。本章全面回顾了与本研究密切相关的学术成果，深入探讨了提示工程技术在优化大型语言模型性能方面的应用。同时，本章也批判性地分析了现有方法的局限性，并明确阐述了本文研究的创新之处。

第三章基于错误类型检索和多轮程序辅助纠错的数学文字问题求解。本章详细介绍了一种新的数学文字问题求解方法，该方法结合了思维链和程序辅助学习，并引入了错题样本的检索策略。章节内容包括多轮程序辅助纠错框架、结构化样本生成、动态错误类型检索机制，以及通过一系列实验验证方法的有效性。

第四章基于自学习大语言模型智能体的数学文字问题求解。本章提出了一个创新的自学习智能体框架，旨在克服第 3 章方法和现有智能体的局限性。章节详细描述了框架的设计，包括感知、执行和记忆模块。通过实验和案例分析，展示了该框架在提升数学文字问题求解能力方面的优势。

第五章结论与未来展望。本章对整个研究进行了全面总结和深入讨论，客观指出了本研究存在的不足，并对未来可能的研究方向提出了展望。

第 2 章 相关理论与技术

2.1 面向大语言模型的提示工程

随着 GPT-3 等大规模语言模型的出现，研究者们发现模型的性能可以通过精心设计的输入得到显著提升。这一发现最早可以追溯到 Brown 等人^[41]在 2020 年发表的 GPT-3 论文中。他们观察到，通过在输入中包含少量示例^[42]（Few-Shot Learning），模型能够快速适应新任务，而无需进行微调。

在大语言模型的应用中，提示（Prompt）指的是提供给模型的输入，用以引导其生成输出，结构如图 2-1 所示。

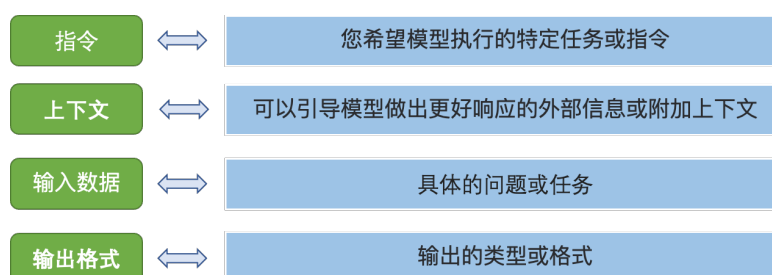


图 2-1 提示的结构^[43]

这个概念的广为人知是由于 Stable Diffusion^[44]等文本生成图像模型的爆火，特定的商业图片可能需要复杂和专业的提示词来确保生成的图像符合预期。通过精心设计和优化提示，可以帮助模型更准确地理解用户的需求，从而生成更符合要求的图像。甚至诞生了专门的提示工程师（Prompt Engineer）这一职业，专门负责设计有效的提示词。提示的不同会带来模型表现的如此大的差异，这便产生了对提示选择和设计的更深入地研究和探索的需求，于是提示工程^[16,41,43,45,46]（Prompt Engineering）便成了近年来在人工智能领域中备受关注的一项技术，它通过设计特定的提示或指令，引导大语言模型生成期望的输出。

在 2022 年末，随着 ChatGPT^[47]的出现，提示工程这一概念进一步得到了推广和发展。实证研究表明，基础模型在特定任务上的表现可能会受到提示的显著影响，变化的幅度往往令人惊讶。例如，最近的研究显示，在 GSM8K^[8] 基准数据集上，模型的表现可以在不改变其参数的情况下波动超过 10%^[32]。这些大语言模型的性能高度依赖于用户输入的质量，促使研究者和实践者更加关注如何设计有效的提示以获得最佳结果。因此，提示工程逐渐发展成为一个独立的研究

方向，涵盖了提示设计、优化和评估等多个方面。

在提示工程的众多技术^[46]中，上下文学习和思维链是两种特别值得关注的方法。这两种方法不仅在理论上具有创新性，而且在实际应用中展现出了显著的效果。接下来，本章将深入探讨这两种方法的原理、特点及其在提升大语言模型性能方面的应用。

2.1.1 上下文学习

上下文学习^[48-51] (In-Context Learning, ICL) 是一种新兴的机器学习范式，源于大规模语言模型 (Large Language Models, LLMs) 的发展。这一概念最初由 Brown 等人^[41] 在 GPT-3 模型中提出，旨在解决传统机器学习方法在面对新任务时需要大量标注数据和计算资源的问题。ICL 通过在推理过程中利用少量示例来执行新任务，而无需显式的参数更新。这种方法的核心在于模型能够在推理阶段理解和利用提供的上下文信息来完成特定任务，从而实现快速适应和泛化。ICL 的出现标志着机器学习范式从“预训练-微调”向更灵活、更高效的方向发展，为解决小样本学习和任务适应性问题提供了新的思路。

上下文学习的定义为：给定一个查询输入文本 x 和一组候选答案 $Y = \{y_1, \dots, y_m\}$ ，预训练语言模型 M 选择得分最高的候选答案作为预测结果，条件是有一个演示集 C 。演示集 C 包含一个可选的任务说明 I 以及 k 个演示示例，因此 $C = \{I, s(x_1, y_1), \dots, s(x_k, y_k)\}$ 或 $C = \{s'(x_1, y_1, I), \dots, s'(x_k, y_k, I)\}$ ，其中 $s'(x_i, y_i, I)$ 是根据任务用自然语言编写的示例。候选答案 y_j 的可能性由整个输入序列的评分函数 f 决定：

$$P(y_j|x) \equiv f_M(y_j, C, x) \quad (2-1)$$

最终预测的标签 $\hat{y}$ 是具有最高概率的候选答案：

$$\hat{y} = \arg \max_{y_j \in Y} P(y_j|x) \quad (2-2)$$

ICL 的优势在于其灵活性和高效性。在传统的机器学习方法中，模型需要通过大量的训练数据和反复的参数更新来学习新任务。而 ICL 则通过少量示例的提示，直接在推理过程中完成学习，这大大减少了训练时间和计算资源的消耗。Dong 等人^[50] 详细阐述了 ICL 的基本原理和应用场景，指出 ICL 能够在无需重新训练模型的情况下，通过少量的示例完成新任务的学习，这为模型的泛化能力提供了新的可能性。Min 等人^[52] 进一步探讨了 ICL 的潜力，提出了通过在训练过程中显式地进行上下文学习来增强模型的表现。他们的研究表明，通过在训练阶段加入上下文示例，模型能够更好地理解和利用上下文信息，从而在推理阶段表现出更强的学习能力。此外，Rubin 等人^[53] 提出了一种基于提示检索的方法，通过使用语言模型自身作为评分函数来检索适合的提示，从而提高 ICL 的效果。这

种方法不仅提高了提示的质量，还增强了模型在不同任务中的表现。Min 等人^[54]对 ICL 的机制进行了深入分析。他们指出，ICL 的关键在于示例的选择和排列方式，这直接影响了模型的学习效果。通过实验，他们发现示例的顺序和相关性在 ICL 中起到了至关重要的作用。最后，Olsson 等人^[55]研究了 ICL 在大语言模型中的应用，发现 ICL 能够显著提高模型的学习能力，特别是在处理复杂任务时表现出色。他们的研究揭示了 ICL 在大语言模型中的潜力，并为未来的研究提供了新的方向。

综上所述，上下文学习作为一种新兴的机器学习范式，通过在推理过程中利用少量示例来完成新任务，展现出了巨大的潜力和广泛的应用前景。相关研究表明，通过优化提示的选择和排列方式，ICL 能够显著提高模型的学习和泛化性能。

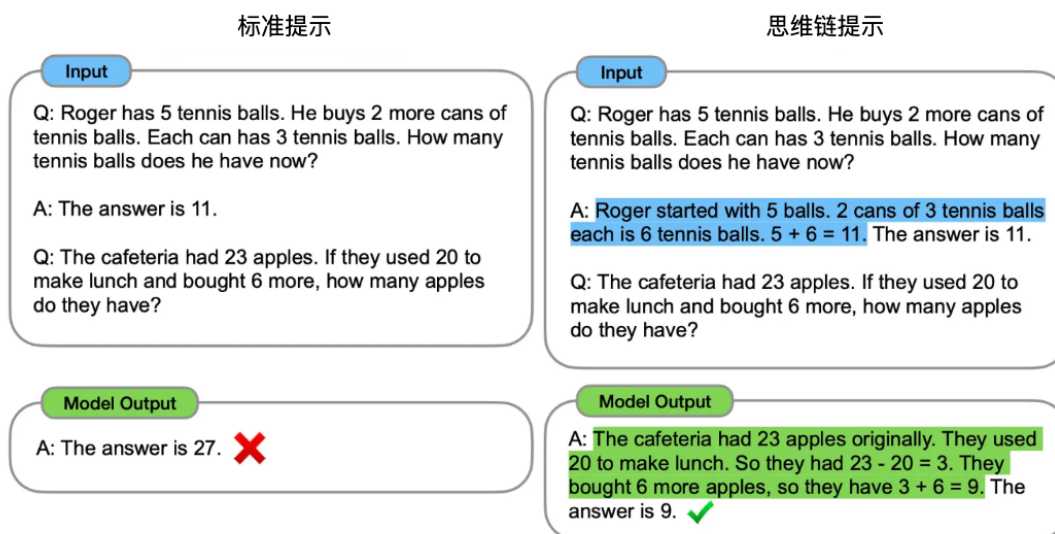
本文的方法主要基于动态检索的上下文学习，通过构建包含错误和纠错过程的错题样本库，为模型提供更加适合的上下文信息，从而提高其在数学文字问题求解中的表现。

2.1.2 思维链

尽管传统的少样本提示方法在提升大语言模型处理分类、翻译和生成等任务的能力方面取得了显著成效，但在面对需要更复杂认知能力的任务时，如算术运算、常识推理和符号逻辑等，其效果却不尽如人意。Wei 等人^[15]提出了思维链方法 (Chain-of-Thought, CoT)，通过在示例样本中引入中间步骤，显著提高了大语言模型在复杂任务上的表现。基础的思维链如图 2-2 所示，本质上也是一种上下文学习，关键区别在于样本中含有解决问题的步骤。这一方法在近年来的自然语言处理和人工智能研究中得到了广泛关注和应用。CoT 方法的核心思想是通过分解复杂问题为一系列相互关联的子问题，从而逐步推导出最终答案。这种方法不仅提高了模型的推理能力，还增强了其可解释性。

思维链方法的基本概念源于人类在解决复杂问题时的思维过程。人类通常会将一个复杂的问题分解为若干个较小的子问题，并通过逻辑推理逐步解决这些子问题，最终得出整体答案。CoT 方法试图模拟这一过程，通过在模型中引入逻辑链，使其能够更有效地处理复杂的推理任务。具体来说，CoT 方法通常包括以下几个关键点：

1. 示例构建：为目标任务创建包含详细推理步骤的示例。这些示例展示了从问题到答案的完整思考过程。
2. 提示设计：将构建的示例作为提示的一部分，与目标问题一起提供给模型。这种设计旨在引导模型模仿示例中的推理模式。
3. 推理生成：模型基于给定的提示和问题，生成包含中间推理步骤的回答。这个过程中，模型会自然地展现出类似人类的思维链。

图 2-2 思维链方法 (CoT) 的示意图^[15]

4. 答案提取：从模型生成的完整推理过程中，提取出最终答案。

这种方法利用了大型语言模型的模仿学习能力，使其能够在复杂任务中展现出更强的推理能力。思维链方法在多个领域中得到了成功应用，特别是在需要复杂推理的任务中。例如，在数学题解答、逻辑推理、常识问答等任务中，CoT 方法显著提升了模型的性能。以下是几个具体的应用场景。数学题解答：在数学题解答任务中，CoT 方法通过将复杂的数学问题分解为若干个基本运算步骤，使模型能够逐步推导出正确答案，例如，Wei 等人^[15]在他们的研究中展示了 CoT 方法在数学题解答中的应用，显著提高了模型的解题准确率。逻辑推理：在逻辑推理任务中，CoT 方法通过模拟人类的逻辑思维过程，使模型能够更好地理解和推理复杂的逻辑关系，例如，Nye 等人^[30]在他们的研究中展示了 CoT 方法在逻辑推理任务中的应用，显著提升了模型的推理能力。常识问答：在常识问答任务中，CoT 方法通过将复杂的常识问题分解为若干个子问题，使模型能够更准确地推导出答案，例如，Kojima 等人^[31]在他们的研究中展示了 CoT 方法在常识问答中的应用，显著提高了模型的问答性能。

随后 Kojima 等人^[31]的研究发现，当使用的指令为 “Let’s think step by step” 时，模型在数学推理上的表现显著提升。这种方法被称作零样本思维链 (Zero-shot Chain-of-Thought)。这表示在零样本场景下，模型仍然能够将问题分解为多个步骤，并进行推理，而不需要额外的样本。这种方法的好处在于不需要针对每一个任务设计特定的提示，就可以获得各个推理任务上的显著提升，极大地提高了大语言模型的泛化能力。

同样具有这样性质的还有自一致性思维链^[19] (Self-Consistency Chain-of-

Thought)。这种方法在思维链的基础上，通过采用较高的采样温度^[56,57] (Temperature，控制模型生成结果的随机性的参数) 让模型多次采样，从而生成不同的推理路径，对结果进行多数投票作为最终结果。如图 2-3 所示。

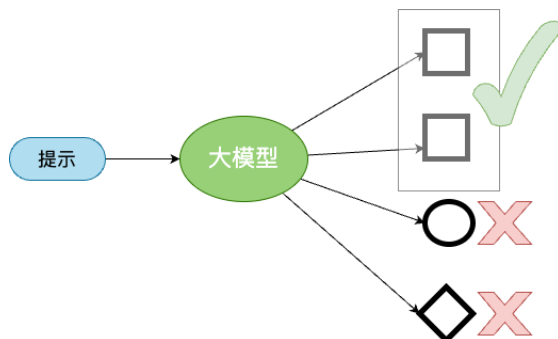


图 2-3 SCoT 的示意图^[19]

这种方法在一定程度上解决了思维链方法中可能存在的推理错误和不确定性，提高了答案的准确性，但是会带来计算成本的大幅上升。论文中使用的设置是 40 次采样，但是存在边际效应，即当采样次数超过一定阈值后，性能提升会变得不明显。实际使用中可能只要采样 10 次左右即能获得较好的性能。自一致性思维链是通过对大语言模型输出结果进行多次采样的方法，类似的方法还有思维树^[33] (Tree of Thoughts, ToT)，思维图^[34] (Graph of Thoughts, GoT) 等。

受到零样本思维链论文中两段推理的启发，最少到最多提示^[17] (Least-to-Most Prompting, LtM) 被提出。该方法巧妙地运用了分治策略，将复杂问题化繁为简，逐步解决，如图 2-4 所示。其核心步骤如下：

1. 问题拆解 (Problem Reducing)：首先，自上而下地分析问题，引导模型将复杂问题分解为若干个更易处理的子问题。此步骤为后续的逐步解决奠定基础。

2. 子问题有序解答 (Sequentially Solve Subquestions)：接着，采用自下而上的方法，依次解决每个子问题。

在这个过程中，每个子问题的答案都会作为下一个子问题的上下文信息，形成一个连贯的推理链。这种循序渐进的方法确保了问题解决的连贯性和完整性。通过这种方法，问题的复杂度由少到多逐渐增加，这也正是“Least-to-Most”（由少至多）这一名称的由来。这种策略不仅提高了模型处理复杂问题的能力，还增强了推理过程的可解释性和可追踪性。

上述各种思维链方法具有多方面的优势，使其在复杂推理任务中表现出色。以下是几个主要的优势：(1) 增强推理能力：通过分解复杂问题，CoT 方法使模型能够逐步推导出答案，从而显著提升了其推理能力。这种逐步推理的过程类似于人类的思维过程，使模型能够更好地处理复杂任务。(2) 提高可解释性：CoT 方法通过显式地展示推理过程，使模型的推理过程更加透明和可解释。这对于理

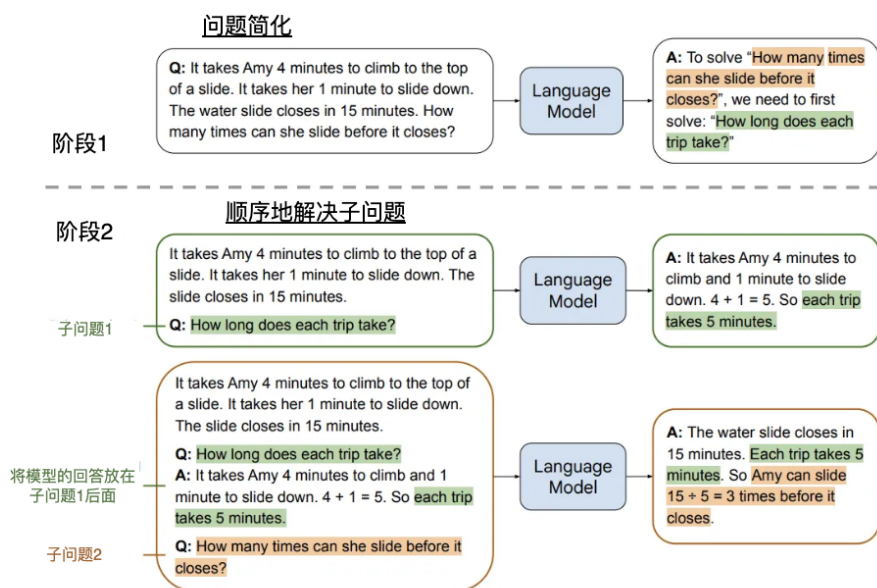


图 2-4 LtM 的示意图^[17]

解模型的决策过程和调试模型具有重要意义。(3) 灵活性和可扩展性: CoT 方法具有较高的灵活性和可扩展性, 能够适应不同类型的推理任务。通过适当的调整, CoT 方法可以应用于各种复杂推理任务中。

本文的研究结合了思维链的推理能力, 使模型能够生成详细的推理步骤, 同时通过多轮交互进一步优化推理过程。

2.1.3 工具集成

尽管结合了各种提示技术, 语言模型能够准确地给出解决问题的推理步骤, 但这并不意味着它一定能够正确地解决问题。事实上, 推理过程中的一个微小算术错误都可能导致最终结果的错误。这种现象通常被称为语言模型的组合性差距^[58] (Compositionality Gap)。值得注意的是, 这个差距并不会随着模型规模的增大和复杂度的提升而自动缩小。

造成这一现象的根本原因在于语言模型本身不擅长进行精确计算。为了解决这个问题, 一个理想的方案是将计算过程从推理中分离出来。具体而言, 让语言模型专注于推理, 而将具体的计算任务交由专门的计算模块处理。为了实现这一目标, 研究人员引入了外部工具来弥补语言模型在计算方面的不足。

这种方法与思维链方法有所不同, 被称为工具集成方法 (Tool-Integration)。它通过在模型中整合外部工具来解决复杂任务。对于数学推理^[59] 任务而言, 最为有效的工具主要是代码解释器。

在这一方向上, Gao 等人^[20] 提出了一种创新的程序辅助语言模型 (Program-aided Language Model, PAL), 几乎是同一时间, Chen 等人^[36] 也提出了类似的思

维程序提示 (Program-of-Thought, PoT)。这两种方法都采用了一种独特的问题解决方法, 将复杂问题巧妙地分解为一系列包含自然语言和 Python 代码的混合推理步骤。通过这种方式, 模型能够生成一个结构化的解决方案, 随后利用 Python 解释器执行这些代码段, 从而高效且准确地解决问题。

本文采用了类似 PAL 的程序辅助方法, 但进一步引入了错误检测和纠正机制, 从而提高了代码生成的准确性。

2.2 基于大语言模型的智能体

2.2.1 智能体的概念与定义

基于大语言模型的智能体^[29,60,61]是指在动态环境中具备自主感知、决策与执行能力的智能实体, 其区别于传统预设规则系统的重要特征在于类人的认知行为模式。该智能体系统通过大脑、感知和执行三大功能模块的协同运作, 展现出超越常规软件系统的环境适应性和任务处理能力。核心特征可归纳为:

1. 自主性: 具备独立的环境感知与行为决策能力, 可依据环境状态进行动态策略调整。这种特性使智能体能在无人干预条件下持续运作, 并在复杂场景中保持决策有效性。
2. 感知能力: 通过物理传感器或软件接口获取多维环境信息, 形成对交互场景的全面认知基础。
3. 认知决策: 基于感知信息进行多维度分析, 通过概率推理和逻辑推演生成最优决策方案, 具备风险权衡和方案优选能力。
4. 执行能力: 通过物理执行机构或数字接口实现环境状态的主动干预, 执行维度涵盖实体操作 (如机械臂控制) 和虚拟交互 (如 API 调用) 双重领域。

近年大语言模型在自然语言理解、逻辑推理和知识整合方面的突破性进展^[62,63], 为智能体系统提供了强大的认知内核。研究者通过思维链优化^[64]、多模态融合^[65,66]和工具集成策略, 成功扩展了智能体的环境交互维度, 使其在复杂任务处理中展现出类人的问题分解与动态适应能力。

2.2.2 智能体的架构

在人工智能领域, 基于大语言模型的智能体的研究和应用正成为新的焦点。传统软件程序通常需要用户明确输入指令, 按照既定的逻辑执行任务, 缺乏对环境的主动感知和适应能力, 更无法在复杂多变的情况下做出自主决策。与传统软件程序不同, 智能体是一种能够在特定环境中展现出自主性、感知能力、决策能力和行动能力的实体。基于大语言模型的智能体采用类脑认知架构, 包含大脑、感知、执行三大核心组件, 形成“感知-认知-执行”的闭环处理机制。该架构通过模块化设计实现信息流的层级处理, 有效模拟人类处理复杂任务的认知过程。

1. 大脑模块

作为系统的认知中枢，大脑模块依托大语言模型构建，承担信息整合、记忆管理和决策生成核心功能：(1) 语义理解与生成：通过预训练获得的深度语言表征能力^[67]，实现自然对话的上下文建模和意图识别，支持多轮对话的连贯性维护。(2) 知识图谱集成：融合结构化知识库与非结构化语料，形成覆盖常识与专业领域的动态知识体系^[63]，为复杂推理提供语义支撑。(3) 记忆管理机制：采用分层存储架构，短期记忆维护对话上下文（受限于 Transformer 的注意力窗口），长期记忆通过向量数据库^[68]实现知识的高效检索与更新。(4) 任务规划与推理：运用思维链技术实现复杂任务的递进式分解，生成可执行的动作序列。

2. 感知模块

作为环境交互的前端系统，感知模块具备以下核心功能：(1) 多模态信号处理：支持文本、图像^[65]、语音^[66]等异构数据的同步解析，通过特征融合构建统一语义空间。(2) 数据规范化处理：将原始感知数据转换为结构化表征，包括但不限于图像特征向量化、语音信号文本化等预处理流程。

3. 执行模块

作为决策执行终端，执行模块通过多通道输出实现环境干预：(1) 自然语言生成：根据认知状态生成符合语境的文本响应，支持对话延续和指令反馈^[69]。(2) 工具调用接口：通过 API 网关集成外部资源（如搜索引擎、数据库），扩展智能体的功能边界^[64]。(3) 物理执行机构：在具身智能场景中驱动机械装置完成实体操作，实现数字决策到物理行为的映射。

实现包含上述模块的大语言模型的智能体，需要多种关键技术的协同作用，以下是其中最为重要的若干关键技术：

1. 自然语言理解：自然语言理解是基于大语言模型的智能体与用户进行有效交互的基础。大语言模型通过预训练，积累了丰富的语言知识和语义信息，能够准确理解用户的自然语言输入。

2. 上下文管理：上下文管理是智能体实现连贯对话和任务处理的关键。短期记忆可以类比为受限于 Transformer 架构的上下文窗口，它能够处理并存储输入的信息；而长期记忆则类似于一个外部的存储系统，智能体可以迅速地访问和检索所需的信息。

3. 知识表示与推理：知识表示与推理能力是智能体解决复杂问题的核心。知识图谱为语言模型提供了更强大的上下文信息，使其能够在处理复杂问题和需要深度理解的场景中表现出色。

4. 工具使用与扩展：为了增强智能体的能力，工具使用与扩展技术不可或缺。智能体可以通过调用外部工具，如搜索引擎、计算器、数据库等，来获取更广泛的信息和执行更复杂的任务^[65]。

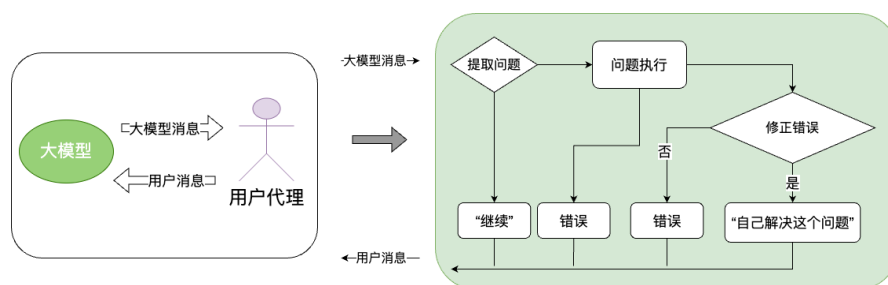
2.2.3 智能体的相关应用

在数学问题解决领域，智能体方法的应用显示出了巨大的潜力。它结合了思维链方法的推理能力和工具集成方法的精确计算能力，同时引入了更高层次的决策和规划能力。这使得智能体能够更加灵活和高效地处理复杂的数学问题，特别是那些需要多步骤推理和计算的问题。

最具代表性的智能体之一是 ReAct^[64]，它将语言模型与外部工具和环境进行集成，使其能够根据任务需求灵活地调用外部工具、执行操作，并根据反馈进行决策。ReAct 的核心思想是将推理（Reasoning）和行动（Acting）交替进行，形成一个动态的问题解决过程。具体来说，ReAct 包含以下几个关键步骤：1. 思考（Thought）：模型首先分析当前情况，思考下一步应该采取什么行动。2. 行动（Action）：根据思考的结果，模型选择并执行一个具体的行动，如搜索信息、调用工具等。3. 观察（Observation）：模型观察行动的结果，获取新的信息或反馈。4. 反思（Reflection）：基于观察到的结果，模型对当前的解决方案进行评估和调整。这个循环过程不断重复，直到问题被解决或达到预定的终止条件。通过这种方式，ReAct 能够处理需要多步推理和交互的复杂任务，展现出强大的问题解决能力和灵活性。在数学问题解决方面，ReAct 方法可以灵活地结合思维链推理和外部工具使用。例如，在解决一个复杂的数学问题时，ReAct 可以：1. 使用思维链方法分解问题并制定解决策略。2. 调用计算器或 Python 解释器执行精确计算。3. 根据计算结果调整后续的推理步骤。4. 在需要时搜索相关的数学公式或定理。这种结合多种能力的方法使得 ReAct 在处理复杂数学问题时表现出色，能够克服单一方法的局限性，提供更全面和准确的解决方案。

ReAct 是一个通用的框架，而在数学问题解决方面，Wu 等人^[40]提出了一个专用框架 MathChat。MathChat 是一个创新的框架，旨在模拟大语言模型助手与用户之间的对话过程，其工作流程如图 2-5 所示。在这个框架中，“用户”实际上是一个智能体，模拟与大语言模型助手进行交互的真实用户行为。MathChat 的核心目标是通过助手和用户的协作来解决复杂的数学问题。该框架通过精心设计的初始化提示，引导大语言模型助手以特定的方式与用户合作解决问题。这种对话式的设计充分利用了最先进大语言模型在聊天功能上的优化，使交互更加自然和高效。MathChat 的一个显著优势在于其支持多轮对话的能力。这一特性在处理需要多步推理和工具使用的复杂数学问题时尤为重要。通过持续的对话交互，MathChat 能够逐步分解问题，验证中间结果，并在必要时调整解题策略，从而更有效地应对各种复杂的数学挑战。

本文提出的自学习智能体架构借鉴了 MathChat 等方法的多轮交互的做法，还创新性地引入了感知、记忆和执行三大核心模块。这三个模块的协同运作使得

图 2-5 MathChat 的示意图^[40]

智能体能够更加全面和深入地处理复杂的数学问题。

2.3 本章小结

本章系统阐述了大语言模型在数学问题解决中的关键技术及应用。首先解析提示工程的核心方法：通过上下文学习实现少样本任务适应，运用思维链技术分解复杂推理，并借助工具集成弥补模型计算短板。其次，阐述了智能体系统的三维架构：大脑模块负责语言交互与推理规划，感知模块处理多模态输入，执行模块执行决策与工具调用。关键技术涵盖自然语言理解、知识推理、工具扩展及多模态交互，共同支撑智能体在复杂场景中的应用。全文整合技术优势与创新方向，为构建高效数学问题解决系统提供理论框架与实践路径。

第3章 基于错误类型检索和多轮程序辅助纠错的数学文字问题求解

3.1 引言

近年来,提升大型语言模型数学能力的研究蓬勃发展,主要形成了两大研究轨迹:一是以思维链为代表的自然语言推理方法,二是结合外部工具的方法。第一种方法不依赖外部工具,通过以思维链为代表的提示工程方法,逐步推理得到正确答案。代表性的研究包括思维链^[15]、自一致性^[19]、最少到最多提示^[17]等。第二种方法则常与外部 Python 解释器等工具结合,被称为工具集成方法。在这种方法中,外部代码执行器承担了复杂的计算任务,大大减轻了语言模型在解决问题时的负担。这种方法特别适用于处理计算需求较高的数学问题。代表性的研究有 PAL^[20]、PoT^[36]。

这两种方法都存在一定的局限性,思维链方法在处理复杂数学问题时,由于缺乏具体的计算过程,其计算结果的正确性难以保证。工具集成方法虽然能够提供具体的计算过程,得到准确的计算结果,但在处理复杂数学问题时,可能会产生无法执行的代码或者难以用代码表示的逻辑。

针对这些问题,很多现有研究尝试结合二者的优点,使用包含思维链和工具集成的对话形式解决数学问题,以弥补思维链关于计算的弱点,同时,用多轮对话的形式弥补工具集成方法难以一次成功的缺点。但是这些方法要么依赖于特定的闭源模型,比如 Zhou 等人^[70]利用 GPT-4 代码解释器 (GPT-4 Code Interpreter) 取得的显著成果,要么依赖于微调现有的开源模型, WizardMath^[26]、MuggleMath^[71]等。这些方法虽然有效,但是成本过高,且在很大程度上缺乏广泛的适用性。

针对上述现有方法存在的问题:部分方法通过微调模型进行改进的方式需要大量的计算资源和数据,而现有不依赖微调的方法面临着推理与计算分离,反馈与纠错机制缺失的挑战,使用基于检索的上下文学习还面临着检索策略单一的局限性。本章使用基于检索的上下文学习,通过提前构建结合思维链和代码的错题样本库,将动态检索的错题样本提供给大语言模型作为演示样本,在不需要微调的情况下,提升大语言模型在数学问题上的表现。

在本章中,错题样本检索是本章研究的核心内容和创新点之一,现有的基于检索的上下文学习^[72]分为基于相似性检索和多样性检索两个方面。其中,相似性的度量可以基于语言相似度(术语匹配或语义匹配)、结构相似度(句子结构、

推理结构匹配等)或其他标准。大多数研究都集中在语言相似度上,而基于结构相似度的研究较少,这通常是由于在许多任务中提取查询结构时存在挑战^[28]。除了相似性,一些工作发现,样本的多样性很重要。考虑多样性的动机包括避免重复的样本^[73],带来不同的视角^[74],并在覆盖单词或语法结构方面最大化样本对测试查询的覆盖^[28]。但基于多样性的检索的出发点往往是没有足够相似的样本,所以需要具有多样性的样本带来更多的参考。

而 Fu 等人^[18]提出了复杂思维链 (Complex CoT),其研究发现:基于文本相似度和相似推理复杂度的样本对数学推理具有帮助,其中推理复杂度由查询长度或推理步骤定义。这表明,结合相似性和复杂性考虑可能是增强数学推理任务方法的一种有希望的策略。受此启发,本章提出的方法旨在从相似性着手,检索具有相似推理复杂度的样本,本章方法与复杂思维链的区别在于:1. 样本检索的时机不同:复杂思维链在初次提示时使用,而本章方法在初次生成失败后,纠错时使用。2. 样本检索的策略不同,复杂思维链基于文本相似度和推理复杂度匹配推理正确的样本,而本章方法基于代码执行后 Python 解释器返回的错误类型和数学问题的语义相似度检索包含错误修正过程的样本。

本章的主要贡献在于:

(1) 多轮程序辅助纠错框架:提出无需模型微调的多轮交互式纠错框架,通过动态调用 Python 解释器验证执行结果并检测错误,实现渐进式代码修正,显著提升复杂数学问题的求解成功率;

(2) 结构化样本生成:设计基于推理路径的层次化样本结构,每个节点融合自然语言推理、程序代码与执行状态三元组信息,并利用公开数据集自动生成包含错误模式与修正路径的多样化训练样本;

(3) 动态错误类型检索机制:开发结合错误类型匹配与上下文相似度的动态检索算法,通过实时分析程序异常特征,从样本库中自适应选取最相关的纠错案例指导模型修复代码逻辑。

(4) 在 MATH-minival 和 GSM8K 数据集上的实验结果表明,该方法在解决数学文字问题上的准确率较基线模型提升 1.4%-7.5%,验证了错误类型检索和多轮程序辅助纠错方法的有效性。

本章所用术语如表3-1所示

表 3-1 第三章术语表

术语	解释
数学文字问题 (Math Word Problem)	以自然语言描述的数学问题, 通常包含问题背景、已知条件和待求解的问题。涵盖代数、几何、概率等方面的问题, 但模态为语言
思维链 (Chain-of-Thought, CoT)	一种基于上下文学习的方法, 通过引导模型生成中间推理步骤来解决复杂问题, 显著提升模型的推理能力。
程序辅助提示 (Program-aided Prompting, PAL)	结合自然语言推理和程序执行的方法, 利用外部工具 (如 Python 解释器) 进行精确计算, 弥补语言模型在计算上的不足。
上下文学习 (In-Context Learning)	通过少量示例或上下文信息引导模型完成任务的学习范式, 无需显式参数更新。
错误类型检索	根据代码执行返回的错误类型和问题的语义相似度, 从样本库中检索包含错误修正过程的样本, 用于模型的多轮纠错。
多轮程序辅助纠错	在代码生成失败后, 通过多轮交互和错误类型检索, 动态选择相似的错题样本, 逐步修正代码中的错误。
错题样本库	包含数学文字问题及其解题过程 (包括错误和修正过程) 的样本库, 用于上下文学习和错误修正。
语义相似度	衡量两个问题在语义上的相似程度, 通常通过向量表示和相似度计算实现, 用于样本检索和匹配。

3.2 问题定义

3.2.1 问题定义

在解决数学文字问题的过程中, 数学文字问题 P 可以表示为一个包含问题描述 D 和期望解答 A 的对: $P = (DA)$ 。其中, 问题描述 D 是一个自然语言文本, 描述了问题的条件和要求; 期望解答 A 是一个数学表达式、数值或一系列步骤, 代表问题的解答。

M 是一个大语言模型, 用于生成自然语言文本: $M: D \rightarrow T$ 。模型 M 由大量的文本数据训练而成, 具备理解和生成自然语言文本的能力。

在 多 轮 交 互 的 情 境 下, 本 章 方 法 定 义 一 个 交 互 序 列 $I = \{(DR_1)(F_1R_2)\dots(F_{n-1}R_n)\}$, 其中 D 是初始问题描述, R_i 是模型在第 i 轮的响应, F_i 是第 i 轮的反馈或额外信息。每一轮的响应 R_i 可能包含解答过程 S_i 和中间结果 T_i 。本章方法还引入一个错题样本集 $E = \{(D_1S_1A_1)(D_2S_2A_2)\dots(D_kS_kA_k)\}$, 其中 k 是样本数量, S_i 表示问题 D_i 的解答过程, A_i 是问题 D_i 的正确答案。

在每一轮交互中, 给定当前的问题状态 (包括初始描述 D 、之前的交互历史和当前的反馈 F_i) 以及错题样本集 E , 模型 M 需要生成新的响应 R_{i+1} 。

形式化地表示, 在第 i 轮交互中, 大语言模型模型 M 的目标是生成响应:

$$R_{i+1} = M(EDI_{1:i}F_i) \quad (3-1)$$

其中, $I_{1:i}$ 表示前 i 轮的交互历史。在多轮交互的过程中, 如果模型在第 $i+1$

轮生成的响应 R_{i+1} 被认为是满意的解答，则将其作为最终解答 T ，即 $T = R_{i+1}$ 。最终的解答 T 是多轮交互后的结果，其正确性通过与期望解答 A 的比较来评估，即 $T \approx A$ 。这个过程允许模型通过多轮交互逐步改进其解答，直到达到满意的结果或达到预定的交互轮数。

3.3 方法介绍

本节介绍一种基于错题样本的多轮程序辅助提示方法，该方法通过构建和检索包含错误修正过程的样本库，利用多轮交互和错题样本库中相似错误修正过程来提升大语言模型在数学文字问题上的解题准确率。这种方法的创新之处在于通过错误类型和语义相似度进行检索，得到具有相似错误修正过程的错题样本作为上下文学习的样本，使大语言模型能够从最为相似的修正案例中学习并改进。具体地，本节首先介绍多轮程序辅助纠错方法概览，然后介绍错题样本的结构和生成方法，最后介绍基于错题样本的动态错误类型检索。

3.3.1 多轮程序辅助纠错方法概览

本章提出的基于错误类型检索的多轮程序辅助纠错提示方法，整体流程如图3-1所示。

1. 首先，给定一个数学文字问题，使用图中绿色字体所示提示词和固定的少量成功解决问题的样本作为输入，引导大语言模型生成思维链和 Python 代码。
2. 接着，将生成的 Python 代码提交给 Python 解释器执行。
3. 如果代码执行成功，则过程结束，执行结果作为最终回答。
4. 若代码执行失败，系统会根据 Python 解释器返回的错误类型和问题语义相似度，从预先构建的错题样本库中选择合适的样本。
5. 将选中的样本与原问题一起作为新的输入，再次提交给大语言模型，进行下一轮的思维链和代码生成。
6. 重复步骤 2 到 5，直到程序正常运行或达到预设的最大尝试次数（在图中未体现）。

这种方法通过多轮交互和检索包含相似纠错过程的样本，不断优化解题过程，提高了复杂数学文字问题的解决成功率。

3.3.2 错题样本的结构

本文中的错题样本是基于检索的样本，其中含有对错题的修正过程，这与上下文学习中经常使用的固定的少样本有所区别，本章中错题样本的结构如图3-2所示，具体包括如下：

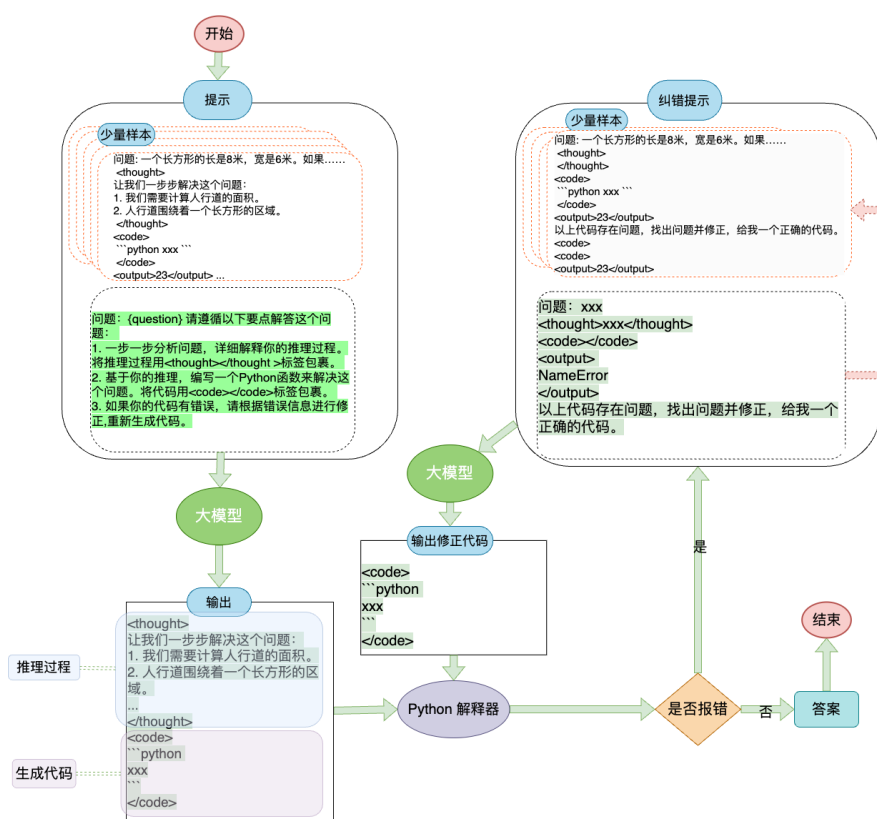


图 3-1 错误类型错题样本的多轮程序辅助纠错提示方法流程示意图

1. 问题：数学文字问题，如图3-2蓝色部分所示，以自然语言形式表达，涉及到数学公式的部分用 Latex 格式表示。

2. 自然语言推理过程：如图3-2中黄色部分所示，被 `<thought>` 标签包裹。这部分包含了模型对问题的理解和解题思路的阐述。它对于激发模型的推理能力至关重要，因为它模拟了人类解题时的思考过程。通过提供清晰的推理步骤，可以帮助模型更好地理解问题结构和解题策略。现有方法大多对其重视程度不够，本研究认为其对于激发模型的推理能力至关重要。其必要性验证在节3.4.5中进行。

3. Python 代码：如图3-2中紫色部分所示，被 `<code>` 标签包裹。这部分是大语言模型输出的能够解决这个问题的 Python 代码，一个错题样本中包含初次生成的代码和初次代码运行出错后的纠正代码，用于纠正的代码数量不固定，不少于一个，不多于最大尝试数量。相比于纯自然语言的推理，代码的方式可以更加精确的计算出问题的答案，并且清晰的显示模型解决问题的逻辑。

4. 程序输出结果：如图3-2中红色部分和绿色部分所示，被 `<output>` 标签包裹。这部分是 Python 代码经过 Python 解释器执行后得到的返回值。当程序运行错误时，包含的是错误类型，错误类型用于之后的错题检索；当程序运行成功时，直接返回问题的结果。

这种结构化的样本设计不仅提供了完整的问题解决过程，还展示了对错误

Question: Find $\frac{2k+5}{3} - \frac{4-5k}{6}$ simplifies to the form $ak+b$ where a and b are integers. What is $\frac{a}{b}$?

<thought>
First, we need to simplify the expression. We can do this by finding a common denominator and then combining the numerators. Then we extract the values of a and b , and finally divide a by b .
</thought>

```
<code>
"""python
from sympy import symbols, simplify, Rational
def a_over_b():
    k = symbols('k')

    # Simplify the expression to the form ak + b
    expression = simplify(Rational(2*k+5,3) - Rational(4-5*k,6))
    # Get a and b
    a = expression.as_coefficients_dict()[k]
    b = expression.as_coefficients_dict()[1]
    # Calculate a/b
    result = simplify(a/b)
    return result
result = a_over_b()
print(result)
"""
</code>
```

<output>
TypeError: invalid input: 2*k + 5
</output>

The code above has encountered a problem. Now point out its mistakes and then correct them.
The error in the first attempt is due to an improper use of the 'Rational' function. The 'Rational' function should have two parameters, both integers, to represent a fraction. However, in the first attempt, the parameters are polynomial expressions. The solution is to use the '/' operator to represent the division between these expressions.

Let's correct the code as follows:

```
<code>
"""python
from sympy import symbols, simplify

def a_over_b():
    k = symbols('k')

    # Simplify the expression to the form ak + b
    expression = simplify((2*k+5)/3 - (4-5*k)/6)

    # Get a and b
    a = expression.as_coefficients_dict()[k]
    b = expression.as_coefficients_dict()[1]

    # Calculate a/b
    result = simplify(a/b)

    return result
result = a_over_b()
print(result)
"""
</code>
```

<output>
3/2
</output>
The ratio $\frac{a}{b}$ is $\boxed{\frac{3}{2}}$.

图 3-2 错题样本结构示意图

代码的修正过程。它使模型能够学习到更全面的问题解决策略，包括推理、编码和调试。利用合适的检索策略，可以使模型在面对相似问题和相同错误时得到正确修正的概率增大。

3.3.3 错题样本的生成

在 MATH^[24] 和 GSM8K^[8] 数据集的训练集上，利用提示工程和数据集中的标准答案，自动化地生成和验证上文中提到的错题样本。具体地，对于数据集中的每个问题 $P = (DA)$ ，本章方法生成一系列的推理过程 $R = \{r_1 r_2 \dots r_n\}$ ，其中每个推理过程 r_i 可以表示为：

$$r_i = \{(t_1 c_1 e_1)(t_2 c_2 e_2) \dots (t_m c_m e_m)\} \quad (3-2)$$

其中， t_j 表示第 j 步的自然语言推理， c_j 表示对应的程序代码， e_j 表示 Python 解释器执行 c_j 的反馈。

对于每个推理过程 r_i ，本章定义一个错误修正函数 f ：

$$f(r_i) = \begin{cases} r_i & \text{如果 } r_i \text{ 正确解答了问题} \\ f(r'_i) & \text{否则} \end{cases} \quad (3-3)$$

其中 r'_i 是对 r_i 进行修正后的推理过程。修正过程考虑了 Python 解释器的执行反馈 e_j ，用于指导下一步的修正。这个过程会迭代进行，直到得到正确的解答或达到预定的最大迭代次数 k ：

$$R_{final} = \{f^k(r_i) | r_i \in R\} \quad (3-4)$$

最终生成的样本集 S 包含了所有正确的推理过程以及它们的修正过程：

$$S = \{(Pr) | r \in R_{final} \text{ 正确解答了问题 } P\} \quad (3-5)$$

这个过程不仅包括正确的推理过程，还包含了错误的尝试、Python 解释器的执行反馈和修正过程，为后续的错题样本检索提供了素材。

提示词

问题: {question}

请遵循以下要点解答这个问题：

1. 一步一步分析问题，详细解释你的推理过程。将推理过程用<thought></thought>标签包裹。
2. 基于你的推理，编写一个Python函数来解决这个问题。将代码用<code></code>标签包裹。
3. 如果你的代码有错误，请根据错误信息进行修正,重新生成代码。

图 3-3 样本生成提示词

首先，使用如图3-3所示的提示词和少量固定的错题样本生成规定格式的反馈，（这里的固定指的是上下文学习中通常使用的，针对一类问题所固定使用的样本，有区别于基于检索的样本）包括自然语言形式的推理过程和 Python 代码。将 Python 代码提取出来，使用 Python 解释器运行代码，如果代码执行出错，则将错误信息反馈给模型，要求其修正代码。这个过程会重复多次，直到得到正确的结果。程序在运行一定的迭代次数后，如果仍然没有成功运行或者没得到正确结果，则舍弃该推理过程。得到程序的正确返回后将结果与标准答案对比，如果正确将该推理过程保留。整体的错题样本生成过程如图3-4所示。

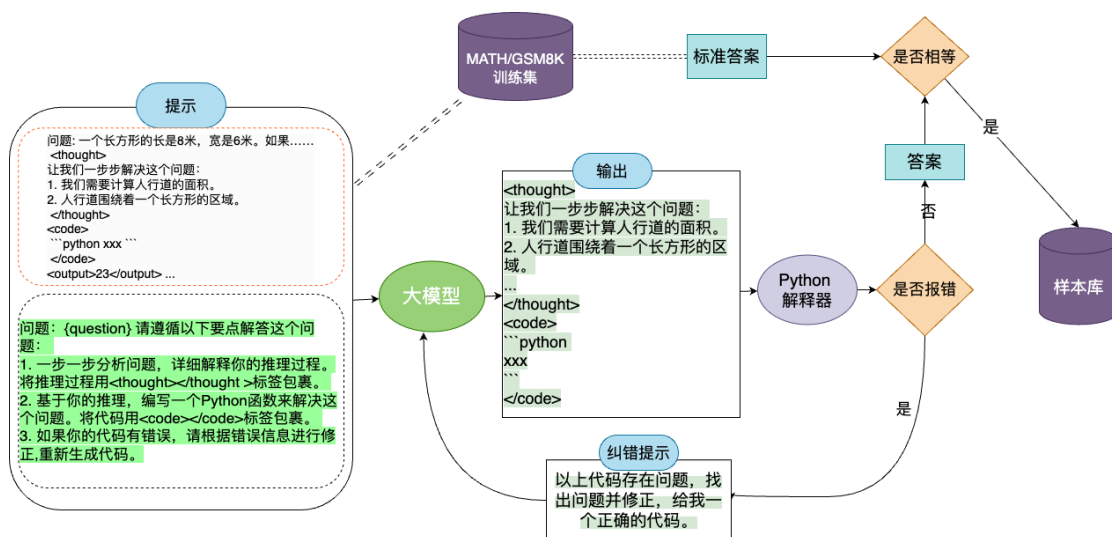


图 3-4 错题样本生成过程示意图

使用这种自动化的生成方式，从 MATH 和 GSM8K 数据集的训练集上生成 1.3 万个样本，其中包含 4897 个修正成功的样本。这 4897 个样本将用于后续的错题样本检索。

3.3.3.1 错题样本质量控制

为了挑选出答案正确的错题样本，本章通过比对生成的答案与对应问题来源数据集中的标准答案，确保错题样本的正确性。在使用 MATH 数据集中提供的基于规则的判断函数对模型输出的答案和标准答案进行比对时，可能会产生大量误判结果，主要是有以下几个原因：

- 同一个结果但是格式不同，比如一个是分数，一个是小数，但是他们是相等的。
- 受限程序的输出格式，程序计算的结果一般是数值，但是这与某些比如 0.999999999... 和 1 在数学上是相等的。
- 表示形式的不同，比如 11,180 和 11180 是相等的，pi 和 3.14159265358 是

相等的

这些情况难以用简单的规则进行匹配,所以本文在规则的基础上,利用一个由大语言模型驱动的判断函数,来处理基于规则的判断函数认定为错误的答案,加强对结果判断的准确性。这个大语言模型驱动的判断函数所使用的提示词如图3-5所示。

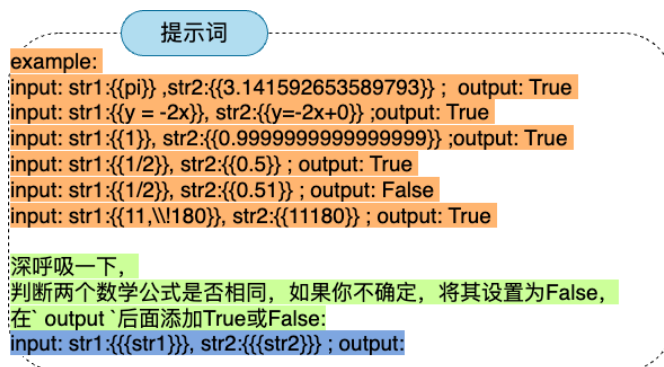


图 3-5 判断函数提示词

相比于仅使用基于规则的判断函数,结合规则和大语言模型的方法准确率更高,结合的方式为:当规则判断两个结果不相等时,使用大语言模型进行判断,如果大语言模型判断为相等,则认为两个结果相等。上述逻辑的形式化表达如 algorithm1所示。

Algorithm 1: 判断函数示例伪代码

Input:

- 1: *result*: 待判断的结果
- 2: *ground_truth*: 标准答案

Output:

- 3: 布尔值,表示两个结果是否等价
 - 4: **if** *is_equiv*(*result*, *ground_truth*) **then**
 - 5: **return** True
 - 6: **else if** *is_equiv_by_llm*(*result*, *ground_truth*) **then**
 - 7: **return** True
 - 8: **else**
 - 9: **return** False
 - 10: **end if**
-

其中 *is_equiv* 函数是基于规则的判断函数, *is_equiv_by_llm* 函数是基于大语言模型的判断函数。

表 3-2 判断函数性能对比（基于 500 个 MATH 样本）

方法	判断函数	准确率	判断错误数量
CoT	基于规则	0.622	23
	规则 + 大语言模型	0.66	4
PAL	基于规则	0.618	60
	规则 + 大语言模型	0.732	3

在 MATH 数据集的测试集中随机选取了 500 个样本进行测试，结果对比如表3-2所示。该结果展示了两种方法在思维链（表中标记为 CoT）和程序辅助提示（表中标记为 PAL）生成样本上的表现，基于规则的方法判断错误的数量更多，导致其准确率相较于实际的准确率偏低。尤其是对于使用代码运行出的结果，结果相差过大，所以结合规则和大语言模型的判断函数是必要的。

3.3.4 基于错题样本的动态错误类型检索

本章中基于错题样本的错误类型检索是核心内容和创新点之一，现有基于检索的上下文学习^[72]聚焦于相似度、多样性^[28]。这些方法的初衷是通过对少样本的整体质量的提升，而不是单个样本的质量，单次直接生成答案，来提升大语言模型的生成效果。但目前对上下文学习的生效机制尚不明确^[72,75]，多样性样本的作用还不明确。而 Fu 等人^[18]的研究发现，基于文本相似度和相似推理复杂度的样本对数学推理具有帮助。

本章的基于错题样本的错误类型检索方法，聚焦于单个样本的质量从而提升整体质量，通过选择与当前问题最相关的样本，来提升大语言模型的生成效果。从相似问题且出现相同错误的修正样例中，模型可以学到解决同样问题的思路，相似问题且具有相同错误的推理过程往往具有相似的复杂度，这也符合 Fu 等人^[18]的研究发现。通过选择与当前问题在语义和错误类型上都相似的错题样本，可以为模型提供更有针对性的指导，从而提高模型修正代码中错误的概率，从而提高最终回答的正确率。

3.3.4.1 样本数据库构建

本章中的基于错题样本的错误类型检索主要基于 Milvus^[68] 数据库实现，有以下几个关键设置：

将每个数学问题转换为向量表示，以捕捉其语义特征。本文采用 Sentence Transformers 库中的 all-MiniLM-L6-v2^[76] 模型进行向量化。选用该模型的主要原因如下：效率，作为轻量级模型，在性能和计算成本之间取得良好平衡。通用性，经过多样化数据训练，适用于各种文本嵌入任务。性能，尽管体积小，但在多项基准测试中表现出色。多语言能力，能处理多种语言，有利于处理多样化的数学

问题。这些特性使得 all-MiniLM-L6-v2 模型非常适合高效且有效地进行数学问题向量化。

索引构建。使用 Milvus 数据库构建高效的向量索引，在 IVF_FLAT 和 HNSW 两种主流索引方案中，本研究选择了 IVF_FLAT 索引。IVF_FLAT 通过先将数据聚类，搜索时只在相关聚类中查找的方式实现快速检索；而 HNSW 则是构建多层图结构实现快速搜索。选择 IVF_FLAT 而非 HNSW 的原因是：平衡了查询速度和准确性，对于中等规模的数据集，IVF_FLAT 提供了良好的查询性能，同时保持较高的准确度。适合频繁更新，相比 HNSW，IVF_FLAT 更容易进行增量更新，这对于检索样本库很重要。内存占用较小，相比 HNSW 需要存储大量图结构信息，IVF_FLAT 的内存消耗更低，适合资源受限的环境。

3.3.4.2 相似度搜索

根据当前问题的向量表示，在 Milvus 中进行 TopK 近似最近邻 (ANN) 搜索，使用内积 (IP) 作为相似度度量。选择 IP 的原因是：适合归一化向量，all-MiniLM-L6-v2 模型输出的是归一化向量，对于这类向量，IP 等价于余弦相似度，但计算效率更高。保留方向信息，相比欧氏距离，IP 更好地保留了向量的方向信息，这对于捕捉语义相似性很重要。计算效率高，IP 计算简单，有利于提高大规模检索的效率。

如图3-6所示，在进行相似度搜索的同时，本研究将错误类型作为检索条件之一。这个过程可以被形式化表达如下：

设 Q 为当前问题， E 为错误类型， S 为样本集， k 为需要选择的样本数量。

定义相似度函数 $\text{sim}(Qs)$ ，其中 $s \in S$ 。

定义错误类型匹配函数：

$$\text{match}(Es) = \begin{cases} 1 & \text{如果 } s \text{ 的错误类型与 } E \text{ 相同} \\ 0 & \text{否则} \end{cases} \quad (3-6)$$

则最终的样本检索函数可表示为：

$$\text{select}(QESk) = \text{TopK}_{s \in S} \{ \text{sim}(Qs) \cdot \text{match}(Es) \} \quad (3-7)$$

其中，TopK 函数返回前 k 个最高分的样本。

这种多维度的匹配策略显著提高了样本的相关性，从而增强了大语言模型从这些样本中学习的效率和效果。通过选择 k 个最相关的样本，本章方法可以为模型提供最具相关性的上下文信息，为纠正错误的过程提供重要参考。

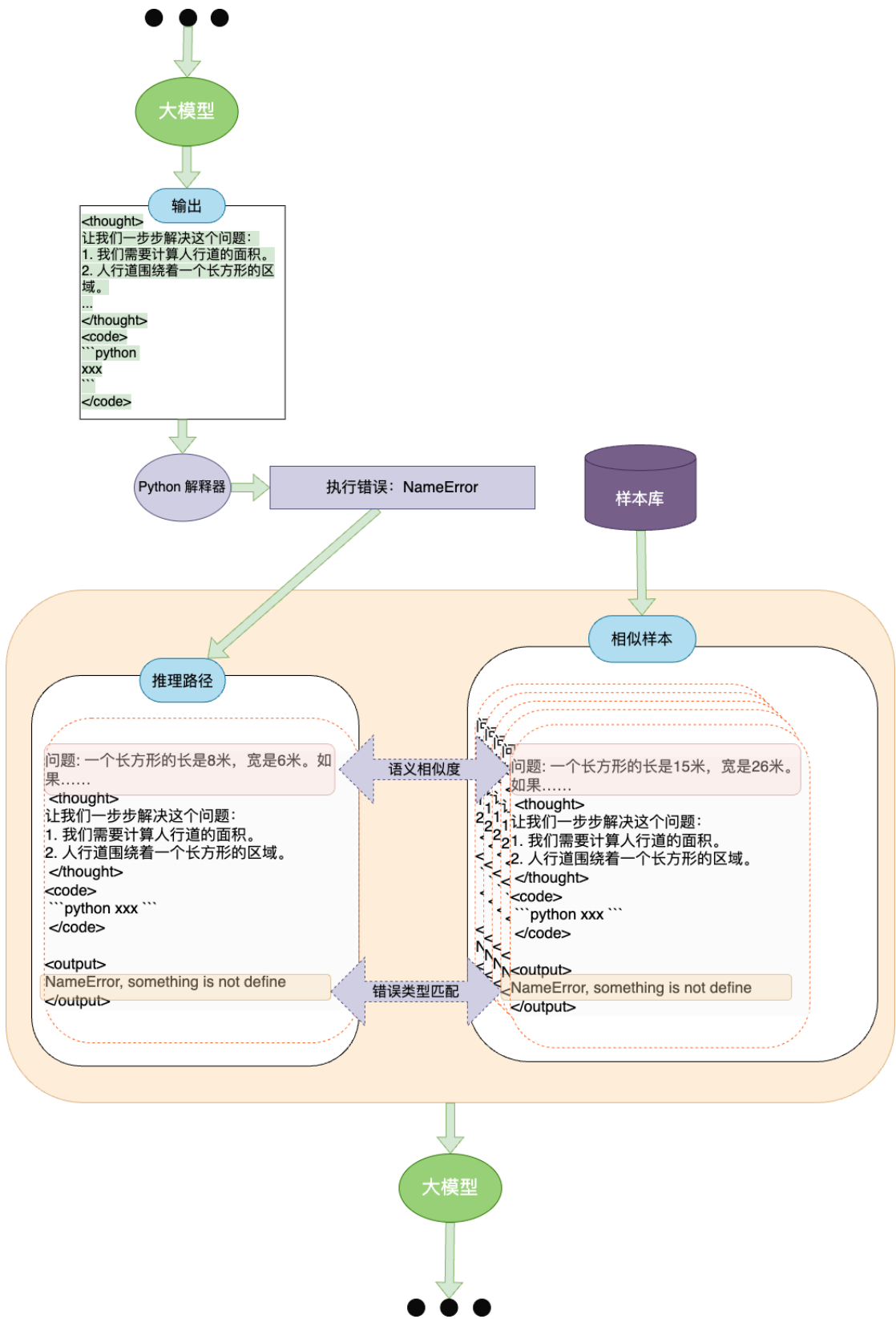


图 3-6 错题样本检索示意图

3.4 实验与结果

3.4.1 数据集介绍

本文采用了两个数据集，分别是 MATH-minival 数据集和 GSM8K 数据集，二者的详细介绍如下：

MATH^[24] 数据集是一个由 12500 个具有挑战性的竞赛数学问题组成的集合，每个问题都配有完整的逐步解决方案，这可以用来教导模型如何生成问题答案的推导和解释。由于 MATH 的测试集较大（5000 个样本），调用 API 进行全量测试的成本较高，所以在本章实验中使用 MATH 测试集的一个广泛使用的子集 MATH-minival 作为测试集，数据集包含 500 个样本，在 OpenAI 的论文^[77]中被用作测试集。该测试集分布较为均匀，包含各种类型和难度的问题，可以真实的反应大语言模型在数学推理方面的能力。

GSM8K^[8] 数据集由 OpenAI 发布，包含 8500 个高质量的、语言多样化的小学数学问题。这些问题由人类编写，分为 7500 个训练问题和 1000 个测试问题。它们需要 2 到 8 个步骤来解决，主要涉及基本的算术运算（+、-、×、÷）以得出最终答案。该数据集常用与测试大语言模型在数学推理方面的能力。

以上两个数据集由于难度适中，被广泛用于测试大语言模型在数学文字问题方面的解决能力，所以本章实验也将基于这两个数据集进行。

3.4.2 评价方法

本章实验使用节3.3.3.1中提到的算法与数据集中的标准答案进行对比，判断生成的答案是否正确，即将准确率作为评价指标。具体地，对于每个问题 i ，给出如下定义：

$$\text{Correct}_i = \begin{cases} 1 & \text{if is_equiv(result}_i\text{ground_truth}_i\text{)} \\ 0 & \text{otherwise} \end{cases} \quad (3-8)$$

其中，is_equiv 是综合了基于规则和基于大语言模型的判断函数，如节3.3.3.1所述。

然后，计算整体准确率：

$$\text{Accuracy} = \frac{\sum_{i=1}^N \text{Correct}_i}{N} \quad (3-9)$$

其中 N 是测试集中的问题总数。这个准确率作为本章的主要评价指标。

3.4.3 基线方法

本章选择的基线方法主要从以下几个维度进行系统性考虑：

1. 思维链类方法：这类方法通过自然语言推理来解决问题，代表了当前解决数学问题的主流方法。

(1) 零样本思维链^[31] (Zero-Shot CoT)：该方法是在思维链的基础上，使用一个更加简单的提示“Let’s think step by step”来激发模型的逐步推理能力。这种方法不需要额外的样本，而且由于现有大语言模型在预训练和监督微调的过程中就会进行思维链的训练，所以这个方法虽然简单，但是是一个很好的基线。

(2) 少样本思维链^[15] (Few-Shot CoT)：该方法使用少量样本作为提示，来激发模型的逐步推理能力。这种方法需要额外的样本，通常是人工编写的固定样本，所以需要根据不同的任务进行调整。可以验证样本对模型推理的帮助程度。

2. 程序辅助类方法：这类方法结合了自然语言推理和程序执行，与本章方法的技术路线相近。

(1) 零样本 PAL^[20] (Zero-Shot PAL)：该方法在不提供样本的情况下，靠模型本身的能力生成代码，执行得到答案。验证模型在无样本情况下的代码生成和执行能力。

(2) 少样本 PAL^[20] (Few-Shot PAL)：固定有 5 个样本的 PAL 方法，因为有了这些样本，所以生成的代码会更加准确，执行得到的答案也会更加准确。但需要手动编写样本，所以通用性较差。通过固定样本提升代码生成质量，可以对比本章错题样本检索策略的优势。

3. 复杂问题处理类方法：这类方法采用特殊的策略来处理复杂数学问题。

(1) 最少到最多提示^[17] (Least-to-Most Prompting)：最少到最多提示是一种基于最少到最多提示的解题方法，通过将当前问题分解为多个子问题，然后逐步解决这些子问题。与传统思维链不同的是，LtM 在解决每个后续子问题时，会将先前子问题的解决方案纳入提示中，从而为模型提供更丰富的上下文信息。这种渐进式的方法不仅有助于模型更全面地理解问题结构，还能够在解决过程中不断积累和利用已获得的信息，从而提高解决复杂问题的效率和准确性。通过问题分解策略处理复杂问题，可以验证本章方法在处理多步骤推理时的效果。

(2) 复杂思维链^[18] (Complex CoT)：复杂思维链是一种基于问题复杂度的样本选择方法。与传统的随机选择样本不同，复杂思维链方法根据问题的复杂度来选择与当前问题复杂度相似的样本。这种方法的核心思想是，对于复杂问题，使用同样复杂的样本进行推理可以提供更有针对性的指导，从而提高模型的推理能力。具体来说，复杂思维链方法首先对每个样本问题进行复杂度评估。复杂度可以基于问题的字数、解题步骤数等多个维度来衡量。本章实验中选用的是基于

问题字数的复杂度评估方法。然后，对于给定的待解决问题，该方法会选择复杂度与之最接近的若干个样本作为推理的参考。这些样本提供了与当前问题难度相当的解题思路和策略，有助于模型更好地理解 and 解决问题。该方法通过基于问题复杂度进行样本匹配，可以对比本章基于错误类型的样本选择策略的优势。

这些基线方法不仅涵盖了目前解决数学文字问题的主要技术路线，而且各自针对不同的问题特点（如推理复杂度、代码生成、样本利用等），能够从多个角度全面评估本章提出方法的效果。同时，这种分类方式也有助于深入分析本章方法在各个方面的优势和潜在改进空间。

3.4.4 实验设置

本章所用进行的实验所用的大语言模型为 DeepSeek-V2-0628, moonshot-v1-8k, doubao-pro-32k, 这几个模型在国内大语言模型中表现优异，其理解，生成，逻辑推理能力可以满足实验进行上下文学习的要求。

实验中使用的模型参数设置如下：在所有方法中，Temperature 设为 0，Top-p 设为 1.0，最大生成 token 数为 2048。

在本方法中：样本库的初始大小为 700，后拓展至 4897。迭代设置方面，最大纠错迭代次数为 6，每次代码执行的超时时间为 10 秒。程序执行环境使用 Python 3.10 版本，并额外引入 numpy 和 sympy 库。为确保结果的可靠性，每个数据集上的实验重复 5 次，取平均值作为最终结果。

消融实验中，使用的模型为 DeepSeek-V2-0628。

3.4.5 预实验

在实验之前，首先使用 MATH 测试集中随机的 100 个数据对模型进行测试，以确定自然语言推理过程对推理的帮助。该实验将有自然语言推理过程的称为有前缀，没有自然语言推理过程的称为无前缀。通过使用同样的 5 个样本，一个有前缀，一个无前缀，在 PAL 方法中进行实验，以验证自然语言推理过程对代码生成过程的影响。实验结果如下：

表 3-3 自然语言推理过程对代码生成的影响

Model	MATH-minival		GSM8K	
	无前缀	有前缀	无前缀	有前缀
DeepSeek-V2-0628	61.2%	68.4%	85.6%	91.8%
moonshot-v1-8k	39.8%	45.6%	78.2%	84.8%
doubao-pro-32k	58.8%	66.2%	83.7%	90.2%

从表3-3中可以看出，在本文实验的所有模型上，有自然语言推理过程（有前缀）的方法都表现优于无自然语言推理过程（无前缀）的方法。这说明在使用

程序辅助提示方法时，有自然语言推理过程可以更好的提升生成代码的质量。

3.4.6 实验结果

本节将使用上文中提到的评价方法，在 MATH-minival 和 GSM8K 两个数据集上进行实验，并对比各方法的性能。具体的实验结果如下：

表 3-4 各方法在不同数据集上的准确率

Model & Method	MATH-minival	GSM8K
DeepSeek-V2-0628		
Zero-Shot CoT	68.4%	91.8%
5-shot CoT	70.2%	93.6%
zero-shot pal	57.8%	89.2%
5-shot pal	61.4%	91.8%
Complex CoT	72.8%	94.8%
Least-to-Most Prompting	71.6%	94.2%
Ours	74.2% (+1.4%)	95.1% (+0.3%)
moonshot-v1-8k		
Zero-Shot CoT	45.6%	84.8%
5-shot CoT	47.8%	86.4%
zero-shot pal	38.2%	82.0%
5-shot pal	42.6%	84.2%
Complex CoT	50.4%	87.8%
Least-to-Most Prompting	49.0%	87.2%
Ours	53.2%(+2.8%)	88.5% (+0.7%)
doubao-pro-32k		
Zero-Shot CoT	66.2%	90.2%
5-shot CoT	68.8%	92.4%
zero-shot pal	44.2%	87.0%
5-shot pal	53.0%	89.0%
Complex CoT	70.4%	93.8%
Least-to-Most Prompting	69.6%	93.2%
Ours	72.2%(+1.8%)	94.3%(+0.5%)

从表3-4中的实验结果，可以得出以下几点分析：

1. 方法间的比较：本章提出的方法在所有三个模型上都取得了最佳性能，相比其他方法有明显提升，尤其在 MATH-minival 数据集上。在基线方法中，复杂思维链方法表现最为出色，在所有模型上都达到了最好的性能。最少到最多提示方法紧随其后，在某些情况下优于思维链和程序辅助提示方法。程序辅助提示方法（无论是零样本还是少样本）在大多数情况下表现不如思维链方法，这可能是由于一次性生成代码的难度较大，并且大多数模型在预训练阶段主要是通过思维链的形式进行训练，所以思维链方法在数学推理方面表现更好。而本章方法在一定程度上弥补了程序辅助提示方法的不足，结合了多种方法的优势，使得其生成的代码更加准确，从而提高了模型的性能。

2. 模型间的比较：DeepSeek-V2-0628 模型在所有方法上都表现最佳，显示出其强大的数学推理能力。moonshot-v1-8k 模型整体表现较弱，但在应用本章方法后，性能提升最为显著。这表明本章方法在数学推理能力较弱的模型（如

moonshot-v1-8k) 上的提升更为显著, 说明该方法有助于弥补模型本身的不足。

3. 数据集间的比较: 所有模型在 GSM8K 数据集上的表现普遍优于 MATH-minival 数据集, 这可能是因为 GSM8K 数据集的问题相对较为简单, 主要涉及小学级别的数学问题。而 MATH-minival 数据集可能包含更复杂和多样的数学问题。MATH-minival 数据集上的性能差异更大, 反映了该数据集问题的多样性和复杂性, 也突显了本章方法在处理复杂问题时的优势。

总体而言, 实验结果强有力地支持了本章提出的方法的有效性。该方法不仅能提升模型性能, 而且对于有一定基础能力, 但是没有经过大量数学数据训练的模型, 提升效果更为明显。这表明本章方法有潜力成为提升大语言模型数学推理能力的有效工具, 特别是在处理复杂数学问题时。

3.4.7 消融实验

为了进一步验证本章提出方法的有效性, 本节进行了一系列消融实验。这些实验旨在分析方法中各个组成部分的贡献, 以及它们如何影响整体性能, 主要的消融实验包括:

1. 错题样本检索的影响: 该实验比较了使用错题样本检索和不使用错题样本检索 (即使用固定样本) 的性能差异。这有助于理解错题样本检索策略在提高模型性能中的作用。

2. 迭代次数的影响: 该实验测试了不同的最大迭代次数对模型性能的影响。这可以帮助找到最佳的迭代次数, 在性能和计算效率之间取得平衡。这里最大迭代次数为 0 时, 表示不使用多轮对话对代码进行纠错。

3. 样本库大小的影响: 该实验比较了不同大小的样本库对模型性能的影响。这有助于探索样本库大小对模型性能的影响。

表3-5展示了消融实验的结果:

表 3-5 使用 DeepSeek-V2-0628 模型进行消融实验的结果

实验设置	MATH-minival	GSM8K
本方法	74.2%	95.1%
不使用错题样本检索	70.2%	93.8%
最大迭代次数=0	63.4%	94.1%
最大迭代次数=2	69.4%	94.2%
最大迭代次数=4	72.6%	94.8%
最大迭代次数=6	74.2%	95.1%
最大迭代次数=8	73.0%	94.9%
最大迭代次数=10	72.2%	93.4%
样本库大小=700	72.0%	92.8%
样本库大小=4897	74.2%	95.1%

从消融实验结果可以看出:

1. 错题样本检索的影响: 实验结果显示, 错题样本检索相比随机样本选择在

MATH-minival 和 GSM8K 数据集上分别提高了 4.0% 和 1.3% 的性能。这证实了错题样本检索策略在提升大语言模型解决数学文字问题时的作用。通过选择与当前问题更相关的样本，模型能够更好地理解和解决特定类型的问题。

2. 多轮纠错机制的影响：将最大迭代次数设置为 0（即不进行多轮纠错）时，在 MATH-minival 和 GSM8K 数据集上测试结果分别下降了 10.8% 和 1.0%。在 MATH-minival 数据集上，这一显著的性能下降凸显了多轮纠错机制在提高模型性能方面的关键作用。通过允许模型多次尝试并修正错误，多轮纠错机制大大提高了大语言模型解决复杂数学问题的能力。但是，在 GSM8K 数据集上，性能下降较小，这可能是因为 GSM8K 数据集的问题相对简单，可以一次性解决，所以多轮纠错机制的效果不如在 MATH-minival 数据集上显著。模型在解决这些问题时，错误较少，所以多轮纠错机制的效果不如在 MATH-minival 数据集上显著。这个结果强调了在数学问题求解过程中，识别和纠正错误的重要性，同时也说明了本章提出的方法在处理复杂问题时的优势。

3. 迭代次数的影响：实验结果表明，最大迭代次数为 6 时，模型在两个数据集上都达到了最佳性能。但是在 GSM8K 数据集上，迭代次数的增加迭代次数的增加并不显著。当迭代次数少于 6 次时，模型性能略有下降，可能是因为没有充分利用迭代改进的机会。而当迭代次数超过 6 次时，性能并没有提升，甚至有轻微下降，这可能是由于错误积累或过度修正导致的。这个结果为选择最优迭代次数提供了重要参考。

4. 样本库大小的影响：增加样本库的大小从 700 到 4897，在 MATH-minival 和 GSM8K 数据集上分别带来了 2.2% 和 2.4% 的性能提升。这表明更大的样本库能够提供更多样化的示例，有助于模型处理各种类型的问题。但是，受限于优质数据的获取难度，样本库的大小不能无限增加，以目前的实验方法，无法获取到更多高质量的样本。在未来，可以通过其他方法获取更多优质样本，增加样本库的大小，进一步提高模型性能。

总体而言，这些消融实验结果强调了错题样本检索和纠错机制的重要性，同时也揭示了迭代次数和样本库大小对模型性能的影响。这些发现不仅验证了本章方法的有效性，也为进一步优化和应用提供了明确的方向。特别是，纠错机制的显著效果表明，在复杂的数学问题求解中，能够识别和修正错误的能力对于提高整体性能至关重要。

3.4.8 错误类型分布统计与分析

基于本研究构建的 4897 个错题样本，本节对样本中出现的 Python 错误类型进行了统计分析，以深入了解大语言模型在解决数学问题时常见的错误模式及其分布特征。这些统计数据有助于理解错误类型与数学推理过程的关联，也为进

一步优化错误检测和纠错策略提供了依据。

3.4.8.1 错误类型分布

图3-7展示了错题样本库中主要错误类型的分布情况。从统计结果看，最常见的五种错误类型依次是：TypeError（类型错误，28.3%）、NameError（名称错误，23.7%）、ValueError（值错误，17.6%）、IndexError（索引错误，9.8%）和ZeroDivisionError（除零错误，8.2%）。其他错误类型如 AttributeError、SyntaxError、OverflowError 等共占 12.4%。

具体的错误类型分布如表3-6所示，本节对样本库中的 4897 个样本进行了全面统计，完整展示了各类错误的具体数量和比例。

表 3-6 错误类型详细分布

错误类型	样本数量	百分比 (%)	累计百分比 (%)	纠错成功率 (%)
TypeError（类型错误）	1386	28.30	28.30	82.7
NameError（名称错误）	1161	23.71	52.01	86.3
ValueError（值错误）	862	17.60	69.61	75.2
IndexError（索引错误）	480	9.80	79.42	72.6
ZeroDivisionError（除零错误）	402	8.21	87.63	78.4
AttributeError（属性错误）	202	4.12	91.75	68.9
OverflowError（溢出错误）	122	2.49	94.24	63.1
RecursionError（递归错误）	106	2.16	96.41	59.8
KeyError（键错误）	68	1.39	97.79	65.2
SyntaxError（语法错误）	42	0.86	98.65	57.4
ImportError（导入错误）	31	0.63	99.29	61.8
UnboundLocalError（未绑定局部变量错误）	17	0.35	99.63	-
RuntimeError（运行时错误）	10	0.20	99.84	-
ModuleNotFoundError（模块未找到错误）	8	0.16	100.00	-

Distribution of Error Types in Samples

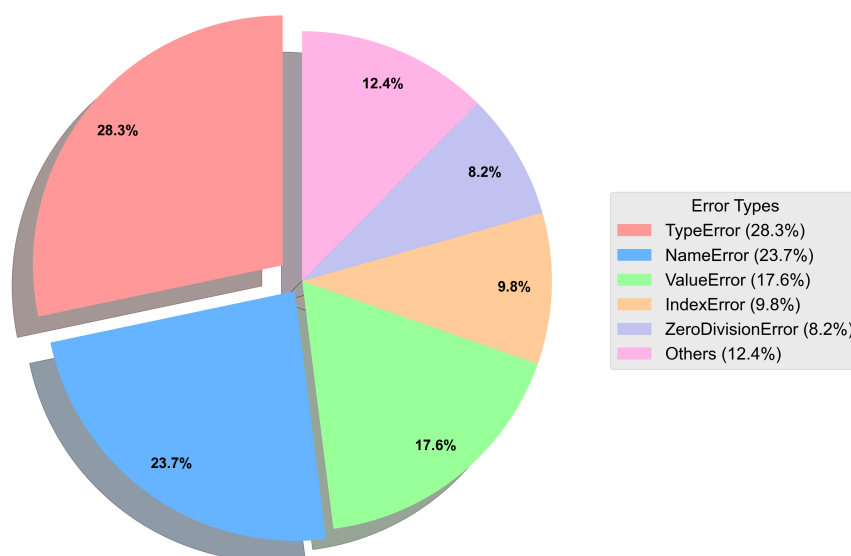


图 3-7 错题样本中主要错误类型分布

3.4.8.2 错误类型与数学领域的关联分析

本节进一步分析了不同错误类型与数学问题领域之间的关联性，如表3-7所示。分析发现，不同数学领域表现出明显不同的错误类型分布特征：

表 3-7 不同数学领域中主要错误类型的分布（%）

数学领域	TypeError	NameError	ValueError	IndexError	ZeroDivisionError
代数	32.6	19.2	15.3	7.4	12.8
几何	25.7	22.9	18.6	12.3	6.5
概率统计	19.8	27.3	22.1	15.7	3.2
微积分	30.1	21.5	17.9	8.6	10.2
数论	33.4	24.7	14.2	5.3	8.9

1. 在代数和数论问题中，TypeError 出现频率最高，分别为 32.6% 和 33.4%，主要涉及数值类型转换和符号运算中的类型不匹配问题；
2. 概率统计问题中 NameError 更为常见（27.3%），这通常与模型在处理概率分布和统计函数时变量命名混淆有关；
3. 几何问题中 IndexError 比例相对较高（12.3%），主要与坐标处理和向量运算中的索引错误相关；
4. ZeroDivisionError 在代数和微积分问题中较为突出，分别为 12.8% 和 10.2%，这与方程求解和极限计算中的分母为零情况有关。

通过深入分析，发现错误类型与数学领域之间存在明显的内在联系，这些关联性主要体现在数学问题的特性和解题方法上：

- **TypeError（类型错误）**：在代数和数论问题中尤为常见，主要因为这些领域涉及大量的符号运算和类型转换。例如，复数运算、分数与浮点数转换、符号计算等场景中，模型可能错误地混用不同数据类型，导致运算不兼容。典型案例包括尝试对字符串执行数学运算，或在代数简化中错误处理不同类型的表达式。
- **NameError（名称错误）**：在概率统计问题中比例最高，这主要源于处理概率分布和统计函数时的变量命名复杂性。统计问题通常涉及多个分布参数、随机变量和条件概率，导致模型在引用变量时容易出错。例如，在条件概率计算中混淆事件表示，或在多步骤推导中变量名不一致。
- **ValueError（值错误）**：在概率统计问题中也较为常见，通常发生在函数输入值不在有效范围内的情况。例如，尝试计算负数的对数、概率值超出 [0,1] 范围、使用不合理的分布参数等场景。
- **IndexError（索引错误）**：在几何和概率统计问题中占比较高，主要与处理多维数据结构有关。几何问题常涉及坐标、向量和矩阵操作，而模型可能错误引用超出边界的索引位置。例如，在计算多边形面积或三维空间变

换时的索引越界问题。

- **ZeroDivisionError (除零错误)**: 在代数和微积分问题中更为常见, 这与处理分式和极限时的边界情况密切相关。例如, 在求解方程系统、计算渐近线或处理有理函数时, 模型未能正确处理分母为零的特殊情况。
- **OverflowError 和 RecursionError**: 这两类错误在递归算法和大数计算中较为常见, 如在数论中处理大整数质因数分解、组合计算中的大组合数, 以及使用递归方法解决问题时出现的栈溢出。

这些关联性为未来优化错误检测和纠错策略提供了重要依据, 通过考虑数学领域特性, 可以更有针对性地构建和检索错题样本, 提高纠错效率。

3.4.9 案例分析

为了更直观地展示本方法的效果, 本章选取了一个具有代表性的案例进行分析。这个案例来自 MATH 数据集, 属于代数问题。如图3-8所示, 这是一个关于代数表达式简化的问题。问题和提示输入大语言模型, 大语言模型先对问题进行理解, 然后生成 Python 代码, Python 解释器进行代码执行, 返回错误类型 `TypeError`, 从样本库中检索, 将具有相同错误类型和问题的语义相似度高的几个推理过程反馈给模型, 可以看到检索到了一个同样是代数表达式简化的问题, 并且这条推理过程成功修正了这个问题, 模型根据这个推理过程作为样本, 修正了代码, 得到了正确答案。

3.5 本章小结

本章提出了一种基于错误类型检索的大语言模型数学问题多轮纠错方法, 无需微调, 通过提示工程实现多轮对话。该方法结合思维链和程序辅助学习的混合方式, 构建了基于错误类型的错题样本库, 设计多轮纠错机制, 并实现基于错误类型的检索式样本选择策略。实验表明, 该方法在 MATH-minival 和 GSM8K 基准数据集上性能显著提升, 优于现有基线模型。本章详述了错题样本构建方法, 包括样本结构、生成方法、质量控制和选择, 并介绍了基于 Milvus 数据库的实现方法。通过消融实验验证了方法有效性, 结果表明错题样本检索显著提升模型性能, 适当迭代次数(6次)能在性能和效率间取得最佳平衡, 合理增加样本库大小有助于提高性能。这些发现为优化样本选择策略、改进迭代过程及拓展样本库提供了重要指导, 有助于进一步提高大语言模型解决数学文字问题的能力。

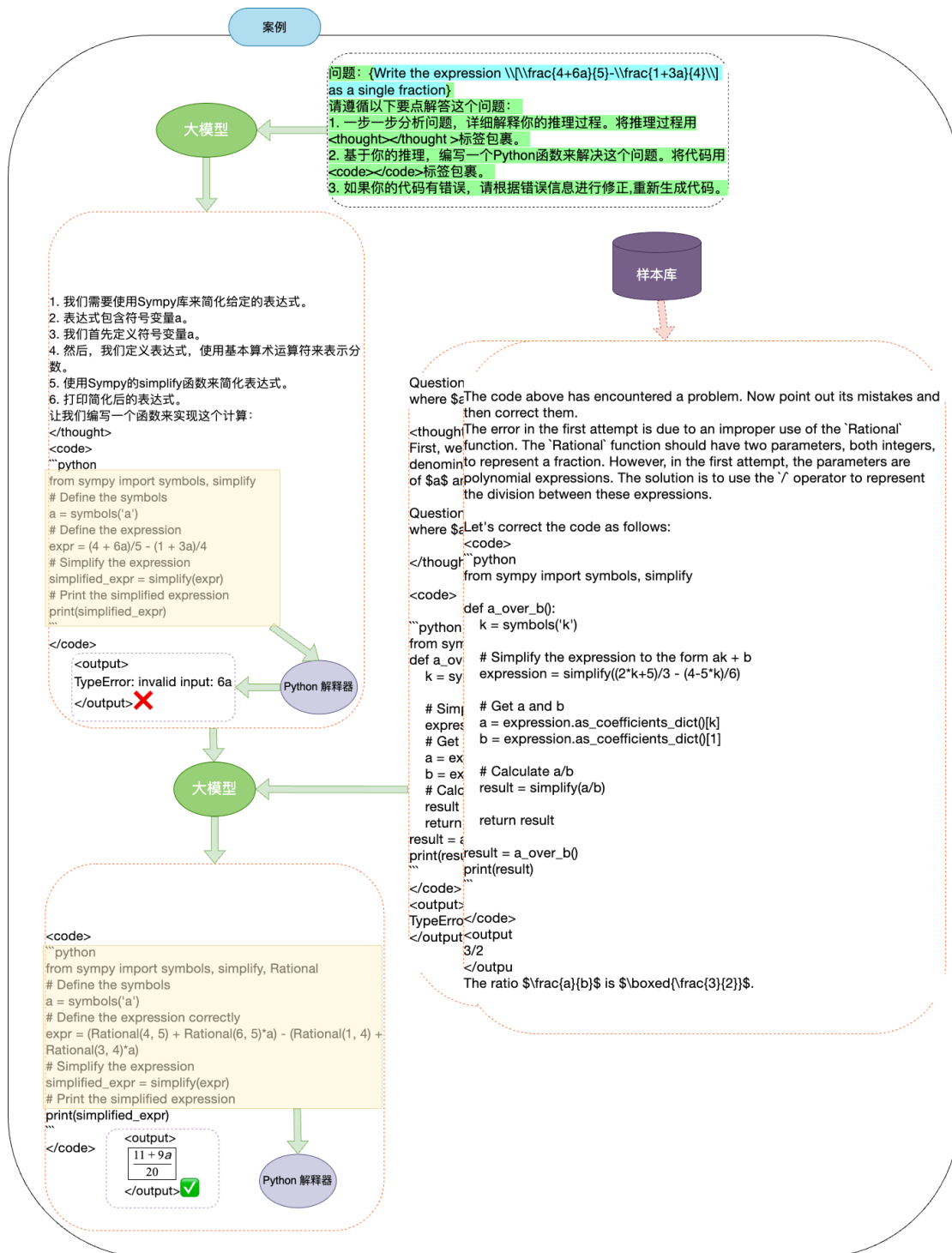


图 3-8 案例分析

第4章 基于自学习大语言模型智能体的数学文字问题求解

4.1 引言

本文在第三章中详细介绍了基于错误类型错题样本的多轮程序辅助纠错提示方法。该方法通过构建结合思维链和程序辅助学习的错题样本库，并利用错题样本检索策略，提升了大语言模型在数学文字问题求解上的表现。根据第三章的消融实验，错题样本库的大小对结果有着正向的影响，但是受限于高质量的数学文字问题规模有限，如何持续创建高质量的数学文字问题存在挑战。

而智能体^[78]作为一类以感知、执行、记忆三大模块为核心的人工智能系统，其感知和执行模块能够模拟人类的学习过程，从经验中学习，并不断更新自己的知识库。这恰好弥补了第三章方法的不足。

现有专门处理数学问题的智能体大多是通过监督微调（SFT）大语言模型的方法实现的，如 Tora^[22]，WizardMath^[26] 等，微调虽然能将一个原本数学能力不足的大语言模型提升到较高的水平，但是仍然存在一些局限性，如成本过高，无法持续学习等。每次的提升都需要对模型进行微调，这不仅需要大量的算力，而且微调后的模型可能无法适应其他类型的问题。具体来说，Ghosh 等人^[79]的研究发现全参数微调会导致知识退化，降低整体响应质量。微调后的模型更容易产生幻觉。微调后的模型在特定任务上的表现可能会提高，但牺牲了通用性，导致在其他领域的表现下降。

在不使用微调，调用基座大语言模型的情况下，数学领域最著名的专用智能体之一是 MathChat^[40]，其因为被集成在开源智能体框架 AutoGen^[80] 中，而广为人知。MathChat 其具有规划，纠错，调用工具等能力，通过提示工程的方法，在已有大语言模型上进行增强。但其不具有持续学习的能力，无法从历史解题经验中学习，记忆机制存在局限性。

为了克服这些限制，本章在第三章工作基础上，进一步提出了一个基于自学习智能体的数学问题求解框架。该框架不使用微调，调用闭源大语言模型作为智能体的大脑，通过提示工程的设计，最大程度上激发大语言模型本身的能力，并且通过记忆模块，实现持续学习。这个框架将整个系统视为一个能够自主学习的智能体，通过持续收集题目、解题、积累经验和更新知识，不断提升自身的问题解决能力。这种方法不仅继承了第三章方法的优点，引入了学习的机制，使系统能够从历史成功路径中学习，并在面对新问题时更加灵活和高效。

本章的主要创新性贡献包括：

1. 为进一步提升大语言模型在数学文字问题的表现，在第三章方法的基础上，为缓解智能体架构的局限性，形成更加系统化的解题策略，建立了自主选择解题策略的智能体架构，分为感知，执行，记忆三大模块。

- 在感知模块中，为持续构造高质量数学文字问题，解决数据的局限性的挑战，感知模块基于原有错题样本库，利用大语言模型的生成能力持续构造数学文字问题，并利用多模型自一致性进行质量控制。
- 在记忆模块中，记忆模块通过构建单次样本库和多次样本库，以及相应的记忆更新策略，通过语义相似度、代码结构差异和错误类型等多维比较，有效维护记忆库的多样性和准确性，实现“错题-分析-修正-强化”的闭环学习，使得智能体的记忆机制的局限性得到缓解。
- 在执行模块中，提出了基于样本动态检索的多策略融合方法，在解题阶段灵活选择多轮或单轮交互，以及相似度匹配或错误类型分析等策略，利用历史解题经验辅助当前问题的求解。

2. 在实验中，该智能体不仅在持续构造高质量数学文字问题上表现出更好的质量和增长性，有效缓解了数据的局限性和智能体的记忆机制的局限性，并且较之基线方法，在 MATH-minival 和 GSM8K 数据集上具有更高的解题准确性。

本章所用术语如表4-1所示：

表 4-1 第四章术语表

术语	解释
多模型自一致性验证	通过多个大语言模型生成答案并比较一致性，用于验证问题答案的正确性和筛选高质量样本。
自学习智能体	包含感知、记忆和执行模块的智能系统，能够通过持续学习和记忆更新提升问题求解能力。
记忆模块	存储解题经验和样本的模块，通过动态更新维护样本库的多样性和准确性。
执行模块	负责解题的核心模块，采用多策略融合方法（如相似度匹配和错误类型分析）进行问题求解。
感知模块	负责生成和验证数学问题的模块，通过数据增强和多模型验证确保问题质量。
动态样本检索	在解题过程中根据当前问题的语义相似度和错误类型动态选择样本，用于上下文学习和错误修正。
样本更新策略	用于维护样本库多样性和准确性的策略，包括语义相似度、代码结构差异和错误类型等多维度比较。
数据污染检测	检测样本库中是否存在与测试集高度相似的样本，确保实验结果的公平性和可靠性。
多策略融合方法	结合多种解题策略（如单轮和多轮交互、相似度匹配和错误类型分析）的方法，灵活选择最优策略进行问题求解。
抽象语法树（Abstract Syntax Tree, AST）	代码的结构化表示，用于比较代码的结构差异。

4.2 问题定义和方法概览

4.2.1 问题定义

本章将数学文字问题求解任务定义为一个持续学习的过程。具体来说，定义以下几个关键组件：

1. 数学文字问题集合 $P = \{p_1 p_2 \dots p_n\}$ ，其中每个问题 $p_i = (q_i a_i)$ 包含问题描述 q_i 和标准答案 a_i 。

2. 自学习智能体 A ，它包含三个主要模块：- 感知模块 S ：负责生成和验证数学文字问题 - 执行模块 E ：负责解决数学文字问题 - 记忆模块 M ：存储和更新解题经验

3. 错题样本库 $K = \{k_1 k_2 \dots k_m\}$ ，其中每个错题样本 k_i 包含问题、解题过程和相应答案。

4. 问题序列 $T = \{t_1 t_2 \dots\}$ ，表示智能体处理的连续问题序列。

对于每个问题 t ，智能体的任务是：1. 通过感知模块 S 收集新的数学问题： $p_t = S(t)$ 2. 利用执行模块 E 和当前错题样本库 K_{t-1} 解决问题： $\hat{a}_t = E(p_t K_{t-1})$ 3. 更新记忆模块 M ，将新的解题经验添加到错题样本库中： $K_t = M(K_{t-1} p_t \hat{a}_t)$

智能体的目标是随着处理问题数量的增加，不断提高其解决数学问题的能力，直到其达到最优水平，即： $\lim_{t \rightarrow \infty} \text{Accuracy}(AP) \rightarrow \text{最优水平}$

其中， $\text{Accuracy}(AP)$ 表示智能体 A 在问题集 P 上的解题准确率。

这个定义强调了智能体的持续学习特性，通过不断接触新问题、解决问题并更新错题样本库，来提升其数学问题求解能力。本章中说述的错题样本含义与第三章中的所述相同。

4.2.2 整体架构

如图4-1所示，自学习智能体框架主要由感知模块、执行模块和记忆模块三大部分组成，它们相互协作，共同完成数学文字问题的求解过程。

1. 感知模块作为智能体与外部环境交互的接口，主要负责题目的搜集和验证。其中，问题生成器用于创建新的数学问题，所使用的方法有数据集问题改写，大语言模型生成两种。而验证器则对问题和答案进行验证，以确保其正确性和合理性，对于基于数据集改写的问题使用数据集中的标准答案进行验证；对于大语言模型生成的题目，使用本文提出的多模型自一致性进行验证。感知模块为执行模块提供高质量的问题输入。

2. 执行模块是智能体解题的核心，采用多策略融合的方法进行解题。执行模块会根据问题的特点和解题阶段，灵活选择策略，具体地，在初次解题时，根据

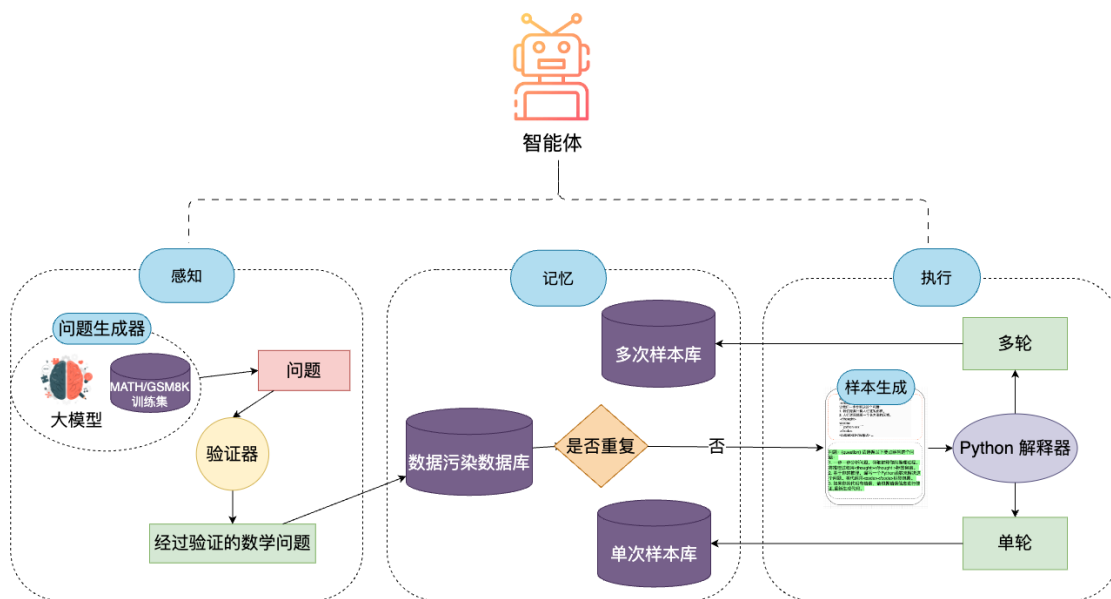


图 4-1 自学习智能体框架的整体架构

问题的语义相似度匹配正确单次推理正确的样本；在首次执行代码出错后，使用返回的错误类型和语义相似度共同选择包含错误修正过程的错题样本，并再次执行代码，直到正常运行或者达到最大尝试次数。执行模块的输出结果会反馈给记忆模块，用于更新知识库。

3. 记忆模块是智能体的长期记忆，以动态样本库的形式存储解题经验。它主要由两部分组成：单次样本库，存储一次成功解题的样本；多次样本库，存储多次尝试后解题成功的错题样本，并记录解题过程中的错误类型。记忆模块会根据执行模块的反馈，采用不同的策略动态更新样本库，以保证样本库的多样性，同时进行数据污染检测，以确保样本质量。

在整个求解过程中，这三个模块紧密协作，形成一个完整的循环：感知模块搜集和验证数学文字问题，执行模块进行解题并生成结果，记忆模块根据执行结果更新样本库。这种设计使得智能体能够在问题求解中不断积累经验，并利用这些经验指导后续的解题，从而实现持续学习和自我改进。

4.3 自学习智能体框架设计

4.3.1 智能体感知模块：题目生成与验证

感知模块是智能体与外部环境交互的重要接口，其设计思想是模拟人类的感知过程，通过主动搜集和理解外界信息，为执行模块提供高质量的输入。具体来说，感知模块主要负责题目的搜集和验证两大任务。在题目搜集方面，感知模块采用了多源搜集策略，既通过改写公开数据集扩充已有题库，又通过问题生成器

主动创建新题目。这种主动感知的方式大大拓宽了题目来源，提高了题目的多样性和丰富度。在题目验证方面，感知模块引入了基于标准答案和多模型自一致性的验证机制，通过对比生成答案与标准答案或多模型一致性，筛选出高质量的题目，从而为执行模块提供可靠的输入。总体而言，感知模块通过主动搜集和严格筛选，充分发挥了智能体的“感知”功能，为整个系统的运作奠定了坚实的基础。

4.3.1.1 问题搜集

为了确保智能体能够接触到多样化和高质量的数学文字问题，本章设计了多源题目搜集策略：

1. 改写公开数据集：以 MATH、GSM8K 等标准数学问题数据集的验证集作为种子数据，利用大语言模型对这些种子数据进行重写、改写，并利用数据集中的标准答案进行验证，这是扩充数据集的常用方法之一。这种方法可以让数据集得到扩充，让基于语义相似度的检索器更加准确，从而提升泛化能力，但是生成的数量有限。

具体方法是将公开数据集中的问题作为输入，使用大语言模型进行重写和改写。重写过程中，本方法保持问题的核心概念和难度不变，但改变其表述方式、背景，以此提高多样性。

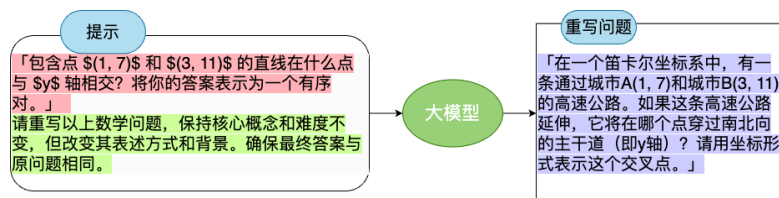


图 4-2 问题重写样例

如图4-2所示，这个新问题保持了原问题的核心数学概念（直线方程和坐标系）和具体数值不变，但改变了具体的场景和描述方式。这种重写方法既保证了问题的多样性，又确保了其与原问题在数学本质和计算结果上的完全一致性。通过将抽象的数学概念转化为现实世界的场景（如高速公路和城市），这种方法不仅扩大了问题库的规模，还有助于提高模型的泛化能力和应用能力。

2. 问题生成器：利用大语言模型直接生成新的数学文字问题。这种数据相比于公开数据集的特征是没有标准答案，所以无法用标准答案进行验证题目是否有效。使用的提示词如图4-3所示：

以上两种问题生成方法是数学问题生成和数据增强的常见方法，但通常情况下，对于生成的问题质量难以保证。特别是对于大语言模型生成的问题，可能存在以下问题：1. 问题不可解：生成的问题可能缺少关键信息或包含矛盾信息，导

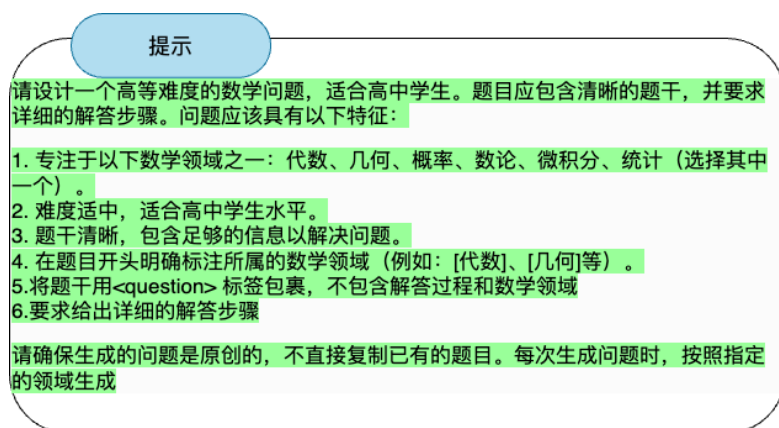


图 4-3 问题生成提示词

致无法求解。2. 问题重复：生成的问题可能与已有问题高度相似或完全相同，降低了数据的多样性。3. 难度不适：生成的问题难度可能过高或过低，不符合目标难度范围。4. 表述不清：问题描述可能存在歧义或语法错误，影响理解和求解。因此，为了确保智能体接收到高质量的输入，需要对生成的问题进行严格的验证和筛选。这个验证过程不仅能够提高题库的质量，还能帮助优化问题生成器的性能。下一节将详细介绍问题验证的方法和流程。

4.3.1.2 问题验证

如上文所述，问题的搜集分为两种：改写公开数据集和大语言模型生成，这两种方式本质的不同在于是否具有标准答案。有标准答案的题目可以利用答案验证生成的题目是否有效，而没有答案的题目需要用过其他的方法验证。接下来分别介绍这两种不同情况所使用的验证方法：

1. 对基于公开数据集改写的题目，使用标准答案进行验证。这个验证过程可以用以下公式表示：

$$V(aa^*) = \begin{cases} 1 & \text{if } f(a) \equiv f(a^*) \\ 0 & \text{otherwise} \end{cases} \quad (4-1)$$

其中， a 是模型生成的答案， a^* 是标准答案， $f(\cdot)$ 是一个标准化函数，用于处理等价但表达不同的答案（如分数和小数的转换），这个函数可以在第三章中的节3.3.3.1中找到。 $V(aa^*)$ 的值为 1 表示验证通过，0 表示验证失败。

2. 对于来自问题生成器的问题，使用大语言模型生成伪答案，并使用改进的自一致性方法进行验证。

本研究采用了一种基于多模型自一致性的验证方法，如图4-4所示，这是对传统自一致性方法^[19]的扩展。本章使用多个不同的大语言模型来生成伪答案，并比较它们的结果。需要强调的是，这些生成的答案并非真正的标准答案，而是

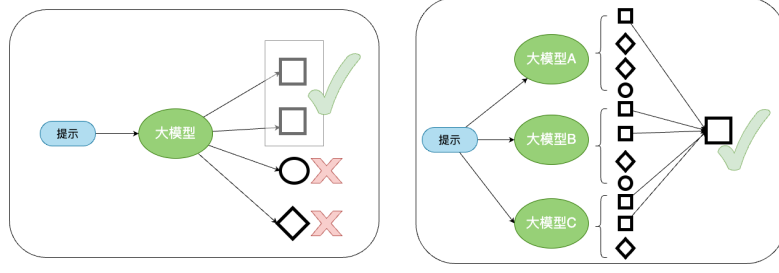


图 4-4 自一致性和多模型自一致性对比

大语言模型基于问题内容推断出的可能答案。形式化表示如下：

$$a^* = \arg \max_a \left(\sum_{m=1}^M \sum_{i=1}^K \mathbb{I}(\Omega_{mi} = a) \right) \quad (4-2)$$

其中， M 是使用的大语言模型数量， K 是每个模型生成的伪答案数量， Ω_{mi} 表示第 m 个模型生成的第 i 个伪答案。

本章选择了三个不同的大语言模型 ($M = 3$)，每个模型生成 5 个伪答案 ($K = 5$)。最终的伪正确答案是所有生成伪答案中出现次数最多的答案，且该次数需要超过预设阈值。具体来说：

1. 每个模型生成 5 个伪答案，总共 15 个伪答案。
2. 统计这 15 个伪答案中每个不同答案出现的次数。
3. 选择出现次数最多的伪答案，记为 a_{max} 。
4. 如果 a_{max} 的出现次数超过预设阈值 T (例如， $T = 7$)，则将 a_{max} 视为伪正确答案。
5. 如果没有伪答案的出现次数超过阈值 T ，或者有多个伪答案同时达到最高出现次数，则舍弃该问题。

这种方法可以表示为：

$$a^* = \begin{cases} a_{max} & \text{if } \max_a \left(\sum_{m=1}^M \sum_{i=1}^K \mathbb{I}(\Omega_{mi} = a) \right) > T \\ \text{舍弃} & \text{otherwise} \end{cases} \quad (4-3)$$

这种改进的自一致性方法不仅考虑了单个模型的自一致性，还引入了多模型间的一致性检验，并通过设置最小一致性阈值来进一步提高验证的可靠性。通过增加每个模型生成的伪答案数量到 5 个，进一步提高了样本量，使得最终选择的多数伪答案更具代表性和可靠性。这种方法在保持计算效率的同时，显著提升了验证结果的准确性和稳定性，同时也能有效地过滤掉那些模型间存在较大分歧的问题。需要注意的是，这种方法生成的是伪答案，而非真正的标准答案，其主要目的是为了验证问题的可解性。本章将其与自一致性方法的对比，放在节 4.4.2.1 中进行。

4.3.2 智能体执行模块：解题策略选择与自动做题系统

执行模块是智能体解题的核心，其设计思想是模拟人类的思考和解题过程，通过灵活选择解题策略和高效执行解题步骤，实现自动化的数学问题求解。具体来说，执行模块采用了多策略融合的方法，根据问题的特点和解题阶段，动态选择相似度匹配或错误类型分析等策略，以适应不同类型的问题和解题需求。同时，执行模块与记忆模块紧密交互，利用历史解题经验辅助当前问题的求解，实现了知识的迁移和复用。在解题过程中，执行模块通过记忆模块中的样本，选择相似度匹配或错误类型分析等策略，以适应不同类型的问题和解题需求。执行结果会实时反馈给记忆模块，用于动态更新知识库，形成了一个闭环的自我学习和优化过程。总体而言，执行模块通过灵活的策略选择和高效的解题执行，充分发挥了智能体的“思考”和“行动”能力，是整个系统智能化的关键。

4.3.2.1 多策略融合方法

智能体的执行模块采用多策略融合方法来选择最适合的解题策略，并实现自动做题系统。根据初次解题阶段和纠错阶段的不同，分别提供单次解题样本和错题样本。其中单次解题样本是指一次生成代码直接成功的做出正确回答的样本，错题样本的含义与第三章的错题样本相同。这种方法结合了相似度匹配和错误类型分析，以适应不同的解题阶段：

1. 初次解题阶段：在第一次尝试解决问题时，执行模块主要根据问题的语义相似度在单次样本库中匹配样本。这可以表示为如公式4-4所示：

$$S_{best} = \arg \max_{S_j \in \mathcal{S}} \text{sim}(g(Q)g(Q_{S_j})) \quad (4-4)$$

其中， Q 是查询的问题， $g(\cdot)$ 是问题的向量表示函数， $\mathcal{S}$ 是样本库， $\text{sim}(\cdot \cdot)$ 是相似度函数。

2. 纠错阶段：如果初次解题失败，进入纠错阶段。此时，执行模块首先考虑在多次样本库中匹配问题的语义相似度，与初次解题阶段方法相同，区别在于所查找的样本库的不同。然后在满足语义相似度阈值的样本中匹配错误类型，形式化的表述如公式4-5所示：

$$S_{best} = \arg \max_{S_j \in \mathcal{S}'} \mathbb{I}(E_{new} = E_{S_j}) \quad (4-5)$$

其中， $\mathcal{S}' = \{S_j \in \mathcal{S} | \text{sim}(g(Q_{new})g(Q_{S_j})) > \theta\}$ 是满足语义相似度阈值 θ 的样本子集， E_{new} 是新问题的错误类型， E_{S_j} 是样本 S_j 的错误类型， $\mathbb{I}(\cdot)$ 是指示函数。这个公式首先筛选出相似度高于阈值的样本，然后在这些样本中选择错误类型

匹配的样本。如果有多个错误类型匹配的样本，可以进一步考虑它们的语义相似度：

$$S_{best} = \arg \max_{S_j \in S''} \text{sim}(g(Q_{new})g(Q_{S_j})) \quad (4-6)$$

其中， $S'' = \{S_j \in S' | \mathbb{I}(E_{new} = E_{S_j}) = 1\}$ 是错误类型匹配的样本子集。

这种多策略融合方法使得智能体能够在不同的解题阶段灵活选择最相关的样本，从而提高解题效率和准确性。初次解题时，它侧重于问题的整体相似性；而在纠错阶段，它会更多地考虑具有相同错误类型的样本，以便更有针对性地解决问题，这部分的有效性在第三章中得到验证。

4.3.2.2 基于第三章方法的解题流程集成

执行模块是智能体解决数学问题的核心，集成了第三章提出的基于错误类型检索的多轮程序辅助纠错提示方法，并进行了进一步的优化：

在初次解题时，使用相似度匹配样本，而在纠错时，使用相似度和错误类型同时匹配样本。

根据执行结果有三种情况：

1. 一次得到正确结果：将该路径加入到单次样本库中，并记录该问题解题过程的上下文。
2. 经过纠错得到正确结果：将该路径加入到多次样本库中，并记录该问题解题过程的上下文。
3. 纠错超过尝试限制：舍弃该路径。

4.3.3 智能体记忆模块：成功解题的记录

记忆模块是智能体的长期记忆，其设计思想是模拟人类的记忆系统，通过动态存储和更新解题经验，为执行模块提供智能化的知识支持。具体来说，记忆模块采用了样本库的形式，包括单次样本库和多次样本库，分别存储一次成功解题和多次尝试后解题成功的样本。样本不仅包含问题和答案，还记录了详细的解题思路和错误类型等信息，为执行模块提供了可供检索的信息。在解题过程中，记忆模块根据执行模块的需求，实时检索和提取相关样本，辅助问题的求解。同时，记忆模块会根据执行模块的结果，动态更新样本库，实现了知识的持续积累和优化，并且在更新样本时采取一定的更新机制，保证样本库的多样性。此外，记忆模块还引入了数据污染检测机制，通过与外部测试集的比对，及时发现和清除可能影响评测结果的污染数据，保证了测试结果的公平性。总体而言，记忆模块通过更新机制和污染检测机制，充分发挥了智能体的“记忆”功能，为整个系统提

供了知识支撑和自我优化能力。

4.3.3.1 样本库字段

本文中样本库以数据库形式保存，具体的数据库设置参考第三章相关内容。单次样本库的字段为：问题编号，问题，问题向量表示，解决思路；多次样本库的样本字段为：问题编号，问题，问题向量表示，解决思路，错误类型。其中各字段含义如下：

- 问题编号：用于唯一标识每个样本的 ID
- 问题：原始的数学问题文本
- 问题向量表示：使用向量编码后的问题表示，用于计算问题语义相似度
- 解决思路：包含解题过程中的推理步骤和代码实现
- 错误类型：仅在多次样本库中存在，记录代码执行后 Python 解释器返回的错误类型

4.3.3.2 样本更新机制

在智能体执行模块成功解题之后，需要将该问题添加到样本库中。而问题生成器可能产生相同或者类似的题目，为了保证样本库的多样性，需要对样本的更新规则进行明确。

1. 对于单次样本库，当智能体成功解题之后，需要将该问题与样本库中的问题进行相似度比较。如果相似度超过一定阈值，则进一步比较代码的结构。具体的方式可以形式化表示为：

首先，计算语义相似度：

$$\text{Sim}(Q_{S_{\text{new}}} Q_{S_j}) = \text{Sim}(g(Q_{S_{\text{new}}})g(Q_{S_j})) \quad (4-7)$$

其中， $g(\cdot)$ 表示问题的向量表示函数， $Q_{S_{\text{new}}}$ 是新添加的样本， Q_{S_j} 是样本库中的已有样本。

如果语义相似度超过阈值 ϵ_1 ，则进一步比较代码结构：

$$\text{AST_Diff}(C_{S_{\text{new}}} C_{S_j}) > \epsilon_2 \quad (4-8)$$

其中， $\text{AST_Diff}(\cdot \cdot)$ 表示两个代码的抽象语法树^[81] (AST) 的最短树编辑距离^[82]， $C_{S_{\text{new}}}$ 和 C_{S_j} 分别表示新样本和已有样本中的代码， ϵ_2 是预设的树编辑距离阈值。

综合这两个条件，可以得到样本接受的判断标准：

$$\forall j(\text{Sim}(Q_{S_{new}} Q_{S_j}) < \epsilon_1) \vee (\text{Sim}(Q_{S_{new}} Q_{S_j}) \geq \epsilon_1 \wedge \text{AST_Diff}(C_{S_{new}} C_{S_j}) > \epsilon_2) \quad (4-9)$$

公式4-9表示新样本要么与所有已有样本的语义相似度都低于阈值 ϵ_1 ，要么在语义相似度高于 ϵ_1 的情况下，代码结构的差异大于阈值 ϵ_2 。

这种方法能够更精确地维护样本库的质量和多样性。它不仅考虑了问题的语义相似度，还在高相似度的情况下进一步比较了代码结构的差异，从而更好地确保了智能体学习过程中样本的多样性，同时避免了过于相似的样本重复添加到库中。

2. 对于多次样本库，本章采用不同的更新策略。由于多次样本库中可能存在多个解题过程，不能直接比较代码结构。因此，本章主要关注问题的语义相似度和错误类型。具体来说：

(1) 首先计算新样本与库中已有样本的语义相似度：

$$\text{Sim}(Q_{M_{new}} Q_{M_j}) = \text{Sim}(g(Q_{M_{new}})g(Q_{M_j})) \quad (4-10)$$

其中， $Q_{M_{new}}$ 是新添加的多次样本， Q_{M_j} 是多次样本库中的已有样本。

(2) 如果语义相似度超过预设阈值 ϵ_3 ，则进一步比较错误类型：

$$\text{ErrorType}(E_{M_{new}}) = \text{ErrorType}(E_{M_j}) \quad (4-11)$$

其中， $\text{ErrorType}(\cdot)$ 表示获取样本中第一个错误类型的函数， $E_{M_{new}}$ 和 E_{M_j} 分别表示新样本和已有样本中的错误类型。

(3) 综合这两个条件，可以得到多次样本库的样本接受判断标准：

$$\forall j(\text{Sim}(Q_{M_{new}} Q_{M_j}) < \epsilon_3) \vee (\text{Sim}(Q_{M_{new}} Q_{M_j}) \geq \epsilon_3 \wedge \text{ErrorType}(E_{M_{new}}) \neq \text{ErrorType}(E_{M_j})) \quad (4-12)$$

这个判断标准确保了新样本要么与所有已有样本的语义相似度都低于阈值 ϵ_3 ，要么在语义相似度高于 ϵ_3 的情况下，第一个错误类型与已有样本不同。这种方法可以有效地维护多次样本库的多样性，同时避免添加过于相似且具有相同错误类型的样本。

4.3.3.3 数据污染检测

在更新样本库的过程中，需要特别注意防止数据污染问题的发生。这一问题与大语言模型预训练阶段可能出现的数据污染^{[83][84]}有相似之处。在预训练阶段，如果训练语料中包含了测试集的数据，模型可能会在训练过程中“记住”这些数

据,从而影响最终的评测结果的可靠性。同样地,在更新样本库的过程中,如果不进行适当的数据污染检测,可能会导致样本库中包含测试集的数据。这种情况下,在评测阶段,模型可能会在样本库中匹配到原题,从而提高评测结果,影响实验的真实性和可信度。为了有效防止这种数据污染,本章使用一个专门的样本库来进行数据污染检测。这个特殊的样本库应该包含各个常用测试集中的题目。具体到本研究中,由于本章使用 MATH-minival 和 GSM8K 作为测试数据集,因此这个样本库应该包含这两个数据集测试集中的所有题目。在每次向样本库添加新样本时,本章实验都会利用这个特殊的样本库进行数据污染检测。如果检测到新样本与测试集中的题目高度相似,样本库就会拒绝添加该样本,以确保样本库的纯净性和实验结果的可靠性。这种方法能够有效地维护样本库的质量,同时保证评测结果的真实性和可信度。具体的检测方法是通过将目标问题与样本库中的问题进行语义相似度匹配,如果相似度超过一定阈值,则拒绝该样本。形式化的表示如下:

$$\text{Polluted}(Q_{\text{new}}) = \begin{cases} 1 & \text{if } \exists Q_i \in \mathcal{D} \text{Sim}(Q_{\text{new}}Q_i) \geq \theta \\ 0 & \text{otherwise} \end{cases} \quad (4-13)$$

其中, Q_{new} 是待检测的新样本, $\mathcal{D}$ 是包含测试集题目的污染检测样本库, $\text{Sim}(\cdot, \cdot)$ 是语义相似度函数, θ 是预设的相似度阈值。当 $\text{Polluted}(Q_{\text{new}}) = 1$ 时,表示检测到数据污染,该样本将被拒绝;否则,样本可以被接受。

需要注意的是,数据污染检测与样本更新机制有所不同。数据污染检测采用较高的相似度阈值,并且仅考虑语义相似度,而不涉及代码结构和错误类型的比较。在数据污染检测领域, n-gram 是一种常用方法,如 Shao 等人^[25]的研究中所采用的方法。然而,本章中的数据污染检测与预训练阶段的检测有所不同。在本研究中,样本库中的样本是作为完整的问题示例使用,而非作为预训练的语料。只有完整的题目才能产生正确的解题路径,部分语句的匹配并不构成数据污染。基于这一特点,本章采用了基于语义相似度的匹配方法进行数据污染检测。这种方法不仅简单有效,而且足以检测出潜在的数据污染问题,确保了样本库的纯净性和实验结果的可靠性。

4.4 实验及结果

4.4.1 实验设置

本章所用进行的实验所用的大语言模型与第三章相同,使用的大语言模型为 DeepSeek-V2-0628, moonshot-v1-8k, doubao-pro-32k。

实验的具体设置如下:

在实验中，本实验对各个阈值进行了如下设置：

- 单轮样本库相似度阈值： $\epsilon_1 = 0.85$
- 单轮样本库树编辑距离阈值： $\epsilon_2 = 20$
- 多轮样本库相似度阈值： $\epsilon_3 = 0.8$
- 数据污染检测阈值： $\theta = 0.95$

这些阈值的选择基于预实验结果和经验值。具体而言：

- ϵ_1 和 ϵ_2 的设置确保了单轮样本库中样本的多样性，同时避免了过于相似的样本被重复添加。
- ϵ_3 的设置允许多轮样本库在保持多样性的同时，接受一定程度的相似样本，只要它们的错误类型不同。
- θ 被设置为一个较高的值，以确保只有极其相似的问题才会被视为潜在的数据污染，从而最大程度地保护样本库的纯净性。

这些阈值的设置在实验中表现良好，但在实际应用中可能需要根据具体任务和数据集特性进行微调。实验中使用的模型参数设置如下：在所有基线方法中，Temperature 设为 0，Top-p 设为 1.0，最大生成 token 数为 2048。在本方法中：在生成样本的过程中，为了让结果更加多样化，使用 Temperature 为 0.7，在评测时，Temperature 设为 0，Top-p 设为 1.0，最大生成 token 数为 2048。迭代设置方面，最大纠错迭代次数为 6，每次代码执行的超时时间为 10 秒。程序执行环境使用 Python 3.10 版本，并额外引入 numpy 和 sympy 库。为确保结果的可靠性，每个数据集上的实验重复 5 次，取平均值作为最终结果。消融实验使用模型为 DeepSeek-V2-0628。

4.4.1.1 数据集与评价指标

本章内容是在第三章的基础上进行的，重点是为了突出智能体进行持续自学习产生的进步，所以数据集与第三章相同，均使用 MATH-minival 和 GSM8K 数据集，而评价指标则使用准确率。

$$\text{Accuracy} = \frac{\text{Correct Answers}}{\text{Total Answers}} \quad (4-14)$$

4.4.1.2 基线方法

本章的实验设置了以下几个基线方法，以便与本章提出的自学习智能体框架进行比较：

1. MathChat: MathChat 是一个专门用于处理数学问题的智能体，其方法集成在著名智能体框架 AutoGen 当中用于处理数学文字问题。其主要的逻辑是让

大语言模型判断需要的解决数学的方法，有以下三种选择：（1）直接编写 Python 程序：对应于单阶段工具使用方法。（2）直接解决：利用大语言模型代理自身的推理能力解决问题。（3）分步解决并使用 Python：如果前两种方法不适用，则选择这种方法进行多步推理和工具使用，并要求大语言模型代理处理程序运行中的错误和意外结果。

与本章智能体的区别在于其不具备记忆模块和错题样本的策略，并且在执行方式上也存在差异。MathChat 主要依赖于预定义的解题策略和大语言模型的推理能力，而本章提出的智能体通过动态样本检索和多轮纠错来增强解题能力。此外，MathChat 缺乏持续学习的机制，无法像本章的智能体那样通过累积经验来不断改进性能。作为最著名的解决数学问题的专用智能体之一，MathChat 作为基线方法可以凸显本章方法的进步。

2.ReAct: ReAct 是一种基于推理和行动结合的推理框架，它通过让模型生成一系列行动来解决问题。每个行动都可能改变问题的状态，而模型需要根据这些变化来决定下一步的行动。本章将给 ReAct 提供计算的工具，使其能够进行数学计算。

3. 静态错题样本库：这代表着使用预先构建的固定数量的错题样本库进行检索，并将检索结果作为额外上下文来辅助大语言模型解题。这个基线代表第三章的方法。将此方法与本章方法进行对比，可以体现持续学习的优势。

本章提出的方法通过与这些基线方法的比较，可以评估自学习智能体框架在数学文字问题求解方面的性能和优势。

4.4.2 预实验

4.4.2.1 多模型自一致性检验

表 4-2 单一模型与多模型自一致性对比

方法	伪答案生成准确率
单一模型	76.0%
多模型自一致性	84.0%

表4-2展示了单一模型与多模型自一致性检验在为 50 个自动生成的问题提供伪答案的准确率对比，结果经过人工校对。结果显示，多模型自一致性检验在生成可信的伪答案方面表现更为出色，实现了准确率提升。具体而言，多模型自一致性检验的伪答案生成准确率达到 84.0%，相比单一模型的 76.0% 提高了 8 个百分点。这一结果表明，多模型自一致性检验能够有效地整合不同模型的输出，从而提高为生成问题提供可信伪答案的能力。这种准确率的提升可以归因于多模型自一致性检验的几个关键优势：1. 多样性：不同的模型可能具有不同的知识背

景和推理方式，通过综合多个模型的输出，可以获得更全面和可信的伪答案。2. 一致性验证：当多个模型对同一伪答案达成一致时，这个答案的可信度会大大提高，有助于生成更高质量的问题-答案对。3. 错误过滤：当某个模型生成明显不合理的伪答案时，其他模型的输出可以起到过滤作用，提高整体伪答案的质量。4. 问题质量控制：通过比较不同模型生成的伪答案，可以识别出潜在的低质量或有歧义的问题，从而在源头上提高生成问题的质量。这种方法不仅提高了伪答案的生成质量，也有助于过滤掉潜在的不合理问题，从而提升整体样本库的质量。

4.4.3 实验结果

表 4-3 各方法在不同数据集上的准确率

Model & Method	MATH-minival	GSM8K
DeepSeek-V2-0628		
MathChat	73.0%	95.0%
ReAct	72.4%	94.6%
第三章方法 (4896 样本)	74.2%	95.1%
Our_Agent (4896 样本)	74.4%	95.1%
Our_Agent (10000 样本)	75.0%	95.2%
Our_Agent (20000 样本)	75.4%	95.2%
Our_Agent (30000 样本)	75.6%	95.3%
Our_Agent (40000 样本)	75.8%	95.3%
Our_Agent (50000 样本)	75.8% (+1.6%)	95.3% (+0.2%)
moonshot-v1-8k		
MathChat	50.8%	88.2%
ReAct	50.0%	87.6%
第三章方法 (4896 样本)	53.2%	88.5%
Our_Agent (4896 样本)	53.6%	88.6%
Our_Agent (10000 样本)	54.6%	88.8%
Our_Agent (20000 样本)	55.4%	88.9%
Our_Agent (30000 样本)	55.8%	89.0%
Our_Agent (40000 样本)	56.0%	89.0%
Our_Agent (50000 样本)	56.2% (+3.0%)	89.1% (+0.6%)
doubao-pro-32k		
MathChat	71.0%	94.0%
ReAct	70.2%	93.6%
第三章方法 (4896 样本)	72.2%	94.3%
Our_Agent (4896 样本)	72.4%	94.3%
Our_Agent (10000 样本)	72.8%	94.4%
Our_Agent (20000 样本)	73.2%	94.4%
Our_Agent (30000 样本)	73.2%	94.5%
Our_Agent (40000 样本)	73.4%	94.5%
Our_Agent (50000 样本)	73.4% (+1.2%)	94.5% (+0.2%)

根据表4-3的实验结果，可以得出以下分析：

1. 性能提升：本章提出的自学习智能体框架在所有测试的模型上都显示出了性能提升。特别是在样本数量从 4896 增加到 50000 时，在 MATH-minival 数据集上，DeepSeek-V2-0628、moonshot-v1-8k 和 doubao-pro-32k 模型分别获得了 1.6%、3.0% 和 1.2% 的准确率提升。这表明本章的方法能够有效利用增加的样本

来提高大语言模型解决数学文字问题的准确率。然而，可以观察到，当样本数量达到一定程度后（约 40000-50000 样本），性能提升开始趋于平缓。例如，对于 DeepSeek-V2-0628 模型，从 40000 到 50000 样本时，MATH-minival 的准确率保持不变。这种趋势表明，虽然增加样本数量通常能提高性能，但存在一个上限，超过这个上限后，继续增加样本可能不会带来显著的性能提升。这一现象可能是由于模型的能力达到了饱和，或者是由于样本多样性的限制，需要进一步研究来确定具体原因。

2. 不同数据集的表现：在 GSM8K 数据集上，性能提升相对较小(0.2%-0.6%)，这可能是因为模型在这个数据集上的初始性能已经很高，留给提升的空间较小。并且这个数据集更加简单，容易单次生成正确答案，纠错过程没有起到作用。相比之下，MATH-minival 数据集上的提升更为显著，这说明本章方法在更具挑战性的任务上可能更有效。

3. 模型间的差异：DeepSeek-V2-0628 模型在两个数据集上都表现出最好的性能，这可能因为其预训练阶段注重数学推理。moonshot-v1-8k 模型虽然整体性分数较低，但在样本增加时显示出了最大的提升（MATH-minival 上提升 3.0%），这表明本章的方法对于初始表现不好但是有一定基础能力的模型可以带来更显著的改进。

4. 方法比较：本章的智能体方法在所有情况下都优于 MathChat 和 ReAct 方法，即使是在最小的样本集（4896 样本）上。这证明了本章的错题样本库和多策略融合方法的有效性。

5. 第三章方法 vs 自学习智能体：本章的智能体方法在 4896 样本时就略微优于第三章方法，这说明多策略融合方法中从单次样本中错题样本的策略是有效的。随着样本数量的增加，性能持续提升。这说明错题样本库的动态更新策略能够有效利用增加的样本来提高模型性能。

总体而言，这些结果证实了本章的自学习智能体框架在提高数学问题解决能力方面的有效性，特别是在处理更复杂的问题时。同时，但随着样本数量的增加，性能提升的速度在逐渐放缓，并且最后趋于稳定，这表明这种方法存在极限，本章的目标就是尽可能探索这个最优结果。同时结果也显示了不同模型和数据集上的差异化表现，为进一步优化和应用本章的方法提供了有价值的洞察。

表4-4展示了本章的方法与第三章介绍的基线方法在不同模型和数据集上的性能对比。从结果可以看出，本章提出的方法在所有测试的模型和数据集上都取得了最佳性能，相比其他方法有明显提升。在基线方法中，MathChat 和 Complex CoT 方法表现较为出色，而 PAL 方法整体表现较弱。这些结果验证了本章方法的有效性，同时也为未来研究提供了新的思路。

表 4-4 与第三章基线的准确率对比

Model & Method	MATH-minival	GSM8K
DeepSeek-V2-0628		
Zero-Shot CoT	68.4%	91.8%
5-shot CoT	70.2%	93.6%
zero-shot pal	57.8%	89.2%
5-shot pal	61.4%	91.8%
Complex CoT	72.8%	94.8%
Least-to-Most Prompting	71.6%	94.2%
MathChat	73.0%	95.0%
ReAct	72.4%	94.6%
Our_Agent (50000 样本)	75.8%	95.3%
moonshot-v1-8k		
Zero-Shot CoT	45.6%	84.8%
5-shot CoT	47.8%	86.4%
zero-shot pal	38.2%	82.0%
5-shot pal	42.6%	84.2%
Complex CoT	50.4%	87.8%
Least-to-Most Prompting	49.0%	87.2%
MathChat	50.8%	88.2%
ReAct	50.0%	87.6%
Our_Agent (50000 样本)	56.2%	89.1%
doubao-pro-32k		
Zero-Shot CoT	66.2%	90.2%
5-shot CoT	68.8%	92.4%
zero-shot pal	44.2%	87.0%
5-shot pal	53.0%	89.0%
Complex CoT	70.4%	93.8%
Least-to-Most Prompting	69.6%	93.2%
MathChat	71.0%	94.0%
ReAct	70.2%	93.6%
Our_Agent (50000 样本)	73.4%	94.5%

4.4.3.1 消融实验

1. 去掉多模型自一致性验证在这个实验中，使用单一模型 (DeepSeek-V2-0628) 来进行答案的生成和验证，而不是使用多个模型进行自一致性验证。其他设置与完整的智能体方法保持一致，包括多策略融合方法和错题样本库。与预实验的区别在于进行了完整实验，在此基础上看到了多模型自一致性验证的提升。

2. 去掉多策略融合方法在这个实验中，去掉了执行模块中的多策略融合方法。具体来说，在初次解题和纠错阶段，本实验都只使用相似度匹配来选择样本，而不考虑错误类型分析。其他设置与完整的智能体方法保持一致，包括多模型自一致性验证和错题样本库。

3. 去掉记忆模块中的更新策略在这个实验中，去掉了记忆模块更新样本的策略。具体来说，本实验保留了所有解题路径，不再进行筛选或删除以确保多样性。这意味着记忆模块可能会积累大量相似的解题路径，而不是保持一个多样化的样本集。其他设置与完整的智能体方法保持一致，包括多模型自一致性验证和多策略融合方法。

本章通过这三个消融实验，可以分别评估多模型自一致性验证、多策略融合方法和记忆模块中的多样性保证对智能体性能的贡献。实验结果将在下一小节中展示和分析。

4.4.3.2 消融实验结果与分析

消融实验的结果如表4-5所示。

表 4-5 消融实验结果

方法	MATH-minival	GSM8K
完整智能体	75.8%	95.3%
去掉多模型自一致性验证	69.2%	88.3%
去掉多策略融合方法	66.2%	88.0%
去掉记忆模块更新策略	70.4%	92.4%

从表4-5的结果可以看出：去掉多模型自一致性验证后，模型在 MATH-minival 和 GSM8K 上的准确率分别下降了 6.6% 和 7.3%，表明多模型自一致性验证可以有效提高对问题的筛选作用，并且生成的伪答案可以作为评判问题是否有效的重要依据。

去掉多策略融合方法后，模型在 MATH-minival 和 GSM8K 上的准确率分别下降了 9.6% 和 7.3%，说明仅仅根据语义相似度在解题阶段选择样本是不够的，需要根据错误类型进一步筛选。这也再次说明了基于错误类型进行的检索是有益于解决数学文字问题的。

去掉记忆模块更新策略后，模型在 MATH-minival 和 GSM8K 上的准确率分别下降了 5.4% 和 3.0%，表明保持样本库的多样性是非常重要的，如果样本库中存在大量重复的样本，同一个问题检索出的样本很大概率相同，这对上下文学习来说是有损耗的。

总体而言，消融实验的结果验证了多模型自一致性验证、多策略融合方法和错题样本库三个创新点的有效性，它们共同促进了智能体在数学问题求解任务上的性能提升。

4.4.4 案例分析

为了更好地理解自学习智能体框架的工作原理和效果，本章选取了几个具有代表性的案例，分别从感知、决策、执行、学习和记忆等核心模块进行详细分析。这些案例展示了智能体如何利用其各个模块来解决复杂的数学文字问题。

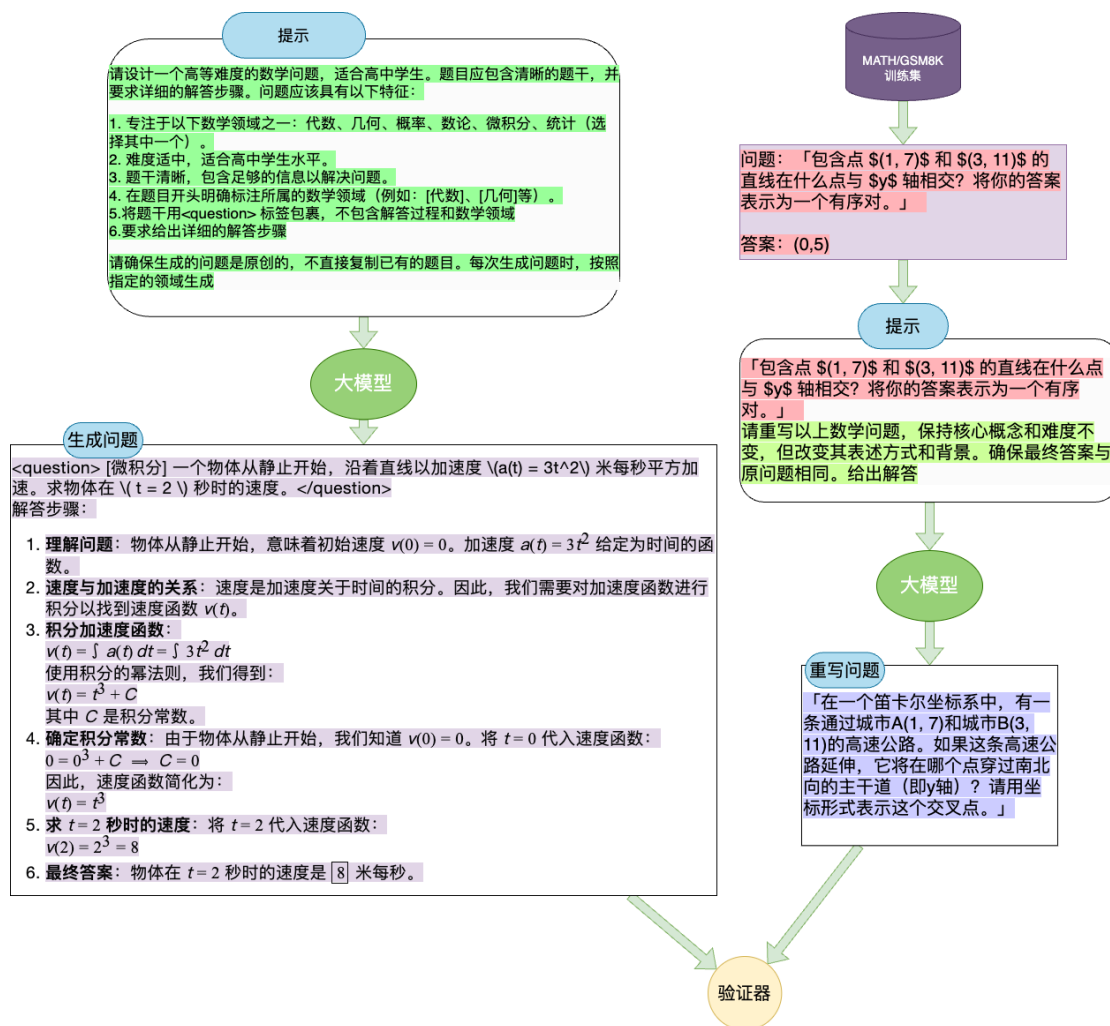


图 4-5 感知模块生成器案例

4.4.4.1 感知模块案例

如图4-5所示，有感知模块中的问题生成有两种形式，一是利用 MATH 和 GSM8K 训练集中的问题重写，二是利用大模型直接生成。通过这两种方法生成的问题送入验证器中验证。

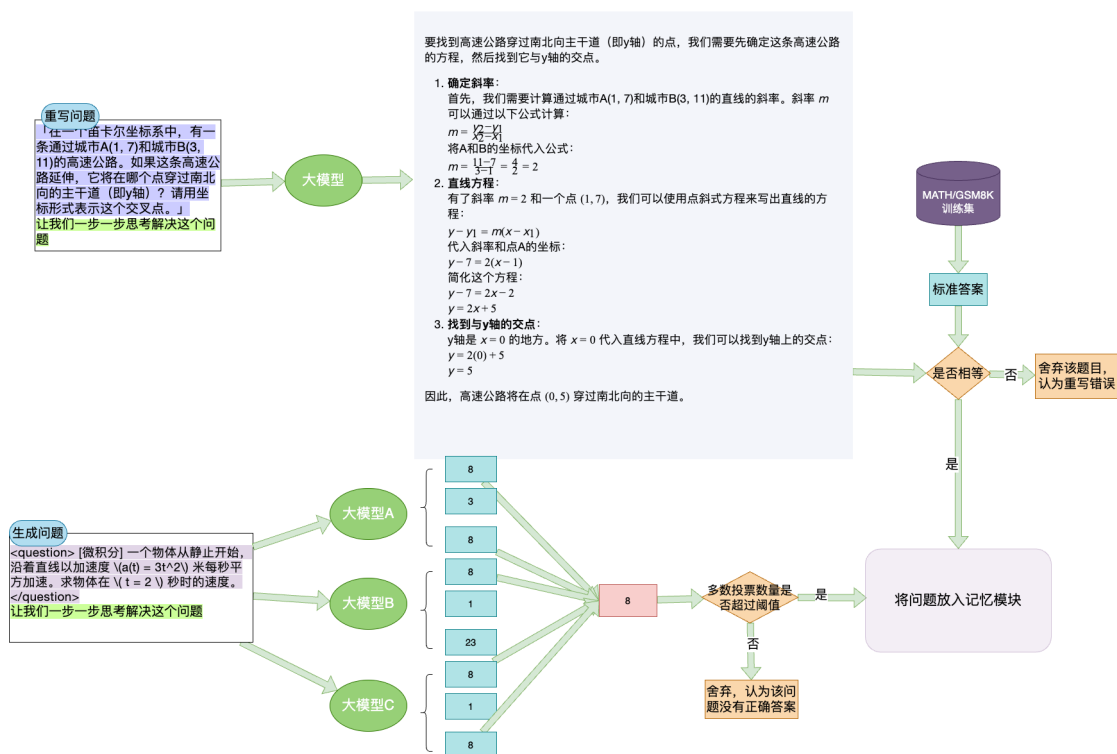


图 4-6 感知模块验证器案例

如图4-6所示，验证器模块中，本章使用了两种验证方法：

1. 针对重写的问题，本章方法直接用思维链让模型生成答案，再将生成的答案与数据库中的标准答案进行对比，如果相同则通过，否则不通过。

2. 针对生成的问题，本章方法使用多模型自一致性验证，即使用多个模型生成伪答案，然后比较这些答案的一致性。如果多个模型的答案高度一致，认为这个问题是有效的。这种方法可以有效地过滤掉不合理或模糊的问题，提高问题库的质量。

在实际应用中，本章方法使用了三个不同的大语言模型来生成答案，并设置了一个一致性阈值。只有三个模型生成的答案的数量大于阈值，才认为这个问题是有效的。

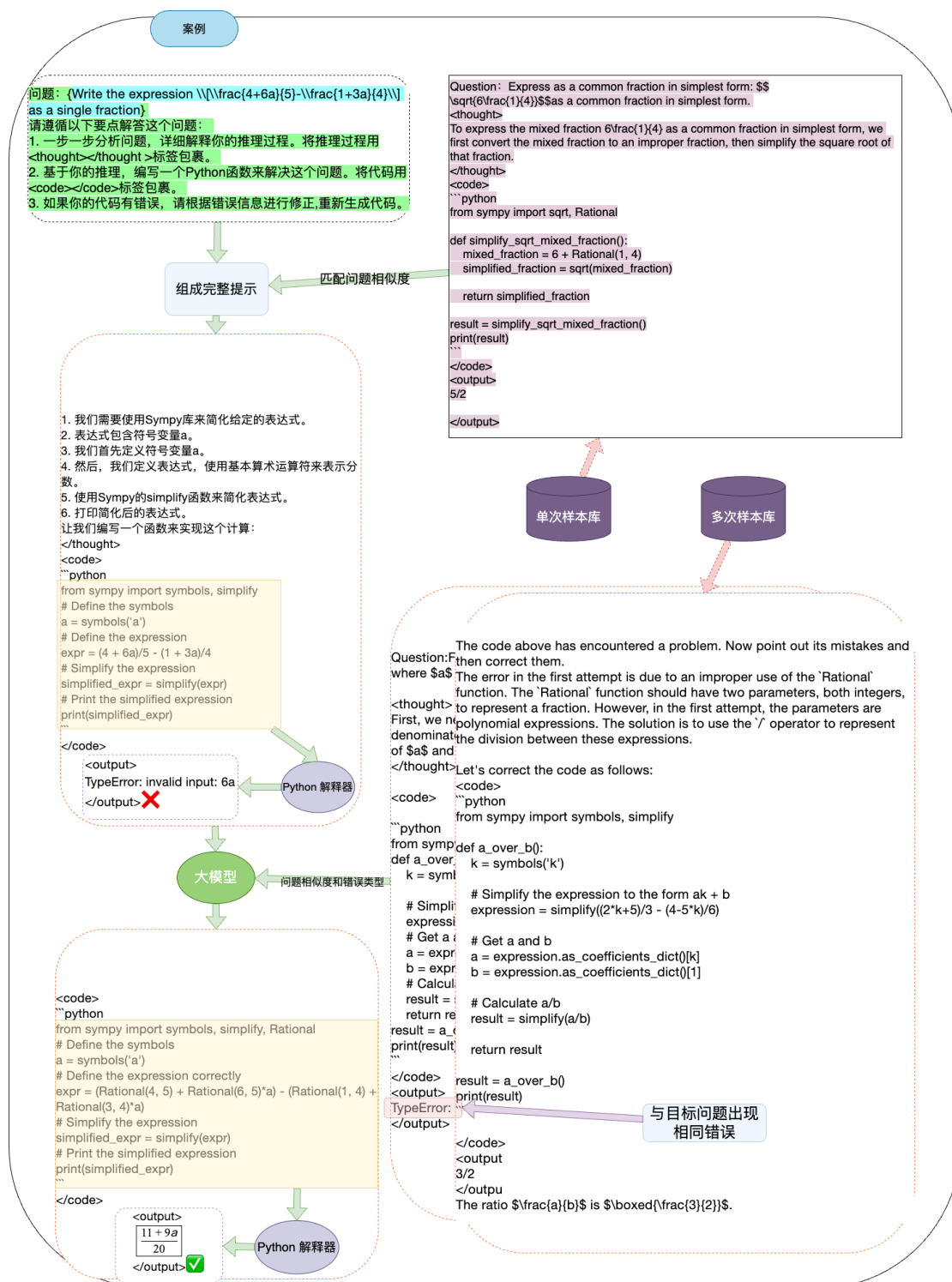


图 4-7 执行模块案例

4.4.4.2 执行模块案例

图4-7展示了执行模块的工作流程。在这个案例中，可以看到智能体如何处理一个具体的数学问题。

1. 初次解题阶段：智能体首先尝试解决问题。它使用相似度匹配从单次样本库中检索最相似的问题。在这个例子中，问题是一个代数化简问题，而匹配到的成功路径同样是一个代数化简的问题，这表明相似度匹配可以很好的找到与当前问题相似的样例。

2. 错误检测：然而，初次解答并不正确。生成的代码执行失败，Python 解释器返回的错误代码为 `TypeError`，这表明智能体在解题过程中遇到了困难。

3. 纠错阶段：进入纠错阶段后，从多次样本库中匹配，不仅考虑问题的相似度，且只保留相同的错误类型。在这个案例中，成功从多次样本库找了一个同样是代数化简，且错误类型是 `TypeError` 的样本。

4. 最终解答：利用新选择的样本，智能体生成了修正后的解答。这次解答正确地处理了代数化简问题，成功解决了问题。

这个案例展示了执行模块如何灵活运用多种策略，结合单次和多次样本库，逐步改进解题过程，最终得到正确答案。

4.4.4.3 记忆模块案例

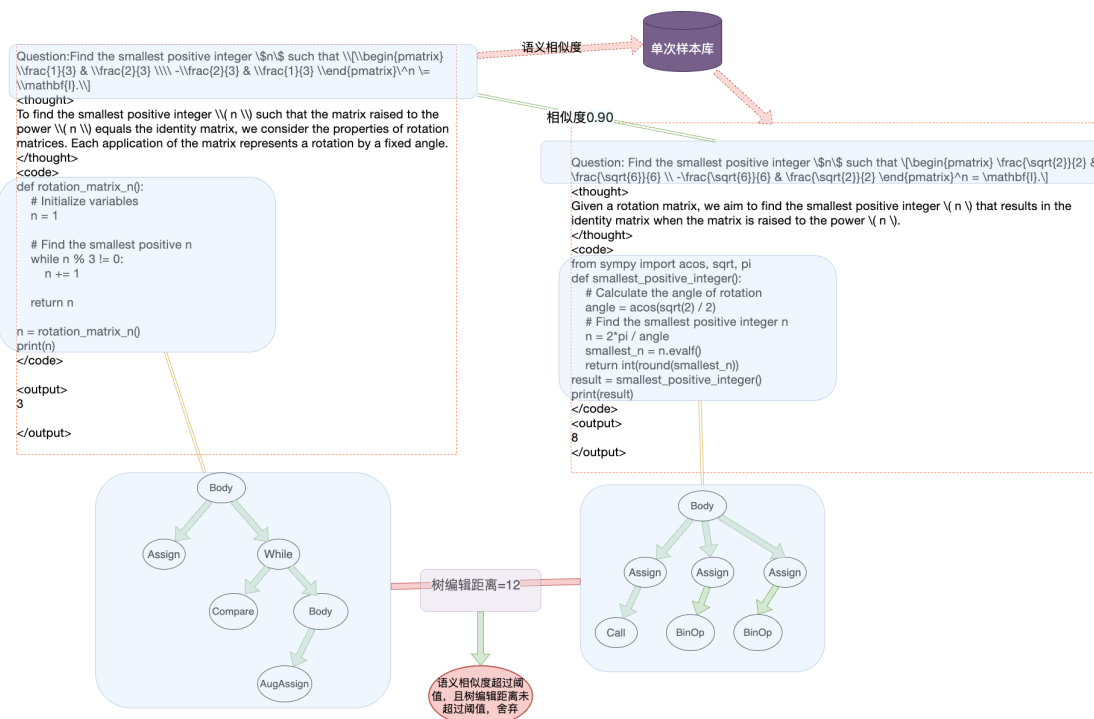


图 4-8 单次样本库更新策略

如图4-8所示，对于单次样本库，更新策略主要考虑两个方面：语义相似度和代码结构差异。当一个新样本到达时，首先计算其与现有样本的语义相似度。如果相似度低于阈值 ϵ_1 ，则直接接受该样本。如果相似度高于阈值，则进一步比较代码结构差异，主要是通过对比两个代码抽象语法树的最短编辑距离。只有当树编辑距离差异大于阈值 ϵ_2 时，才接受该样本。这种策略确保了样本库中问题的多样性，同时避免了重复或过于相似的样本。

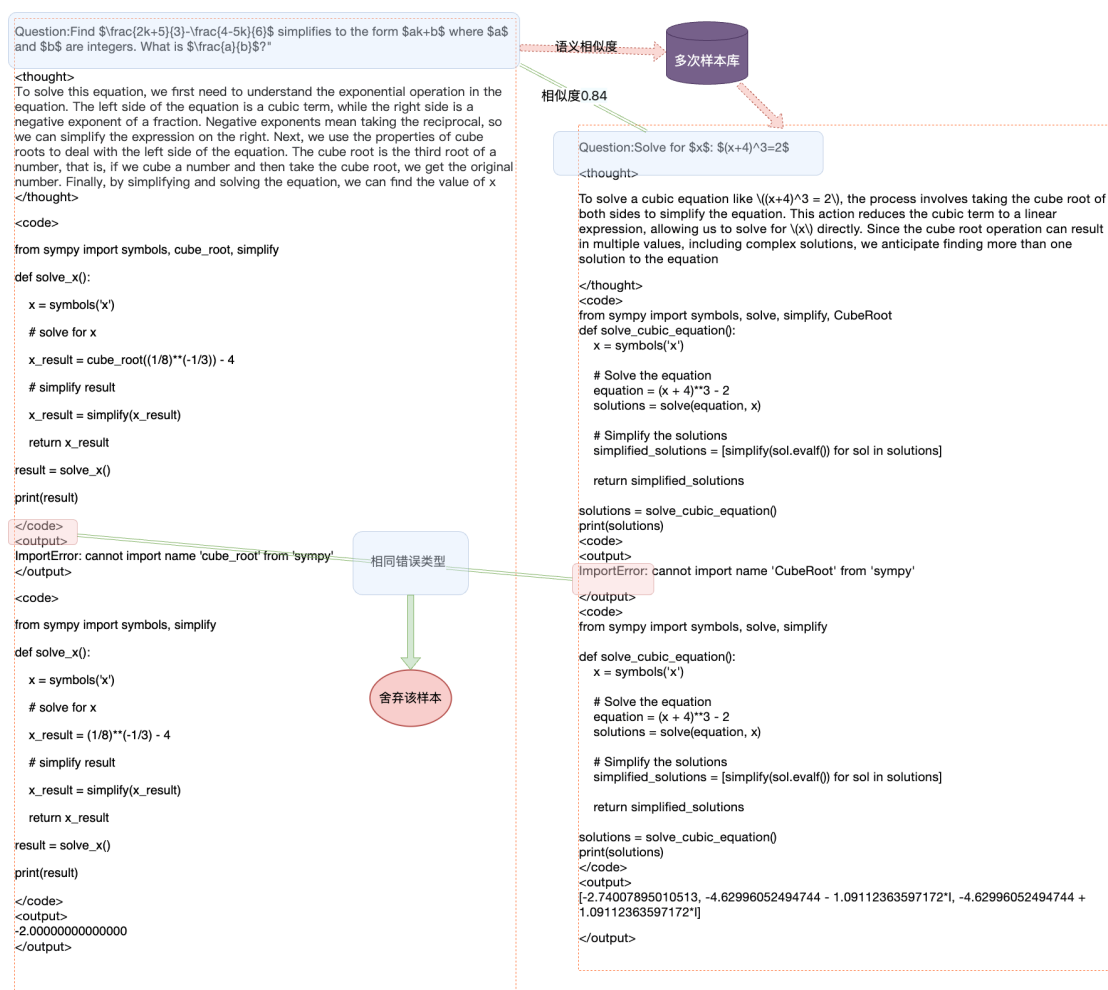


图 4-9 多次样本库更新案例

图4-9展示了多次样本库的更新策略示意图。

对于多次样本库，更新策略采用了稍有不同的方法。首先，系统仍会计算新样本与现有样本的语义相似度。如果这个相似度低于预设的阈值 ϵ_3 ，新样本将被直接接受并添加到库中。然而，若相似度高于这个阈值，系统会进一步比较错误类型。只有当新样本的错误类型与现有相似样本不同时，才会接受这个新样本。这种策略的理念在于，即使问题在语义上相似，但如果在解题过程中出现了不同类型的错误，这个样本仍然具有学习价值。记录这些不同类型的错误可以帮助模型在未来更全面地理解和处理各种可能出现的问题情况，从而不断提升其解题

能力。

两种更新策略都体现了智能体持续学习和自适应的特性，通过动态更新样本库，不断优化和扩展其知识基础。这种方法既保证了样本的多样性，又避免了冗余，从而提高了智能体的学习效率和问题解决能力。

4.5 讨论

4.5.1 自学习智能体框架的优势和局限性

本章通过实验和分析，可以总结出自学习智能体框架的主要优势：1. 持续学习能力：智能体能够不断从问题生成器中产生的新问题和解题经验中学习，持续提升性能。2. 知识积累和利用：通过错题样本库，智能体能够有效积累和利用过往经验。3. 多策略融合：灵活运用多种错题样本策略，提高了解决复杂问题的能力。

然而，该框架也存在一些局限性：1. 初始训练成本高：建立和优化智能体需要大量的接口费用和时间。因为问题生成器产生的问题很大概率是不可解的，所以需要大量的数据进行训练，与之而来的问题是样本库更新速度慢，需要大量的时间成本，才能看到少量的进步。2. 知识表示的局限：将数学知识有效地表示和组织仍然是一个复杂的问题，本文主要研究的是数学文字问题，表示的方式主要是文字，并且重点关注的是最终的答案，而不是解题过程。如何让大语言模型能够用更加形式化的方式处理问题，或者从解答步骤的正确性中受益，仍然是一个值得研究的问题。3. 存在上限：这更多的是对模型已有能力和知识的激发和利用，而不是真正意义上的自学习，因为模型的参数并没有得到更新。随着样本库的增加，性能的提升会逐渐放缓，当样本库足够大时，性能的提升会非常有限。

4.5.2 潜在的改进方向和扩展应用

基于当前研究的局限性，未来的研究可以从以下几个方面进行改进：

1. 提高初始训练效率：通过改进问题生成器，减少生成不可解问题的概率，从而减少初始训练数据的需求。具体来说可以通过一个训练好的生成模型产生更加准确和多样化的数学问题，而不是直接使用大语言模型生成。这种方式可以减少生成不可解问题的概率，从而加快样本库的积累速度。2. 增强知识表示能力：研究如何将数学知识以更形式化的方式表示和组织，以便模型能够更好地理解和利用这些知识。如通过 Lean^[85] 编程语言，将数学推理过程以编程的方式进行表示，并获得自动化的验证。或者像 lightman 等人^[77] 的工作，使用过程奖励而不是结果奖励，来提高模型在数学问题求解上的表现。3. 结合强化学习：将智

能体的学习过程与强化学习结合，通过试错的方式，提高智能体的创新能力。可以结合一些强化学习的微调方式，对模型的参数进行修改，从根本上提高模型解决问题的能力。强化学习的关键在于奖励函数的设计，而对于数学问题求解来说，无论是使用程序辅助的方式还是形式化推理，都可以较为容易的获得正确性的验证，前者可以使用 Python 解释器，后者可以使用 Lean 等编程语言，所以在理论上这种方式是可行的。虽然具有前景，但实现具有难度，因为基于模型生成数据所进行的强化学习和微调需要大量的探索和计算资源，并且可能出现模型崩溃^[86]的问题。

4.6 本章小结

本章提出了一个基于自学习智能体的数学问题求解框架，该框架通过模拟人类的学习过程，实现了持续学习和自适应的能力。本章详细介绍了框架的设计，包括感知、决策、执行、学习和记忆等核心模块，以及它们之间的协作机制。通过一系列实验和案例分析，本章验证了该框架在提升数学问题求解能力方面的有效性。结果表明，与第三章提出的方法相比，自学习智能体在长期性能、适应性和泛化能力等方面都表现出显著优势。尽管如此，本章也揭示了当前框架的一些局限性，如初始成本高、少样本学习能力有限。基于这些发现，本章提出了一系列潜在的改进方向和扩展应用，为未来的研究指明了道路。总体而言，本章的研究为提升大语言模型在数学文字问题求解领域的能力提供了一个新的视角和方法，也为人工智能在其他复杂推理任务中的应用提供了有价值的参考。

第5章 总结与展望

5.1 论文工作总结

数学文字问题通常以自然语言描述，包含了问题背景、已知条件和待求解的问题，这与大语言模型擅长处理的文本输入格式高度契合。此外，数学文字问题的难度通常介于简单的算术运算和高度抽象的数学证明之间，为大语言模型提供了一个理想的挑战水平。这种中等难度不仅能够测试模型的语言理解能力，还能评估其逻辑推理和问题解决能力，同时又不会过于复杂以至于完全超出模型的能力范围。因此，数学文字问题成为了评估和改进大语言模型认知能力的理想测试场景。本文围绕基于大语言模型和智能体解决数学文字问题求解开展研究，指出现有方法存在微调效率较低且成本较高、推理与计算的分离、反馈与纠错机制的缺失、检索策略的单一、高质量数据的局限性、记忆机制的局限性和智能体架构的局限性等挑战。针对上述挑战，本文分别进行了基于错误类型检索和多轮程序辅助纠错的数学文字问题求解研究和基于自学习大语言模型智能体的数学文字问题求解研究，以提高大语言模型对数学文字问题的求解准确率。

1. 基于错误类型检索和多轮程序辅助纠错的数学文字问题求解

本文提出了一种使用多轮对话进行错误修正，并利用提前建立的错题样本库进行基于检索的上下文学习从而提高大语言模型解决数学文字问题的方法。具体而言，通过上下文学习的方式，在样本中结合思维链和程序辅助提示，规范化大语言模型的输出，使其结合推理与计算。随后利用手动构造的少量固定样本，通过运行代码后返回的代码执行错误类型信息，反馈给大语言模型进行多轮修正，建立反馈和纠错机制。利用这种方式基于 MATH 数据集的训练集构造出错题样本库，并通过同时检索问题语义相似度和错误类型的方式解决数据与检索策略的局限性的问题。

相关实验在 MATH-minival 和 GSM8K 这两个数学文字问题数据集上展开，实验结果表明，本文提出的基于错误类型检索和多轮程序辅助纠错方法在数学文字问题的准确率上优于基线方法，在 DeepSeek-V2-0628, moonshot-v1-8k, doubao-pro-32k 这几个模型上都有所提升，而且在 MATH-minival 数据集上提升大于在 GSM8K 数据集的提升更大。这说明该方法具有泛化性，可广泛用于提升大语言模型解决数学文字问题的能力，并且在稍微复杂的问题上更具优势。

2. 基于自学习大语言模型智能体的数学文字问题求解

在上述方法基础上,为进一步解决智能体架构的局限性,形成更加系统化的解题策略,建立了自主选择解题策略的智能体架构,分为感知,执行,记忆三大模块。具体而言,为持续构造高质量数学文字问题,解决数据的局限性的挑战,感知模块基于原有错题样本库,利用大语言模型的生成能力持续构造数学文字问题,并利用多模型自一致性进行质量控制。执行模块使用策略选择机制,根据对话阶段的不同,灵活选择不同的检索策略。记忆模块通过构建单次样本库和多次样本库,以及相应的记忆更新策略,通过语义相似度、代码结构差异和错误类型等多维比较,有效维护记忆库的多样性和准确性,实现”错题-分析-修正-强化”的闭环学习,使得智能体的记忆机制的局限性得到缓解。

相关实验在 MATH-minival 和 GSM8K 这两个数学文字问题数据集上展开,实验结果表明,本文提出的自学习大语言模型智能体可以用感知,执行,记忆三大模块,较好的完成持续构造高质量数学文字问题,维护记忆库的多样性和准确性,并且根据对话阶段的不同有效选择检索策略。从而缓解大语言模型智能体解决数学文字问题面临的高质量数据缺乏、架构局限性和记忆机制局限性等挑战。

5.2 未来研究展望

基于本论文的研究成果和当前人工智能在数学问题求解领域的发展趋势,本文对未来的研究方向提出以下展望:

(1) 增强模型的元学习和迁移学习能力:研究如何提升模型在面对新类型数学问题时的快速适应能力。探索跨领域知识迁移的方法,使模型能够将在一个数学分支学到的知识应用到其他相关领域。开发更高效的少样本学习技术,减少模型在新任务上的学习成本。

(2) 多模态数学问题求解:扩展当前的文本基础模型,使其能够理解和处理包含图表、公式和几何图形的多模态数学问题。研究如何有效地融合不同模态的信息,以提高复杂数学问题的解决能力。

(3) 将历史解题思路融入模型:改变模型的参数,达到自我学习的目的,而不只依赖外部样本。

这些研究方向不仅有望进一步提升 AI 在数学问题求解领域的能力,还可能为人工智能的整体发展带来新的突破。未来的 AI 可以突破数据的限制,通过自我学习,达到高效解决数学文字问题的阶段,乃至解决更复杂的实际科学问题。

参考文献

- [1] Wu T, He S, Liu J, et al. A brief overview of ChatGPT: The history, status quo and potential future development [J]. IEEE/CAA Journal of Automatica Sinica, 2023, 10 (5): 1122–1136.
- [2] Achiam O J, Adler S, Agarwal S, et al. GPT-4 Technical Report [C]. 2023.
- [3] Bubeck S, Chandrasekaran V, Eldan R, et al. Sparks of artificial general intelligence: Early experiments with gpt-4 [J]. arXiv preprint arXiv:2303.12712, 2023.
- [4] Li J, Cheng X, Zhao X, et al. HaluEval: A Large-Scale Hallucination Evaluation Benchmark for Large Language Models [C] // Bouamor H, Pino J, Bali K. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, December 2023: 6449–6464.
- [5] Aleven V, Baraniuk R, Brunskill E, et al. Towards the Future of AI-Augmented Human Tutoring in Math Learning [C]. In International Conference on Artificial Intelligence in Education, 2023: 26–31.
- [6] Gaur V, Saunshi N. Reasoning in Large Language Models Through Symbolic Math Word Problems [C] // Rogers A, Boyd-Graber J, Okazaki N. In Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 2023: 5889–5903.
- [7] Imani S, Shrivastava H, Du L. Mathematical reasoning using large language models. September 5 2024. US Patent App. 18/144,802.
- [8] Ahn J, Verma R, Lou R, et al. Large Language Models for Mathematical Reasoning: Progresses and Challenges [C]. St. Julian's, Malta, March 2024: 225–237.
- [9] Xu X, Xiao T, Chao Z, et al. Can LLMs Solve Longer Math Word Problems Better? [C]. In The Thirteenth International Conference on Learning Representations, 2025.
- [10] Ling W, Yogatama D, Dyer C, et al. Program Induction by Rationale Generation: Learning to Solve and Explain Algebraic Word Problems [C] // Barzilay R, Kan M-Y. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Vancouver, Canada, July 2017: 158–167.
- [11] Lu P, Gong R, Jiang S, et al. Inter-GPS: Interpretable Geometry Problem Solving with Formal Language and Symbolic Reasoning [C] // Zong C, Xia F, Li W, et al. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers), Online, August 2021: 6774–6786.

- [12] Chen L, Li B, Shen S, et al. Large language models are visual reasoning coordinators [J]. *Advances in Neural Information Processing Systems*, 2024, 36.
- [13] Koncel-Kedziorski R, Roy S, Amini A, et al. MAWPS: A math word problem repository [C]. In *Proceedings of the 2016 conference of the north american chapter of the association for computational linguistics: human language technologies*, 2016: 1152–1157.
- [14] Cheng V, Yu Z. Analyzing ChatGPT’s Mathematical Deficiencies: Insights and Contributions [C] // Wu J-L, Su M-H. In *Proceedings of the 35th Conference on Computational Linguistics and Speech Processing (ROCLING 2023)*, Taipei City, Taiwan, October 2023: 188–193.
- [15] Wei J, Wang X, Schuurmans D, et al. Chain-of-thought prompting elicits reasoning in large language models [J]. *Advances in neural information processing systems*, 2022, 35: 24824–24837.
- [16] Schulhoff S, Ilie M, Balepur N, et al. The Prompt Report: A Systematic Survey of Prompting Techniques [J]. *CoRR*, 2024, abs/2406.06608.
- [17] Zhou D, Schärli N, Hou L, et al. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models [C]. In *The Eleventh International Conference on Learning Representations*, 2023.
- [18] Fu Y, Peng H, Sabharwal A, et al. Complexity-based prompting for multi-step reasoning [C]. In *The Eleventh International Conference on Learning Representations*, 2022.
- [19] Wang X, Wei J, Schuurmans D, et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models [C]. In *The Eleventh International Conference on Learning Representations*, 2023.
- [20] Gao L, Madaan A, Zhou S, et al. Pal: Program-aided language models [C]. In *International Conference on Machine Learning*, 2023: 10764–10799.
- [21] Das D, Banerjee D, Aditya S, et al. MATHSENSEI: A Tool-Augmented Large Language Model for Mathematical Reasoning [C] // Duh K, Gomez H, Bethard S. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Mexico City, Mexico, June 2024: 942–966.
- [22] Gou Z, Shao Z, Gong Y, et al. ToRA: A Tool-Integrated Reasoning Agent for Mathematical Problem Solving [C]. In *The Twelfth International Conference on Learning Representations*, 2024.
- [23] Qin J, Liang X, Hong Y, et al. Neural-Symbolic Solver for Math Word Problems with Auxiliary Tasks [C] // Zong C, Xia F, Li W, et al. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Online, August 2021: 5870–5881.

- [24] Hendrycks D, Burns C, Kadavath S, et al. Measuring Mathematical Problem Solving With the MATH Dataset [C]. In Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2), 2021.
- [25] Shao Z, Wang P, Zhu Q, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models [J]. arXiv preprint arXiv:2402.03300, 2024.
- [26] Luo H, Sun Q, Xu C, et al. WizardMath: Empowering Mathematical Reasoning for Large Language Models via Reinforced Evol-Instruct [C]. In The Thirteenth International Conference on Learning Representations, 2025.
- [27] Ouyang L, Wu J, Jiang X, et al. Training language models to follow instructions with human feedback [J]. Advances in neural information processing systems, 2022, 35: 27730–27744.
- [28] Levy I, Bogin B, Berant J. Diverse Demonstrations Improve In-context Compositional Generalization [C] // Rogers A, Boyd-Graber J, Okazaki N. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Toronto, Canada, July 2023: 1401–1422.
- [29] Weng L. LLM-powered Autonomous Agents [J]. lilianweng.github.io, 2023.
- [30] Nye M, Andreassen A J, Gur-Ari G, et al. Show Your Work: Scratchpads for Intermediate Computation with Language Models [J]. CoRR, 2021, abs/2112.00114.
- [31] Kojima T, Gu S S, Reid M, et al. Large language models are zero-shot reasoners [J]. Advances in neural information processing systems, 2022, 35: 22199–22213.
- [32] Yang C, Wang X, Lu Y, et al. Large Language Models as Optimizers [C]. In The Twelfth International Conference on Learning Representations, 2024.
- [33] Long J. Large Language Model Guided Tree-of-Thought. May 2023. arXiv:2305.08291 [cs].
- [34] Besta M, Blach N, Kubicek A, et al. Graph of Thoughts: Solving Elaborate Problems with Large Language Models [J]. Proceedings of the AAAI Conference on Artificial Intelligence, 2024, 38 (16): 17682–17690. Number: 16.
- [35] Vinyals O, Fortunato M, Jaitly N. Pointer networks [J]. Advances in neural information processing systems, 2015, 28.
- [36] Chen W, Ma X, Wang X, et al. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks [J]. Transactions on Machine Learning Research, 2023.
- [37] Yamauchi R, Sonoda S, Sannai A, et al. LPML: LLM-Prompting Markup Language for Mathematical Reasoning. October 2023. arXiv:2309.13078 [cs].
- [38] Evans R, Grefenstette E. Learning explanatory rules from noisy data [J]. Journal of Artificial Intelligence Research, 2018, 61: 1–64.
- [39] He-Yueya J, Poesia G, Wang R E, et al. Solving Math Word Problems by Combining Language Models With Symbolic Solvers. April 2023. arXiv:2304.09102 [cs].

- [40] Wu Y, Jia F, Zhang S, et al. MathChat: Converse to Tackle Challenging Math Problems with LLM Agents [C]. In ICLR 2024 Workshop on Large Language Model (LLM) Agents, 2024.
- [41] Brown T, Mann B, Ryder N, et al. Language Models are Few-Shot Learners [C]. In Advances in Neural Information Processing Systems, 2020: 1877–1901.
- [42] Song Y, Wang T-Y, Cai P, et al. A Comprehensive Survey of Few-shot Learning: Evolution, Applications, Challenges, and Opportunities [J]. ACM Computing Surveys, 2022, 55: 1 – 40.
- [43] Saravia E. Prompt Engineering Guide [J]. <https://github.com/dair-ai/Prompt-Engineering-Guide>, 2022.
- [44] Rombach R, Blattmann A, Lorenz D, et al. High-Resolution Image Synthesis With Latent Diffusion Models [C]. 2022: 10684–10695.
- [45] Sahoo P, Singh A K, Saha S, et al. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications [J]. CoRR, 2024, abs/2402.07927.
- [46] Liu P, Yuan W, Fu J, et al. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing [J]. ACM Computing Surveys, 2023, 55 (9): 1–35.
- [47] Laskar M T R, Bari M S, Rahman M, et al. A Systematic Study and Comprehensive Evaluation of ChatGPT on Benchmark Datasets [J]. CoRR, 2023, abs/2305.18486.
- [48] Akyürek E, Schuurmans D, Andreas J, et al. What learning algorithm is in-context learning? Investigations with linear models [C]. In The Eleventh International Conference on Learning Representations, 2023.
- [49] Chan S, Santoro A, Lampinen A, et al. Data distributional properties drive emergent in-context learning in transformers [J]. Advances in Neural Information Processing Systems, 2022, 35: 18878–18891.
- [50] Dong Q, Li L, Dai D, et al. A Survey on In-context Learning [C] // Al-Onaizan Y, Bansal M, Chen Y-N. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Miami, Florida, USA, November 2024: 1107–1128.
- [51] Xie S M, Raghunathan A, Liang P, et al. An Explanation of In-context Learning as Implicit Bayesian Inference [J]. CoRR, 2021, abs/2111.02080.
- [52] Min S, Lewis M, Zettlemoyer L, et al. MetaICL: Learning to Learn In Context [C] // Carpuat M, de Marneffe M-C, Meza Ruiz I V. In Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Seattle, United States, July 2022: 2791–2809.

- [53] Rubin O, Herzig J, Berant J. Learning To Retrieve Prompts for In-Context Learning [C] // Carpuat M, de Marneffe M-C, Meza Ruiz I V. In Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Seattle, United States, July 2022: 2655–2671.
- [54] Min S, Lyu X, Holtzman A, et al. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work? [C] // Goldberg Y, Kozareva Z, Zhang Y. In Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, Abu Dhabi, United Arab Emirates, December 2022: 11048–11064.
- [55] Olsson C, Elhage N, Nanda N, et al. In-context Learning and Induction Heads [J]. Transformer Circuits Thread, 2022. <https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html>.
- [56] Holtzman A, Buys J, Du L, et al. The Curious Case of Neural Text Degeneration [C]. In 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020, 2020.
- [57] Dathathri S, Madotto A, Lan J, et al. Plug and Play Language Models: A Simple Approach to Controlled Text Generation [C]. In International Conference on Learning Representations, 2020.
- [58] Keysers D, Schärli N, Scales N, et al. Measuring Compositional Generalization: A Comprehensive Method on Realistic Data [C]. In 8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020, 2020.
- [59] Huang J, Chang K C-C. Towards Reasoning in Large Language Models: A Survey [C] // Rogers A, Boyd-Graber J, Okazaki N. In Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 2023: 1049–1065.
- [60] Guo T, Chen X, Wang Y, et al. Large Language Model Based Multi-agents: A Survey of Progress and Challenges [C] // Larson K. In Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24, 8 2024: 8048–8057. Survey Track.
- [61] Mialon G, Dessi R, Lomeli M, et al. Augmented Language Models: a Survey [J]. Transactions on Machine Learning Research, 2023. Survey Certification.
- [62] Bang Y, Cahyawijaya S, Lee N, et al. A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity [C] // Park J C, Arase Y, Hu B, et al. In Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), Nusa Dua, Bali, November 2023: 675–718.

- [63] Petroni F, Rocktäschel T, Riedel S, et al. Language Models as Knowledge Bases? [C] // Inui K, Jiang J, Ng V, et al. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, November 2019: 2463–2473.
- [64] Yao S, Zhao J, Yu D, et al. ReAct: Synergizing Reasoning and Acting in Language Models [C]. In International Conference on Learning Representations (ICLR), 2023.
- [65] Lu J, Clark C, Zellers R, et al. UNIFIED-IO: A Unified Model for Vision, Language, and Multi-modal Tasks [C]. In The Eleventh International Conference on Learning Representations, 2023.
- [66] Huang R, Li M, Yang D, et al. AudioGPT: Understanding and Generating Speech, Music, Sound, and Talking Head [J]. CoRR, 2023, abs/2304.12995.
- [67] McCoy R T, Smolensky P, Linzen T, et al. How Much Do Language Models Copy From Their Training Data? Evaluating Linguistic Novelty in Text Generation Using RAVEN [J]. Transactions of the Association for Computational Linguistics, 2023, 11: 652–670.
- [68] Wang J, Yi X, Guo R, et al. Milvus: A Purpose-Built Vector Data Management System [C]. In Proceedings of the 2021 International Conference on Management of Data, Virtual Event China, June 2021: 2614–2627.
- [69] Lin J, Fried D, Klein D, et al. Inferring Rewards from Language in Context [C] // Muresan S, Nakov P, Villavicencio A. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Dublin, Ireland, May 2022: 8546–8560.
- [70] Solving Challenging Math Word Problems Using GPT-4 Code Interpreter with Code-based Self-Verification [M]. International Conference on Learning Representations, ICLR, 2024-05.
- [71] Li C, Yuan Z, Yuan H, et al. MuggleMath: Assessing the Impact of Query and Response Augmentation on Math Reasoning [C] // Ku L-W, Martins A, Srikumar V. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Bangkok, Thailand, August 2024: 10230–10258.
- [72] Anonymous. In-context Learning with Retrieved Demonstrations for Language Models: A Survey [J]. Submitted to Transactions on Machine Learning Research, 2024. Under review.
- [73] Li B, Zhang Y, Chen L, et al. MIMIC-IT: Multi-Modal In-Context Instruction Tuning. June 2023. arXiv:2306.05425 [cs].
- [74] Yu W, Iter D, Wang S, et al. Generate rather than Retrieve: Large Language Models are Strong Context Generators [C]. In The Eleventh International Conference on Learning Representations, 2023.

- [75] Wang L, Li L, Dai D, et al. Label Words are Anchors: An Information Flow Perspective for Understanding In-Context Learning [C] // Bouamor H, Pino J, Bali K. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, December 2023: 9840–9855.
- [76] Reimers N, Gurevych I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks [C]. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China, 2019: 3980–3990.
- [77] Lightman H, Kosaraju V, Burda Y, et al. Let’s Verify Step by Step [C]. In The Twelfth International Conference on Learning Representations, 2024.
- [78] Xi Z, Chen W, Guo X, et al. The Rise and Potential of Large Language Model Based Agents: A Survey [J]. CoRR, 2023, abs/2309.07864.
- [79] Ghosh S, Evuru C K R, Kumar S, et al. A Closer Look at the Limitations of Instruction Tuning [J].
- [80] Wu Q, Bansal G, Zhang J, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation [J]. arXiv preprint arXiv:2308.08155, 2023.
- [81] Zhang J, Wang X, Zhang H, et al. A novel neural source code representation based on abstract syntax tree [C]. In 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), 2019: 783–794.
- [82] Zhang J, Wang X, Zhang H, et al. TreeCaps: Tree-based capsule networks for source code processing [C]. In Proceedings of the 34th AAAI Conference on Artificial Intelligence, 2020: 9405–9412.
- [83] Jiang M, Liu K Z, Zhong M, et al. Investigating Data Contamination for Pre-training Language Models. January 2024.
- [84] Jiang M, Liu K, Zhong M, et al. Does Data Contamination Make a Difference? Insights from Intentionally Contaminating Pre-training Data For Language Models [C]. May 2024.
- [85] de Moura L, Kong S, Avigad J, et al. The Lean theorem prover (system description) [C]. In International Conference on Automated Deduction, 2015: 378–388.
- [86] Shumailov I, Shumaylov Z, Zhao Y, et al. AI models collapse when trained on recursively generated data [J]. Nature, 2024, 631 (8022): 755–759. Publisher: Nature Publishing Group.

发表论文和参加科研情况说明

(一) 申请及已获得的专利

- [1] 王博, 胡涵清. 用于大语言模型数学问题求解的检索样本选择与多轮纠错方法: 中国, 202411616946.0[P]. 2024-11-13.

(二) 参与的科研项目

- [1] 2024-2024 领域知识认知增强的大模型智能体技术研发与应用, 中国移动通信集团天津有限公司, 企业合作
- [2] 2024-2027 融合语言和认知图谱的心理内隐态度自动化分析与智能对话应用, 国家自然科学基金面上项目

致 谢

时光飞逝，转眼间研究生生涯即将结束。在这里，我要向所有在学习和科研道路上给予我帮助和支持的老师、同学和朋友们表达最诚挚的感谢。

首先要特别感谢我的导师王博老师。我就像一个参数不大的大模型，很多时候需要王老师的提示工程和微调才能找到正确的路。遇到问题时，王老师总是及时的提供建议和帮助，关心我们的心理健康。很幸运能有王老师这样一位导师！

感谢实验室的兄弟姐妹在学习和生活上给予我的帮助和建议。他们丰富的经验和宝贵的建议让我少走了很多弯路。感谢同窗好友们的陪伴与支持，让我的研究生生活充满欢乐与温暖。

最后，要特别感谢我的父母和家人。感谢他们多年来对我经济和精神上的支持。他们的关爱和鼓励是我前进的动力。

衷心感谢所有关心、帮助过我的人，正是有了你们的支持，才有了今天的收获。